

CNS

UNIT II

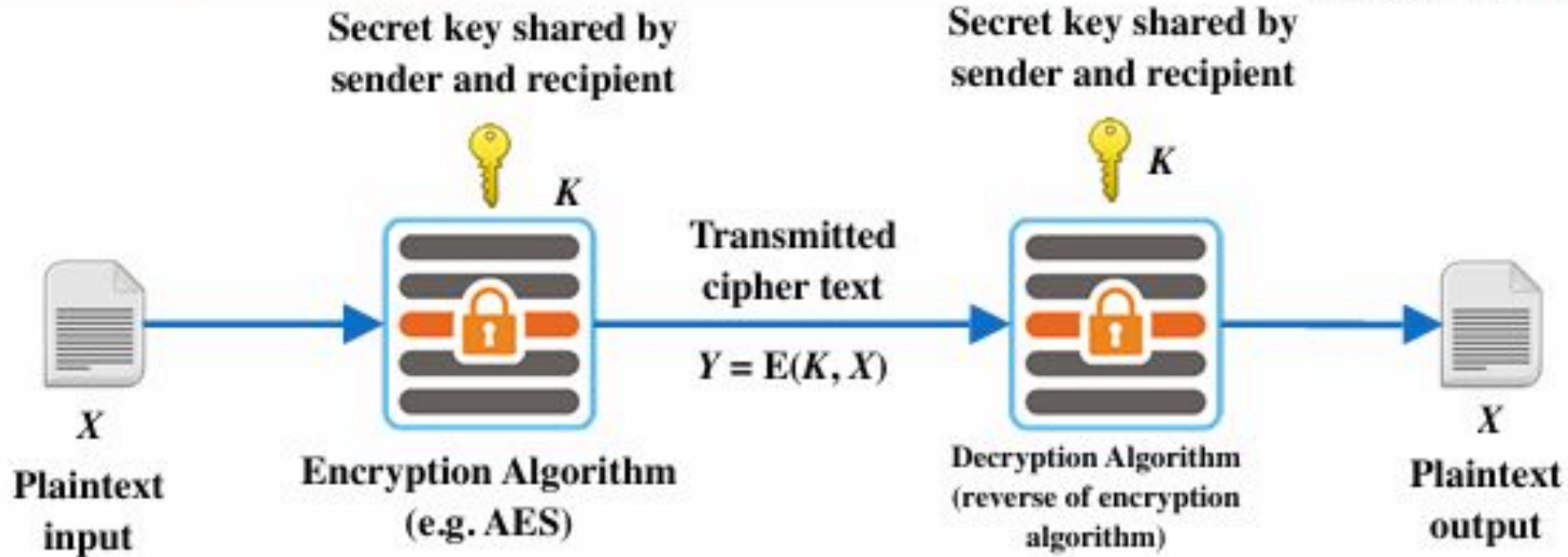
SYLLABUS

UNIT - II

Symmetric key Ciphers: Block Cipher principles, DES, AES, RC5, IDEA, Block cipher operation, Stream ciphers, RC4.

Asymmetric key Ciphers: Principles of public key cryptosystems, RSA algorithm, Elgamal Cryptography, Diffie-Hellman Key Exchange.

Symmetric Key Encryption



- Safe with a strong lock, only Alice and Bob have a copy of the key
 - Alice encrypts: locks message in the safe with her key
 - Bob decrypts: uses his copy of the key to open the safe



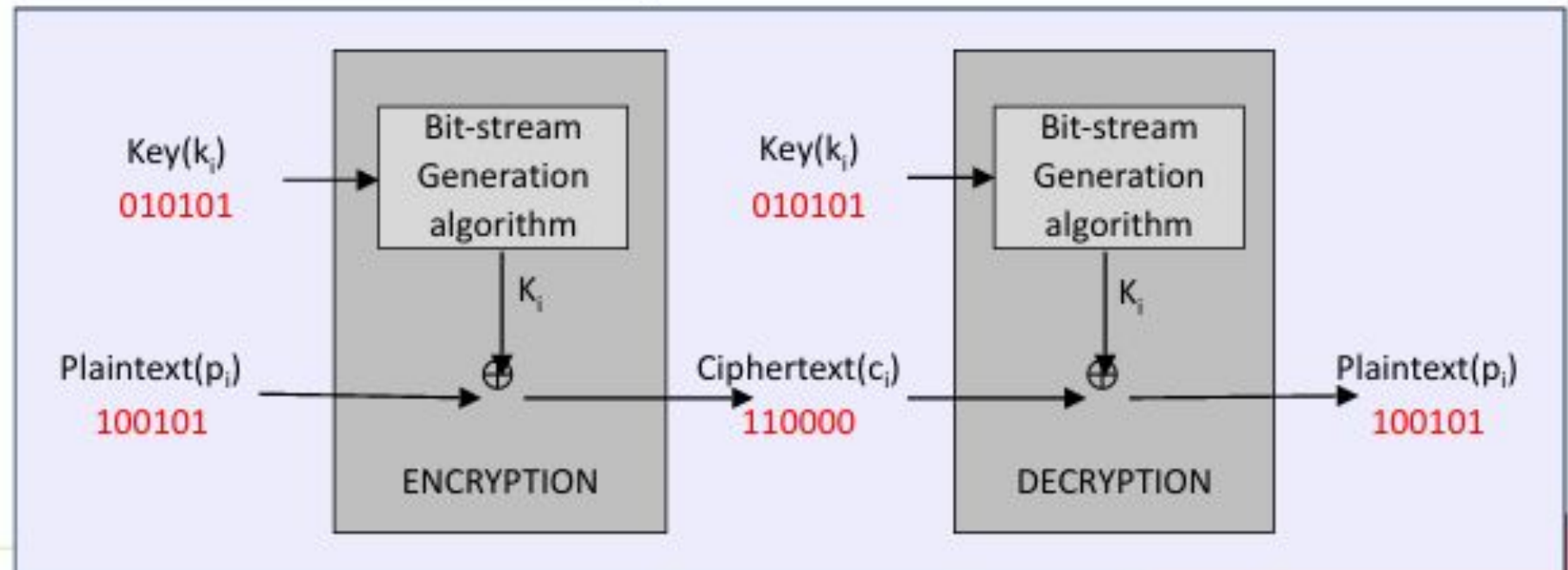
Symmetric Key Encryption example

Stream Cipher

A stream cipher is one that encrypts a digital data stream **one bit or one byte at a time**.

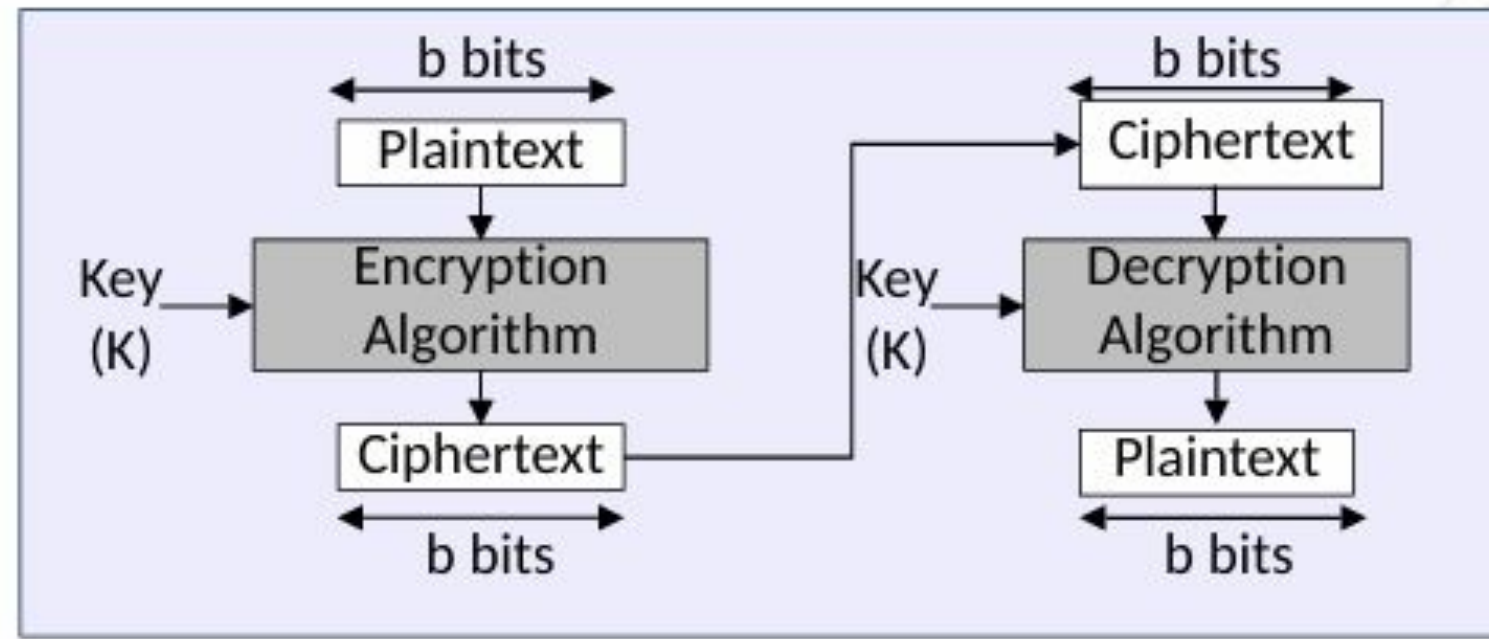
Examples:

1. Autokeyed Vigenère cipher
2. A5/1
3. RC4
4. Vernam cipher.



Block Cipher

- A block cipher is one in which a **block of plaintext** is treated as a whole and used to produce a **ciphertext block** of equal length.
- Typically, a block size of **64 or 128** bits is used.
- **Examples:**
 - Feistel cipher
 - DES
 - Triple DES
 - AES



Block Cipher Design Principles

- **Block ciphers** are built in the Feistel cipher structure. Block cipher has a specific number of rounds and keys for generating ciphertext. Block cipher is a type of encryption algorithm that processes fixed-size blocks of data, usually 64 or 128 bits, to produce ciphertext. The design of a block cipher involves several important principles to ensure the security and efficiency of the algorithm. Some of these principles are:
 1. **Number of Rounds** – The number of Rounds is regularly considered in design criteria, it just reflects the number of rounds to be suitable for an algorithm to make it more complex, in DES we have 16 rounds ensuring it to be more secure while in AES we have 10 rounds which makes it more secure.
 2. **Design of function F** – The core part of the **Feistel Block cipher structure is the Round Function**. The complexity of cryptanalysis can be derived from the Round function i.e. the increasing level of complexity for the round function would be greatly contributing to an increase in complexity. To increase the complexity of the round function, the **avalanche effect** is also included in the round function, as the change of a single bit in plain text would produce a mischievous output due to the presence of avalanche effect.
 3. **Confusion and Diffusion**: The cipher should provide confusion and diffusion to make it difficult for an attacker to determine the relationship between the plaintext and ciphertext. **Confusion means that the ciphertext should be a complex function of the key** and plaintext, making it difficult to guess the key. Diffusion means that a small change in the plaintext should cause a significant change in the ciphertext, which makes it difficult to analyze the encryption pattern.
 4. **Key Size**: The key size should be large enough to prevent brute-force attacks. A larger key size means that there are more possible keys, making it harder for an attacker to guess the correct one. A key size of 128 bits is considered to be secure for most applications.
 5. **Key Schedule**: The key schedule should be designed carefully to ensure that the keys used for encryption are independent and unpredictable. The key schedule should also resist attacks that exploit weak keys or key-dependent properties of the cipher.
 6. **Block Size**: The block size should be large enough to prevent attacks that exploit statistical patterns in the plaintext. A block size of 128 bits is generally considered to be secure for most applications.
 7. **Non-linearity**: The S-box used in the cipher should be **non-linear to provide confusion**. A linear S-box is vulnerable to attacks that exploit the linear properties of the cipher.
 8. **Avalanche Effect**: The cipher should exhibit the avalanche effect, which means that a small change in the plaintext or key should cause a significant change in the ciphertext. This ensures that any change in the input results in a complete change in the output.
 9. **Security Analysis**: The cipher should be analyzed for its security against various attacks such as differential cryptanalysis, linear cryptanalysis, and brute-force attacks. The cipher should also be tested for its resistance to implementation attacks, such as side-channel attacks.
- Overall, a good block cipher design should be resistant to various attacks, efficient, and easy to implement.

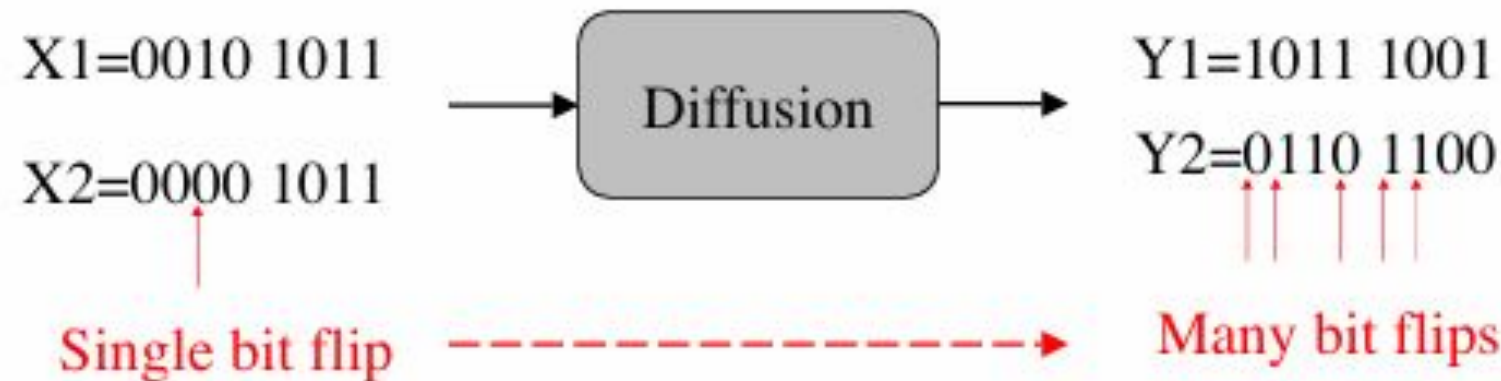
Diffusion and Confusion

Confusion

- Confusion **hides the relationship** between the **ciphertext and the key**.
- This is achieved by the use of a complex **substitution algorithm**.

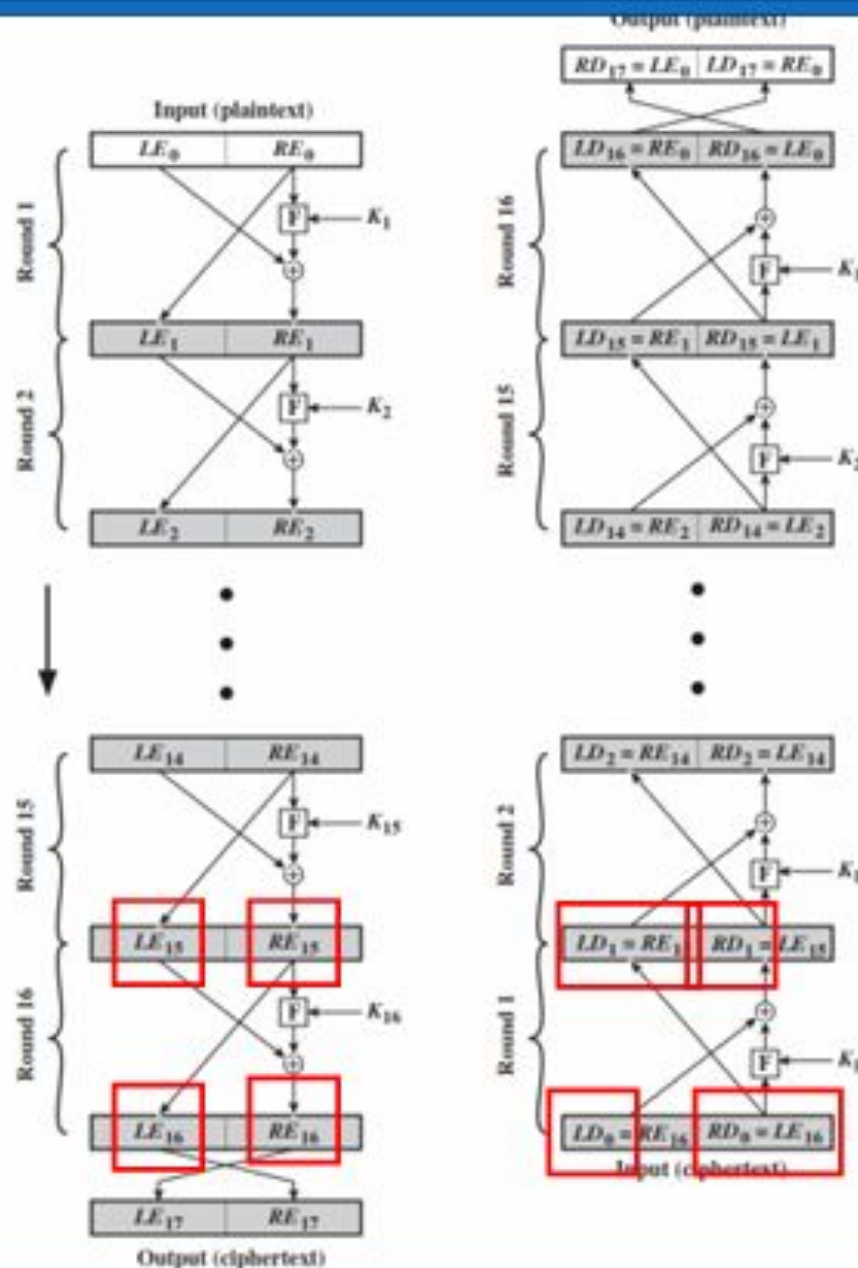
Diffusion

- Diffusion hides the relationship between the **ciphertext and the plaintext**.
- This is achieved by changing **one plaintext digit which affect** the value of many ciphertext digits.



Feistel Network Factors

- **Block size:** Common block size of **64-bit**. However, the new algorithms uses a 128-bit, 256-bit block size.
- **Key size:** Key sizes of **64 bits** or less are now widely considered to be insufficient, and 128 bits has become a common size.
- **Number of rounds:** A typical size is **16 rounds**.
- **Round function F:** This phase consisting of sixteen rounds of the same function, which involves both **permutation** and **substitution** functions. Again, greater complexity generally means greater resistance to cryptanalysis.
- **Subkey generation algorithm:** For each of the sixteen rounds, a **different subkey (K_i)** derived from **main key** by the combination of a **left circular shift** and a **permutation**. Greater complexity in this algorithm should lead to greater difficulty of cryptanalysis.



Feistel Encryption & Decryption

- Prove that o/p of first round of Decryption is equal to 32-bit swap of i/p of 16th round of Encryption
- $LD_1 = RE_{15}$ & $RD_1 = LE_{15}$

- On Encryption Side:

$$LE_{16} = RE_{15}$$

$$RE_{16} = LE_{15} \oplus F(RE_{15}, K_{16})$$

- On Decryption Side:

$$LD_1 = RD_0 = LE_{16} = RE_{15}$$

$$RD_1 = LD_0 \oplus F(RD_0, K_{16})$$

$$= RE_{16} \oplus F(RE_{15}, K_{16})$$

$$= [LE_{15} \oplus F(RE_{15}, K_{16})] \oplus F(RE_{15}, K_{16})$$

XOR Associativity Property

$$\therefore [A \oplus B] \oplus C = A \oplus [B \oplus C]$$

Thus,

$$LD_1 = RE_{15} \quad RD_1 = LE_{15}$$

Data Encryption

Type: Block Cipher

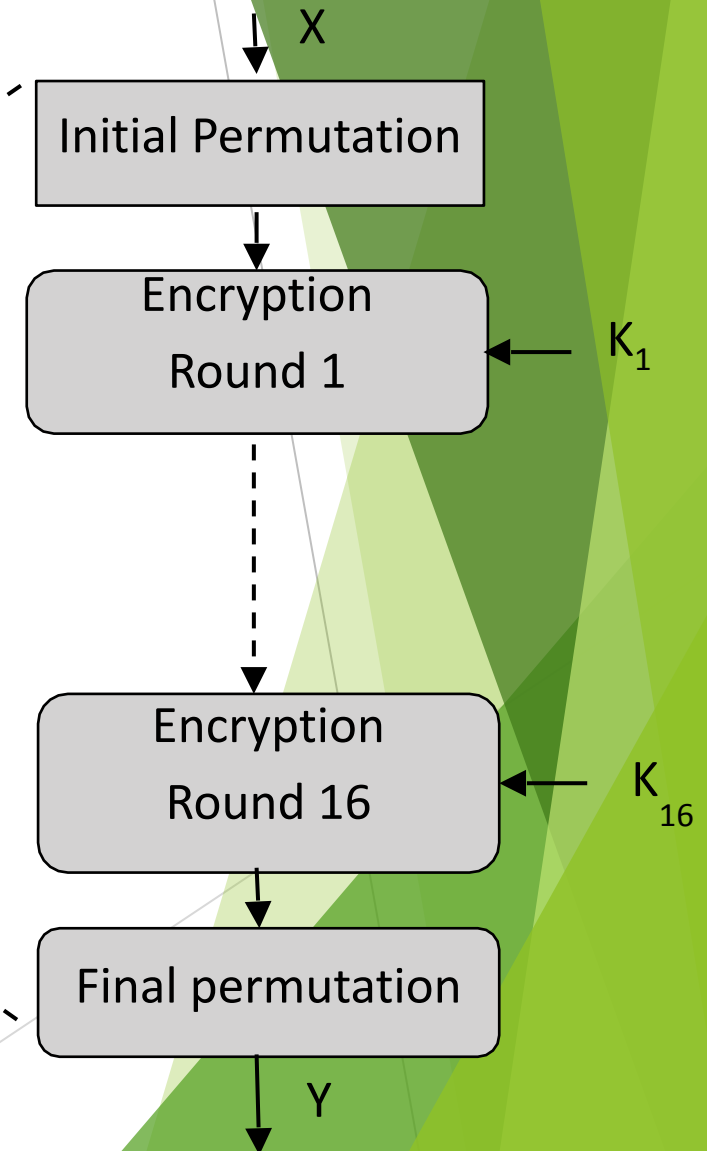
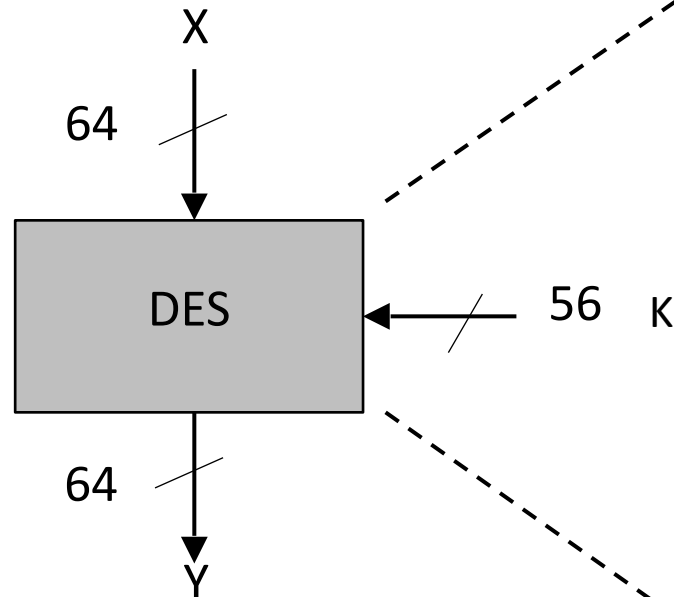
Block Size : 64-bit

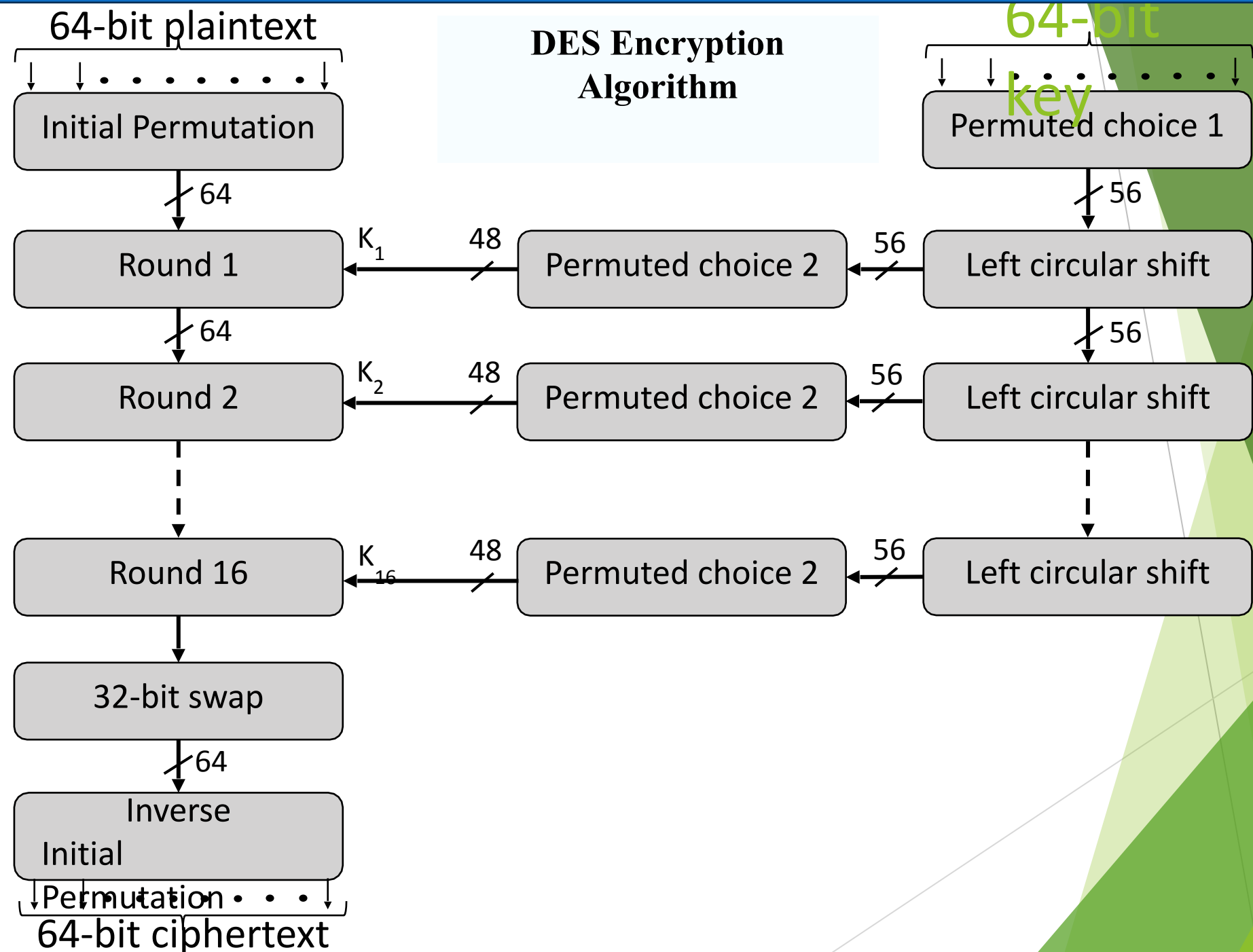
Key Size: 64-bit, with only 56-bit effective

Number of Rounds: 16

Standard

- The Data Encryption Standard (DES) is a symmetric-key block cipher published by the National Institute of Standards and Technology (NIST).
- DES is an implementation of a Feistel Cipher.
- It uses 16 round Feistel structure. The block size is 64-bit. Though, key length is 64-bit, DES has an effective key length of 56 bits,





Initial Permutation

1	2	3	4	5	6	7	8
9	10	11	12	13	14	15	16
17	18	19	20	21	22	23	24
25	26	27	28	29	30	31	32
33	34	35	36	37	38	39	40
41	42	43	44	45	46	47	48
49	50	51	52	53	54	55	56
57	58	59	60	61	62	63	64

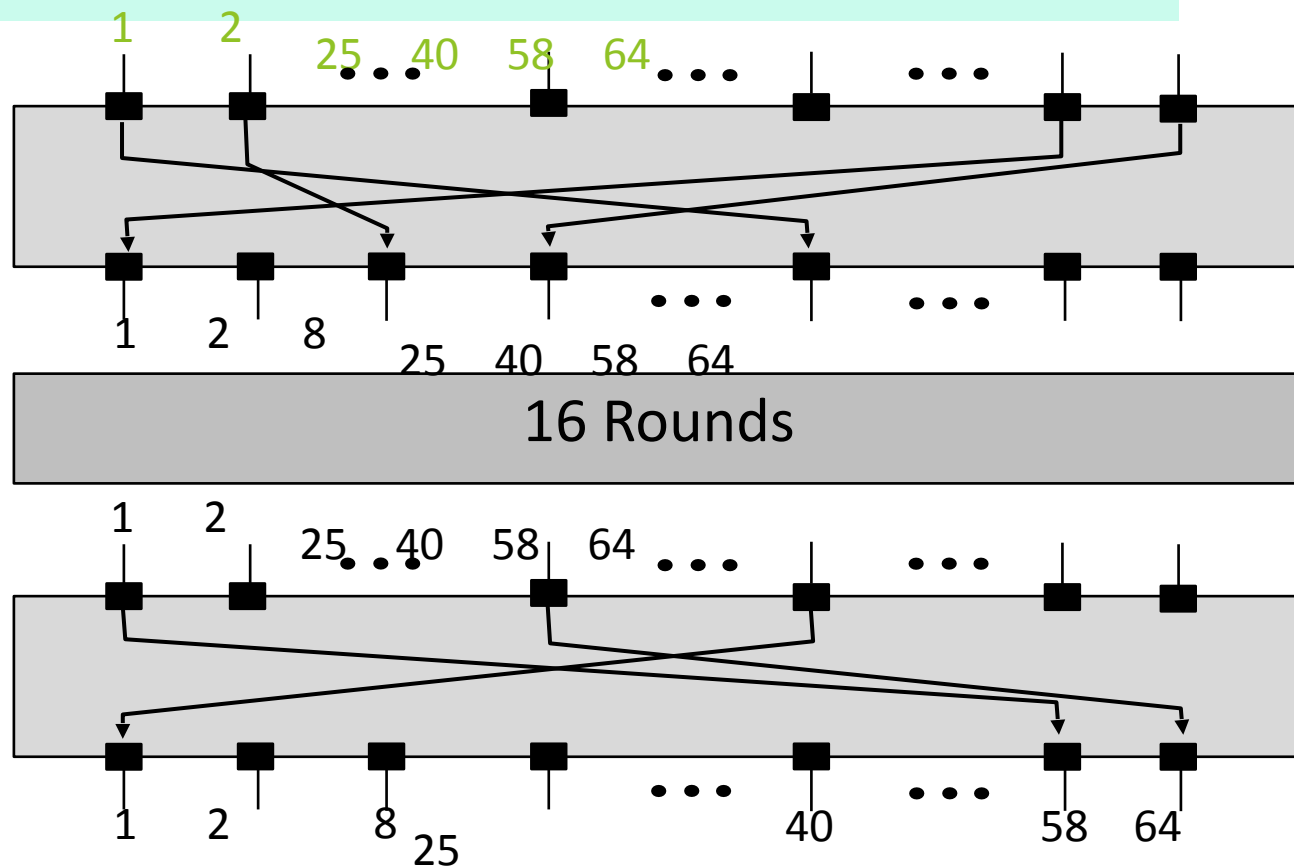


58	50	42	34	26	18	10	2
60	52	44	36	28	20	12	4
62	54	46	38	30	22	14	6
64	56	48	40	32	24	16	8
57	49	41	33	25	17	9	1
59	51	43	35	27	19	11	3
61	53	45	37	29	21	13	5
63	55	47	39	31	23	15	7

Inverse Initial Permutation

40	8	48	16	56	24	64	32
39	7	47	15	55	23	63	31
38	6	46	14	54	22	62	30
37	5	45	13	53	21	61	29
36	4	44	12	52	20	60	28
35	3	43	11	51	19	59	27
34	2	42	10	50	18	58	26
33	1	41	9	49	17	57	25

Initial and Inverse Permutation



- The initial permutation of the DES algorithm **changes the order of the plaintext** prior to the first round of encryption.

The final permutation occurs after the sixteen rounds of DES are completed. **It is the inverse of the initial permutation.**

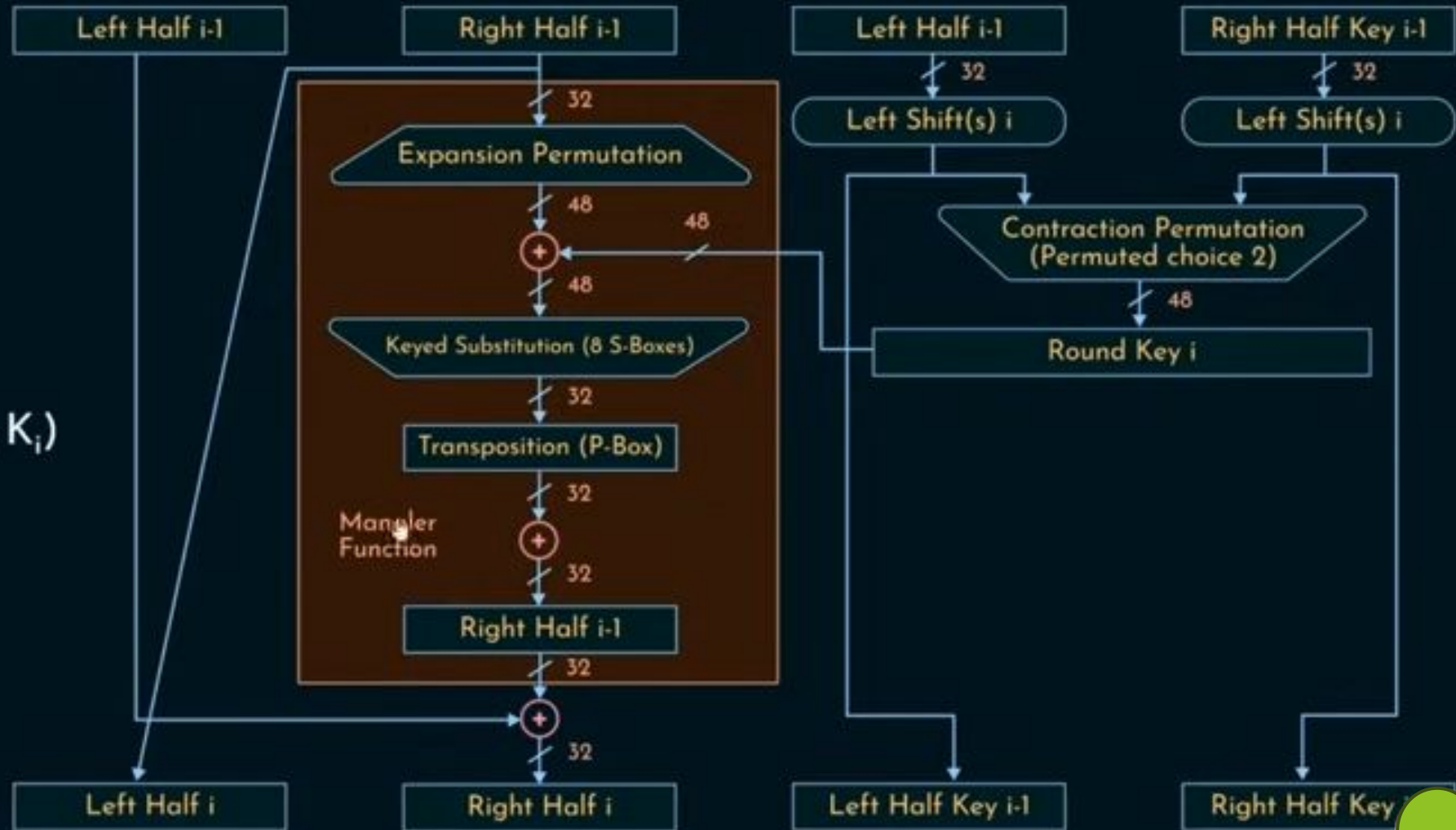
IP							
58	50	42	34	26	18	10	
2	60	52	44	36	28	20	12
4							
62	54	46	38	30	22	14	
6							
57	49	41	33	25	17	9	1
64	56	48	40	32	24	16	
8	59	51	43	35	27	19	11
61	53	45	37	29	21	13	5
63	55	47	39	31	23	15	7

IP ⁻¹							
40	8	48	16	56	24	64	32
39	7	47	15	55	23	63	31
38	6	46	14	54	22	62	30
37	5	45	13	53	21	61	29
36	4	44	12	52	20	60	28
35	3	43	11	51	19	59	27
34	2	42	10	50	18	58	26
33	1	41	9	49	17	57	25

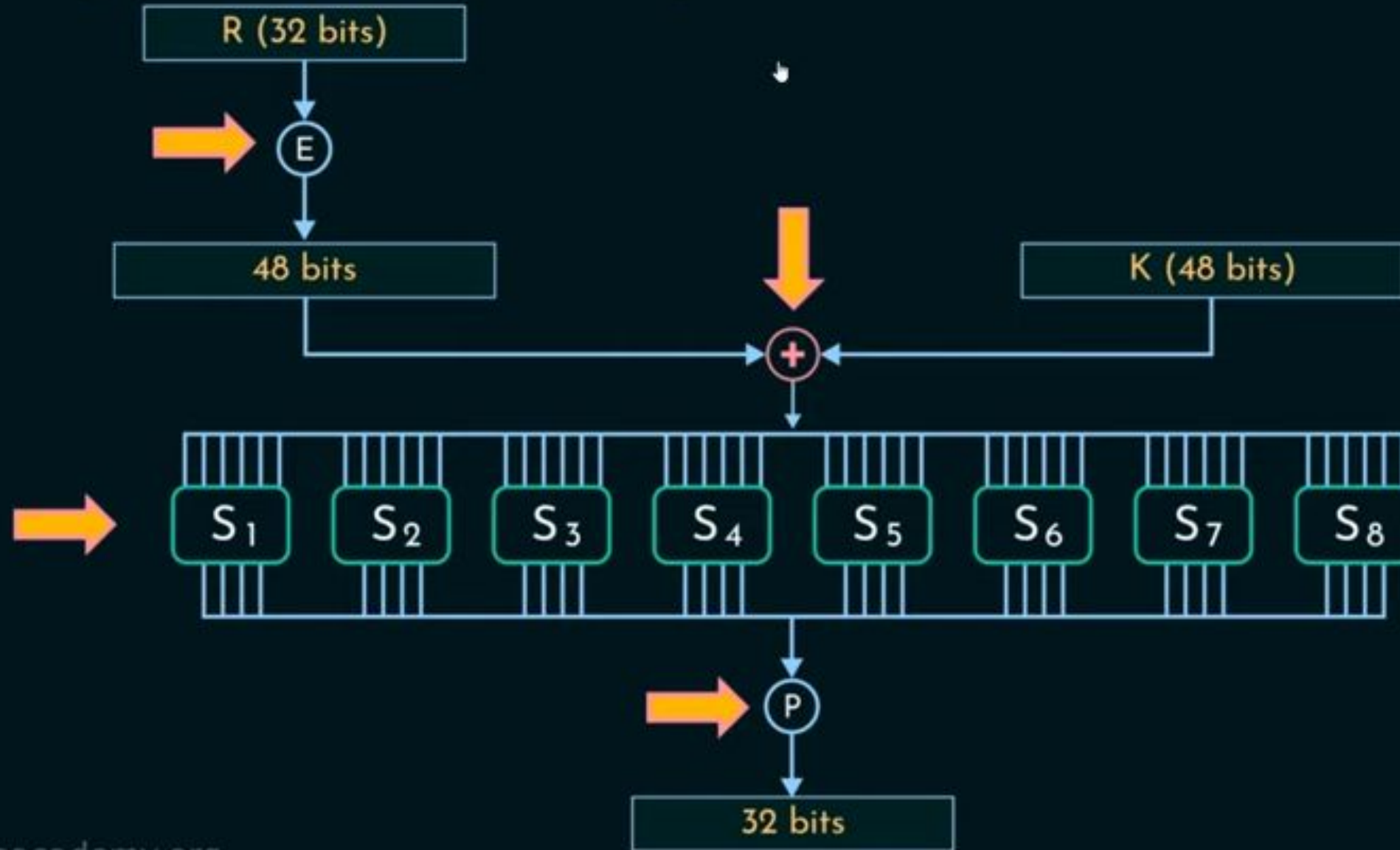
Single Round of DES Algorithm

$$L_i = R_{i-1}$$

$$R_i = L_{i-1} \oplus F(R_{i-1}, K_i)$$



Single Round of DES Algorithm



The Expansion Permutation

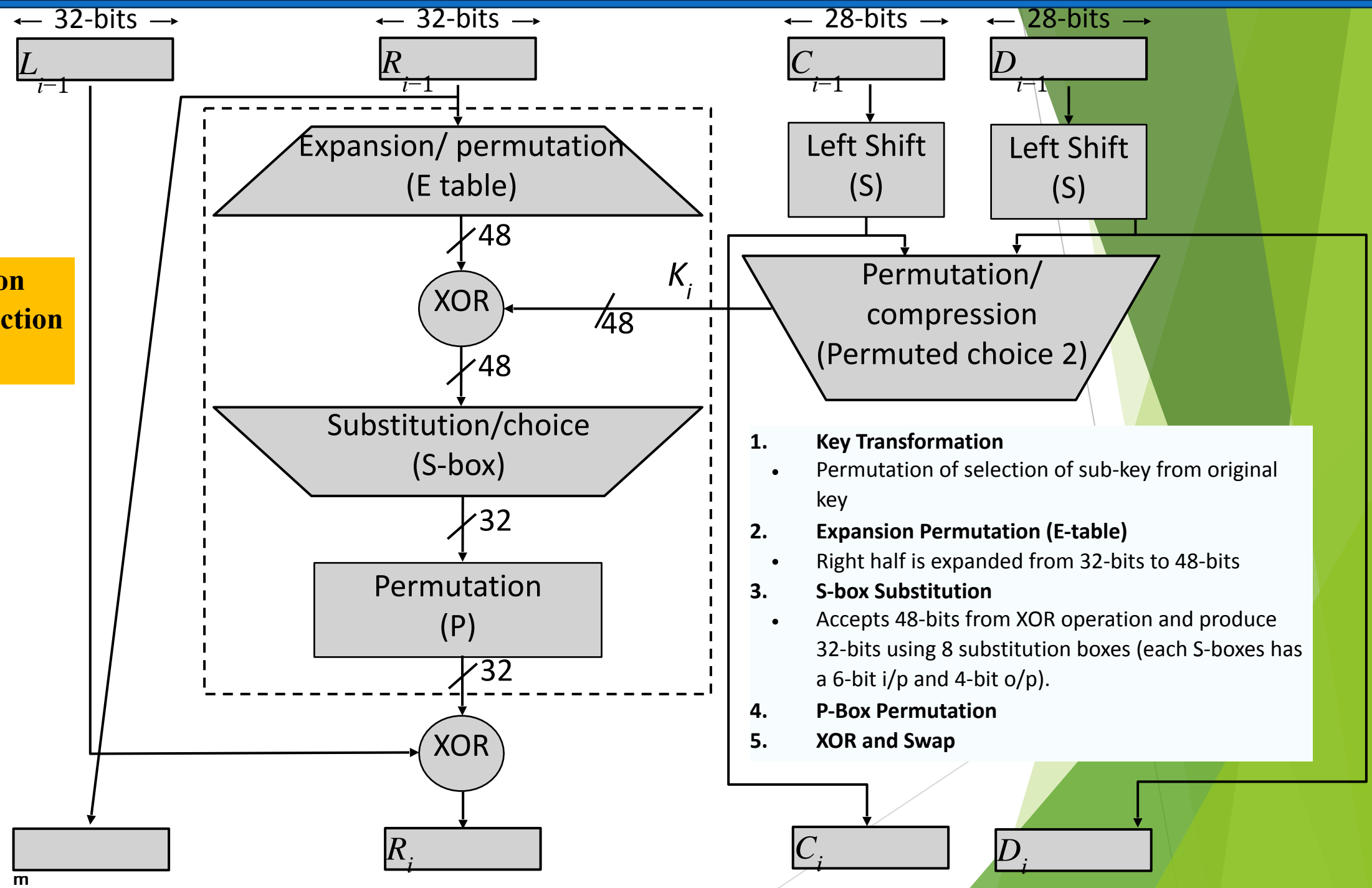
32	1	2	3	4	5
4	5	6	7	8	9
8	9	10	11	12	13
12	13	14	15	16	17
16	17	18	19	20	21
20	21	22	23	24	25
24	25	26	27	28	29
28	29	30	31	32	1

Box S_1

	0000	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111
00	14	4	13	1	2	15	11	8	3	10	6	12	5	9	0	7
01	0	15	7	4	14	2	13	1	10	6	12	11	6	5	3	8
10	4	1	14	8	13	6	2	11	15	12	9	7	3	10	5	0
11	15	12	8	2	4	9	1	7	5	11	3	14	10	0	6	13

For example, $S_1(101010) = 6 = 0110$.

**Data Encryption
Standard: Function
Working**

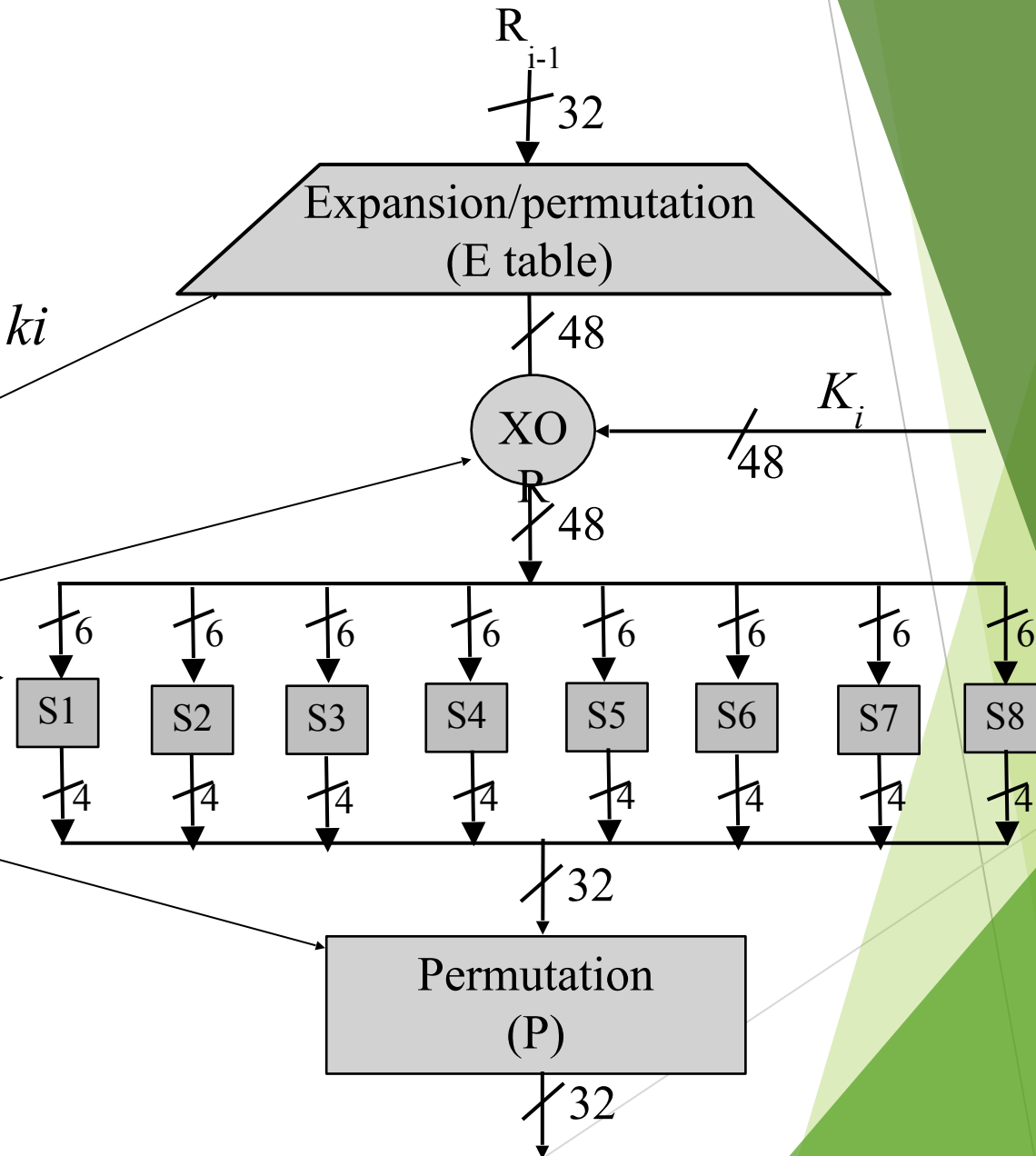


The F function

- Main operation of DES
- f-function inputs:
 R_{i-1} and round key k_i

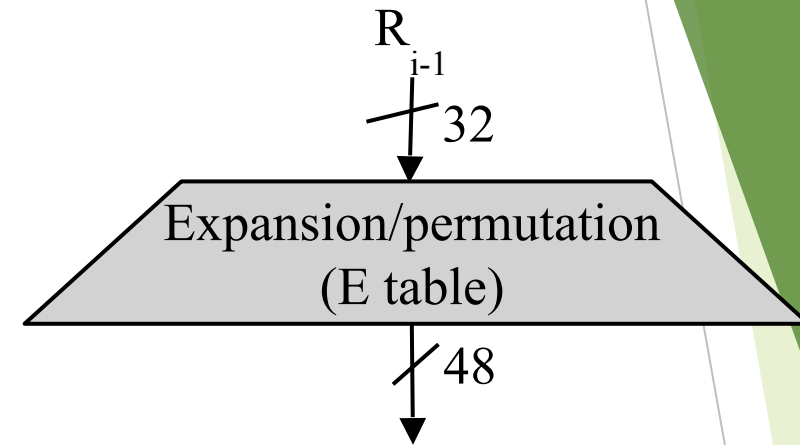
- **4 Steps:**

1. Expansion E
2. XOR with round key
3. S-box substitution
4. Permutation



1. The Expansion Function E

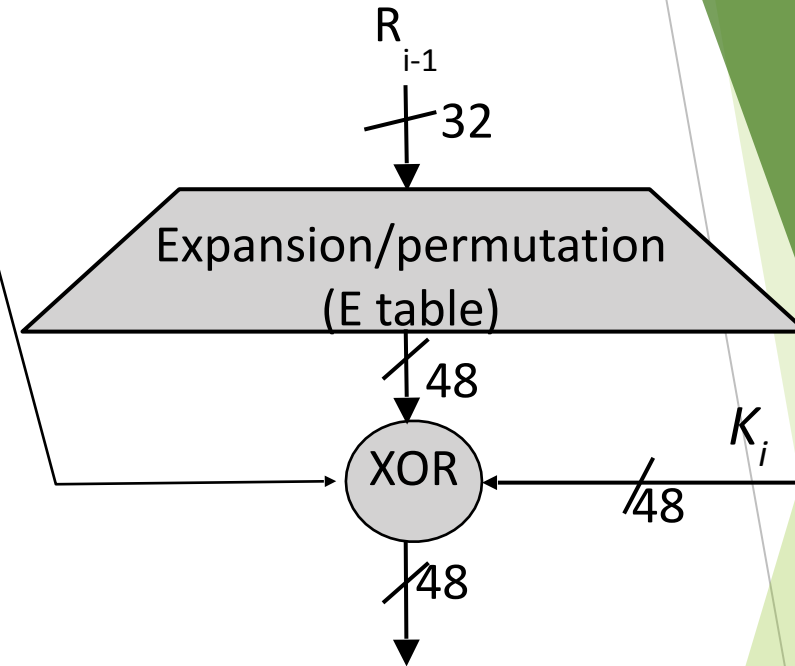
- Main purpose: **Increases diffusion**
- Since R_{i-1} is a 32-bit input and K_i is a 48-bit key, we first need to expand R_{i-1} to 48 bits.
- Input: (8 blocks, each of them consisting 4 bits) - 32 bits
- Output: (8 blocks, each of them consisting 6 bits) – 48 bits



Expansion Table E					
32	1	2	3	4	5
4	5	6	7	8	9
8	9	10	11	12	13
12	13	14	15	16	17
16	17	18	19	20	21
20	21	22	23	24	25
24	25	26	27	28	29
28	29	30	31	32	1

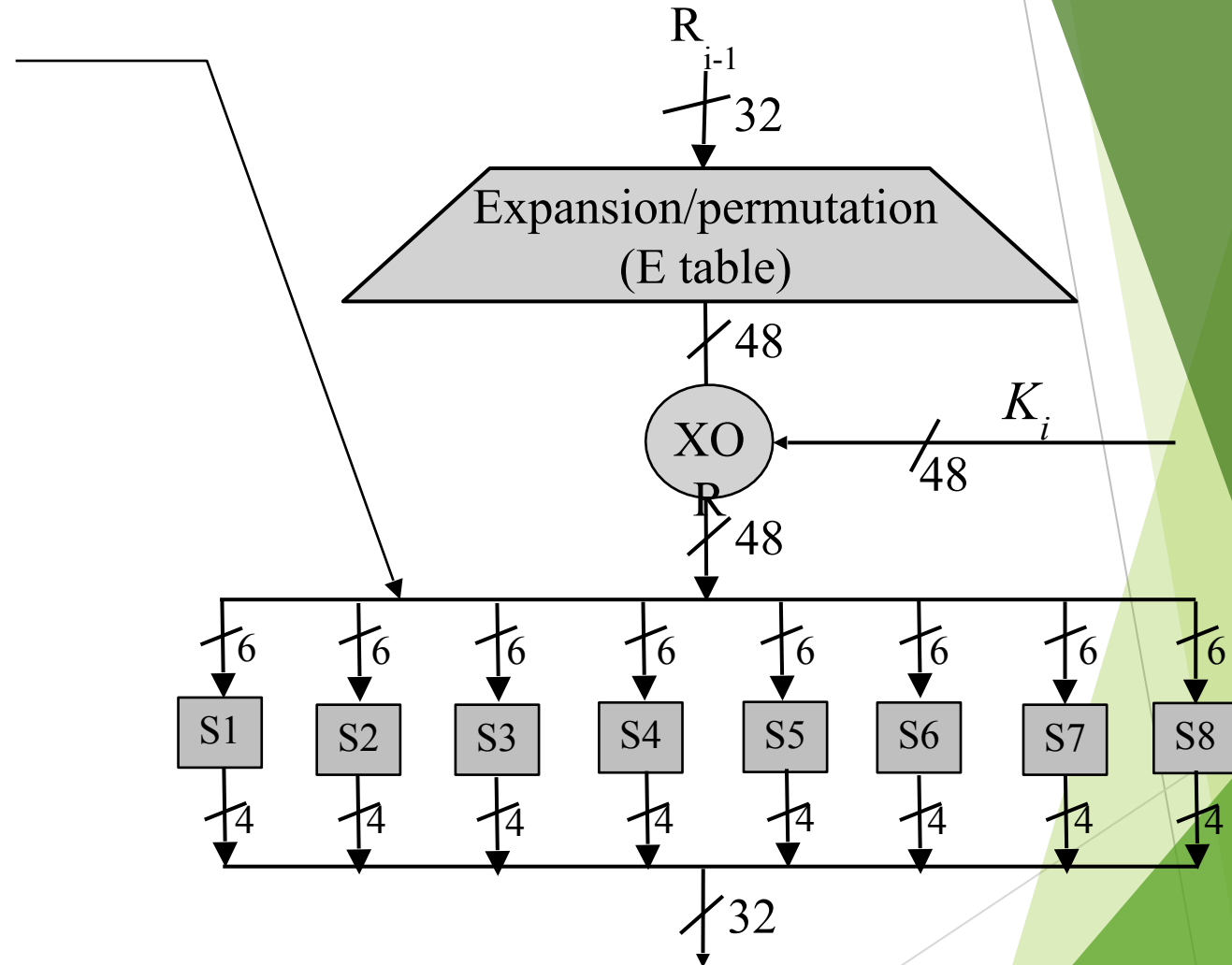
2. Add round key

- XOR Round Key
- After the expansion permutation, DES uses the **XOR** operation on the expanded right section and the round key.
- Note that both the right section and the key are 48-bits in length now.



3. The DES S-Boxes

- S-Box substitution.
- Eight substitution tables.
- 6 bits of input
- 4 bits of output.
- Convert 48 bits to 32 bits
- Non-linear and resistant to differential cryptanalysis.
- Crucial element for DES security!
- Introduces **confusion**.



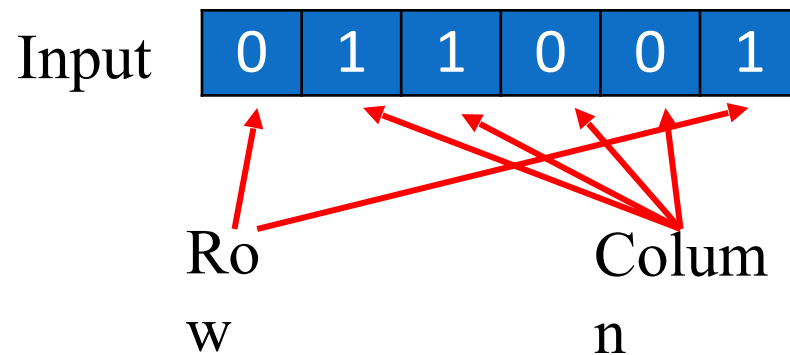
Role of S-box

- The outer two bits of each group select one row of an S-box.
- Inner four bits selects one column of an S-box.

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	14	04	13	01	02	15	11	08	03	10	06	12	05	09	00	07
1	00	15	07	04	14	02	13	10	03	06	12	11	09	05	03	08
2	04	01	14	08	13	06	02	11	15	12	09	07	03	10	05	00
3	15	12	08	02	04	09	01	07	05	11	03	14	10	00	06	13

S-box
1

■ Example:



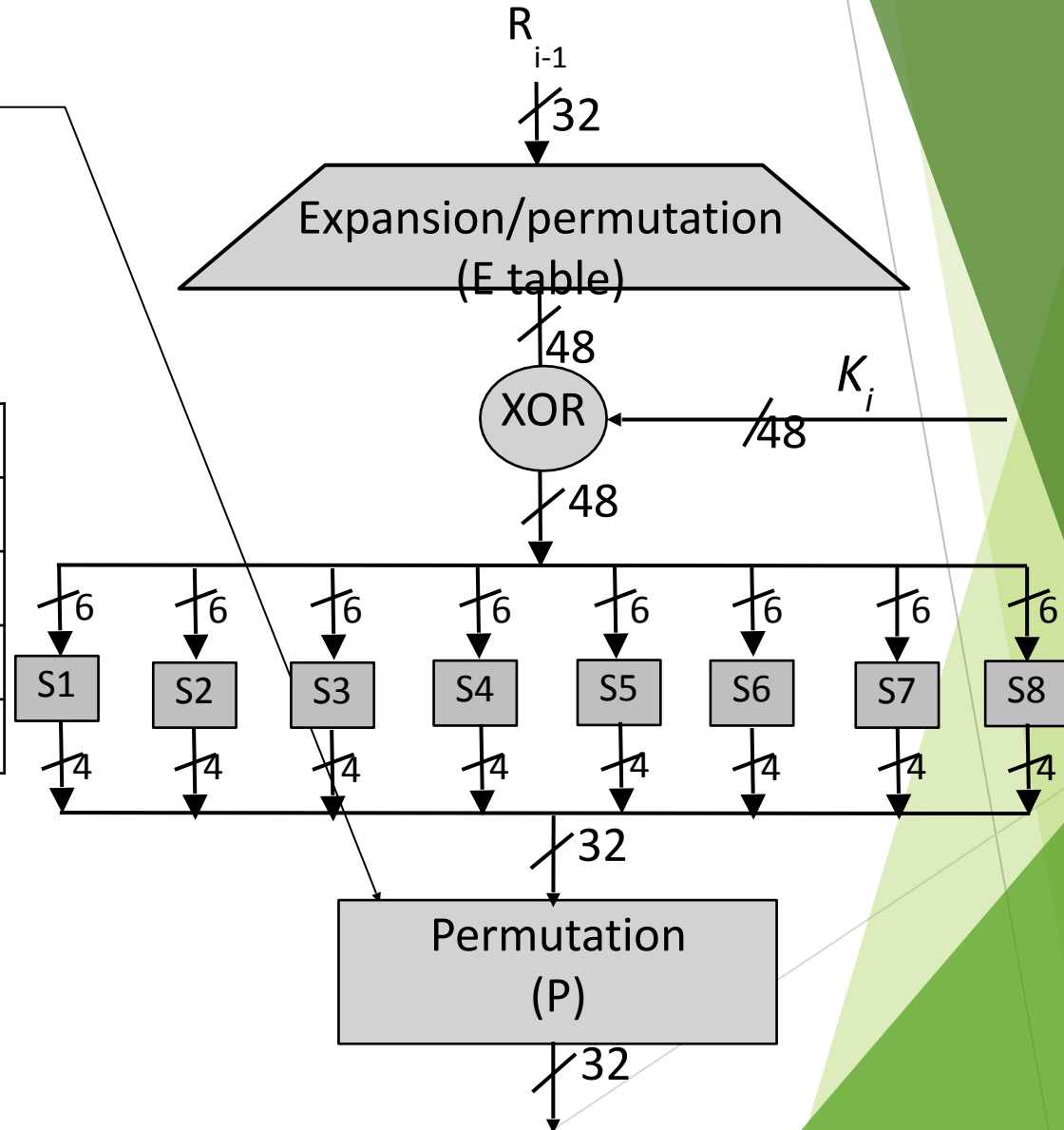
Output

1	0	0	1
---	---	---	---

4. The Permutation P

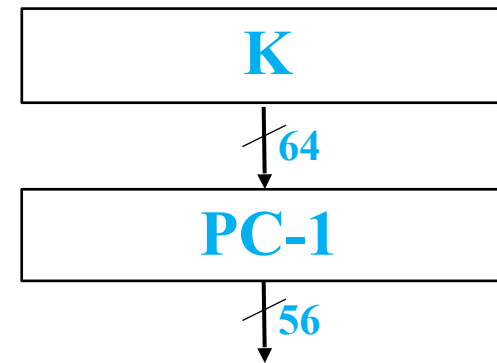
- Permutation P
- Bitwise permutation.
- Introduces **diffusion**.

Permutation Table P							
16	7	20	21	29	12	28	17
01	15	23	26	05	18	31	10
02	08	24	14	32	27	03	09
19	13	30	06	22	11	04	25



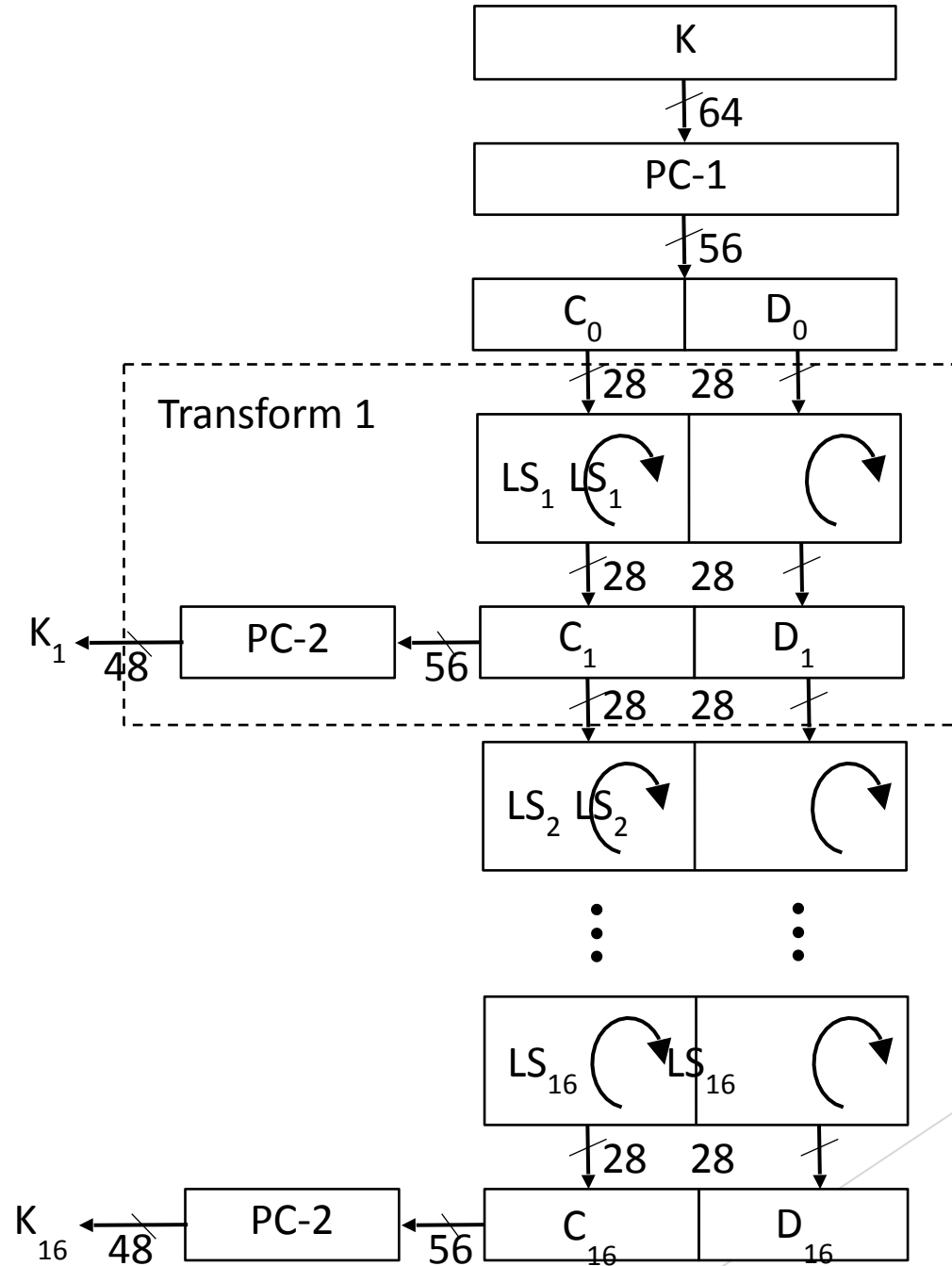
Key schedule of DES

- Derives 16 round keys (or subkeys) k_i of 48 bits each from the original 56 bit key.
- The input key size of the DES is 64 bit: 56 bit key and 8 bit parity
- **Parity bits are removed** in a first permuted choice PC-1: (note that the bits **8, 16, 24, 32, 40, 48, 56 and 64** are not used at all)



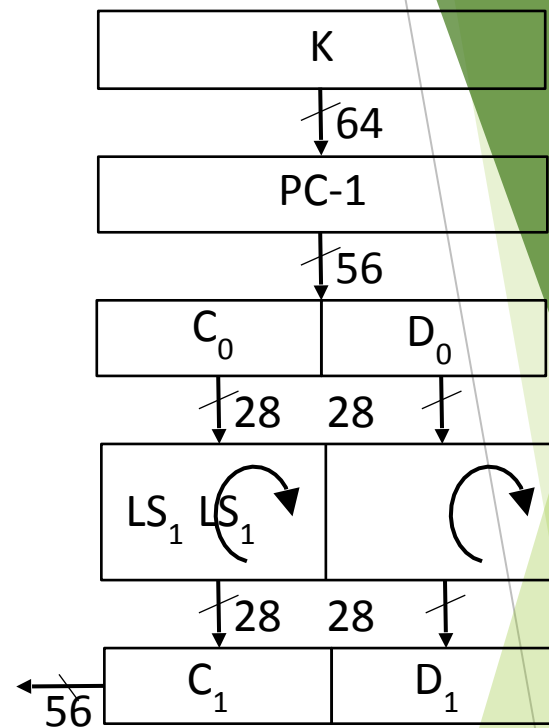
Permuted choice (PC-1)							
57	49	41	33	25	17	09	01
58	50	42	34	26	18	10	02
59	51	43	35	27	19	11	03
60	52	44	36	63	55	47	39
31	23	15	07	62	54	46	38
30	22	14	06	61	53	45	37
29	21	13	05	28	20	12	04

Subkey generation



Key schedule of DES

- Split key into 28-bit halves C_0 and D_0 .
- In rounds $i = 1, 2, 9, 16$, the two halves are each rotated left by one bit.
- In all other rounds where the two halves are each rotated left by two bits.
- These shifted values are input to the next round.

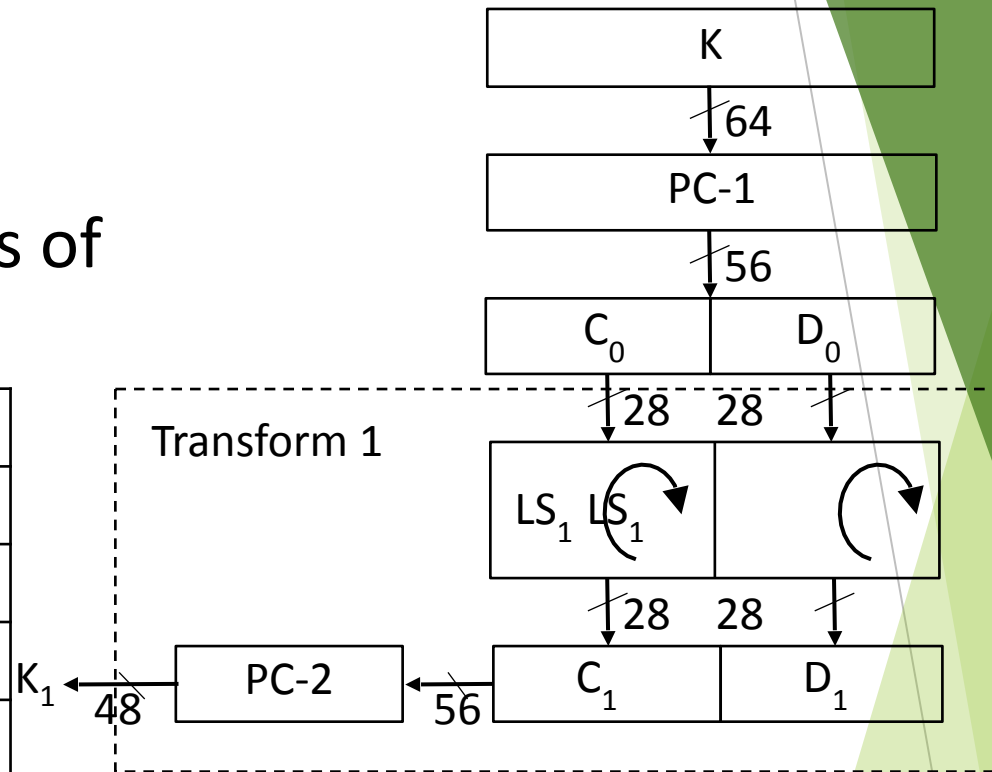


Round	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
key	1	1	2	2	2	2	2	2	1	2	2	2	2	2	2	1

Key schedule of DES

- In each round i permuted choice PC-2 selects a permuted subset of 48 bits of C_i and D_i as round key k

Permuted choice (PC-2)							
14	17	11	24	01	05	03	28
15	06	21	10	23	19	12	04
26	08	16	07	27	20	13	02
41	52	31	37	47	55	30	40
51	45	33	48	44	49	39	56
34	53	46	42	50	36	29	32



Avalanche Effect

- Desirable property of any encryption algorithm is that a change in one bit of the plaintext or of the key should produce a change in many bits of cipher text.
- DES performs strong **avalanche effect**.

Plaintext: 0000000000000000

Key: 22234512987ABB23

Ciphertext: 4789FD476E82A5F1

Plaintext: 0000000000000000**1**

Key: 22234512987ABB23

Ciphertext: 0A4ED5C15A63FEA3

- Although the two plaintext blocks differ only in the rightmost bit, the ciphertext blocks differ in 29 bits.
- This means that changing approximately 1.5 % of the plaintext creates a change of approximately 45 % in the ciphertext.

Strength of DES

- **The use of 56-bit keys:** 56-bit key is used in encryption, there are 2^{56} possible keys. A brute force attack on such number of keys is impractical.
- **The nature of algorithm:** Cryptanalyst can perform cryptanalysis by exploiting the characteristic of DES algorithm but no one has succeeded in finding out the weakness.

DES Analysis

The DES satisfies both the desired properties of block cipher. **These two properties make cipher very strong.**

- **Avalanche effect** – A small change in plaintext results in the very great change in the ciphertext.
- **Completeness** – Each bit of ciphertext depends on many bits of plaintext.

During the last few years, cryptanalysis have found some weaknesses in DES when key selected are weak keys. These keys shall be avoided.

DES has proved to be a very well designed block cipher. There have been no significant cryptanalytic attacks on DES other than exhaustive key

AES (Advanced Encryption Standard)

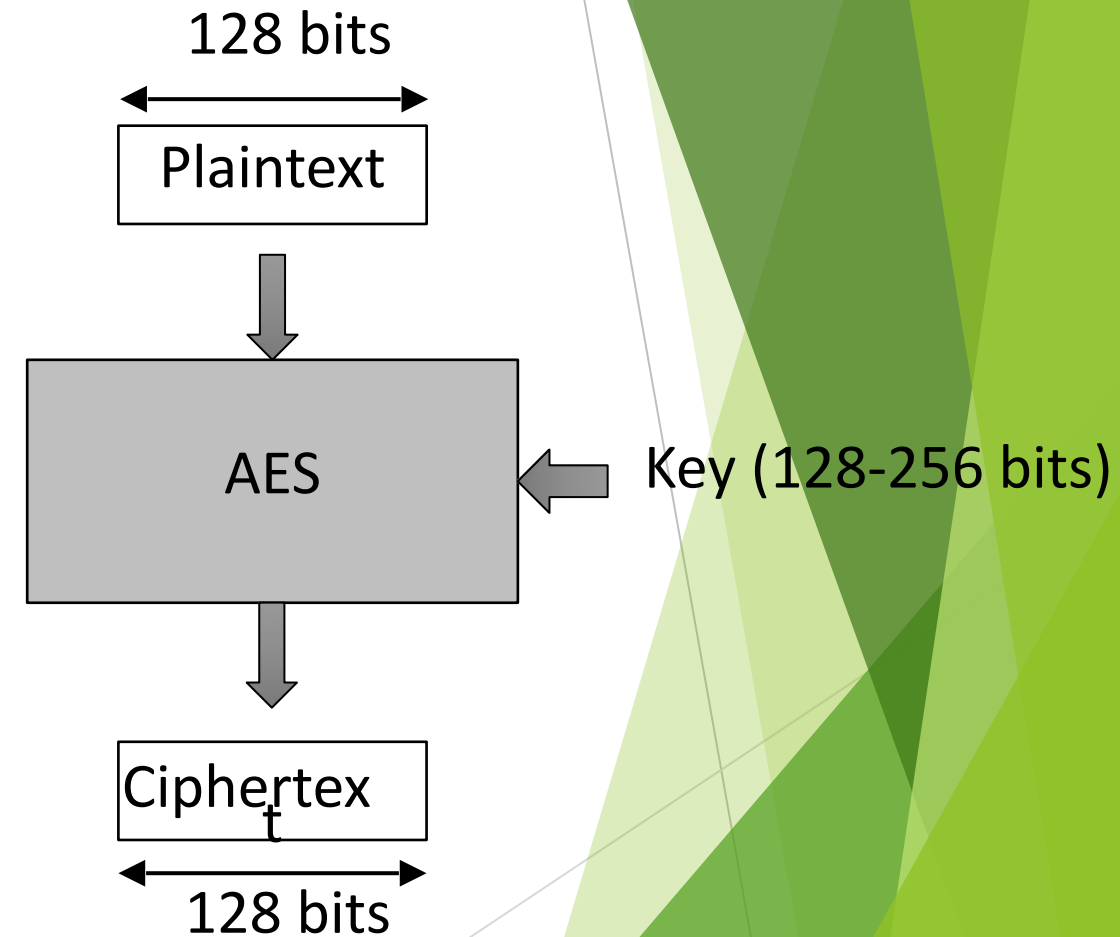
- The Rijndael proposal for AES defined a cipher in which the block length and the key length can be independently specified to be 128, 192, or 256 bits.

Key size (words/ bytes/ bits)	4/16/128	6/24/192	8/32/256
Block size (words/ bytes/ bits)	4/16/128	4/16/128	4/16/128
Round key size (words/ bytes/ bits)	4/16/128	4/16/128	4/16/128
Number of Rounds	10	12	14

- AES designed to have characteristics
 1. Resistance against all known attacks
 2. Speed and code compactness on a wide range of platforms
 3. Design simplicity

AES (Advanced Encryption Standard)

- Advanced Encryption Standard (AES) is a specification for the encryption of electronic data established by the U.S National Institute of Standards and Technology (NIST) in 2001.
- AES is widely used today as it is a much stronger than DES and triple DES despite being harder to implement.
- Points to remember
 - AES is a block cipher.
 - The key size can be 128/192/256 bits.
 - Encrypts data in blocks of 128 bits each.



Advanced Encryption Standard (AES)

After DES was used as an encryption standard for over 20 years and it was cracked in a relatively short time once the necessary technology was available, NIST decided a new standard, the Advanced Encryption Standard (AES), needed to be put into place.

In January 1997, NIST announced its request for AES candidates and outlined the requirements in FIPS PUB 197. AES was to be a symmetric block cipher supporting key sizes of 128, 192, and 256 bits. The following five algorithms were the finalists:

AES Finalists

MARS: Developed by the IBM team that created Lucifer

- RC6: Developed by RSA Laboratories
- Serpent: Developed by Ross Anderson, Eli Biham, and Lars Knudsen
- Twofish: Developed by Counterpane Systems
- Rijndael: Developed by Joan Daemen and Vincent Rijmen

Rijndael

Out of these contestants, Rijndael was chosen. The block sizes that Rijndael supports are 128, 192, and 256 bits. The number of rounds depends upon the size of the block and the key length:

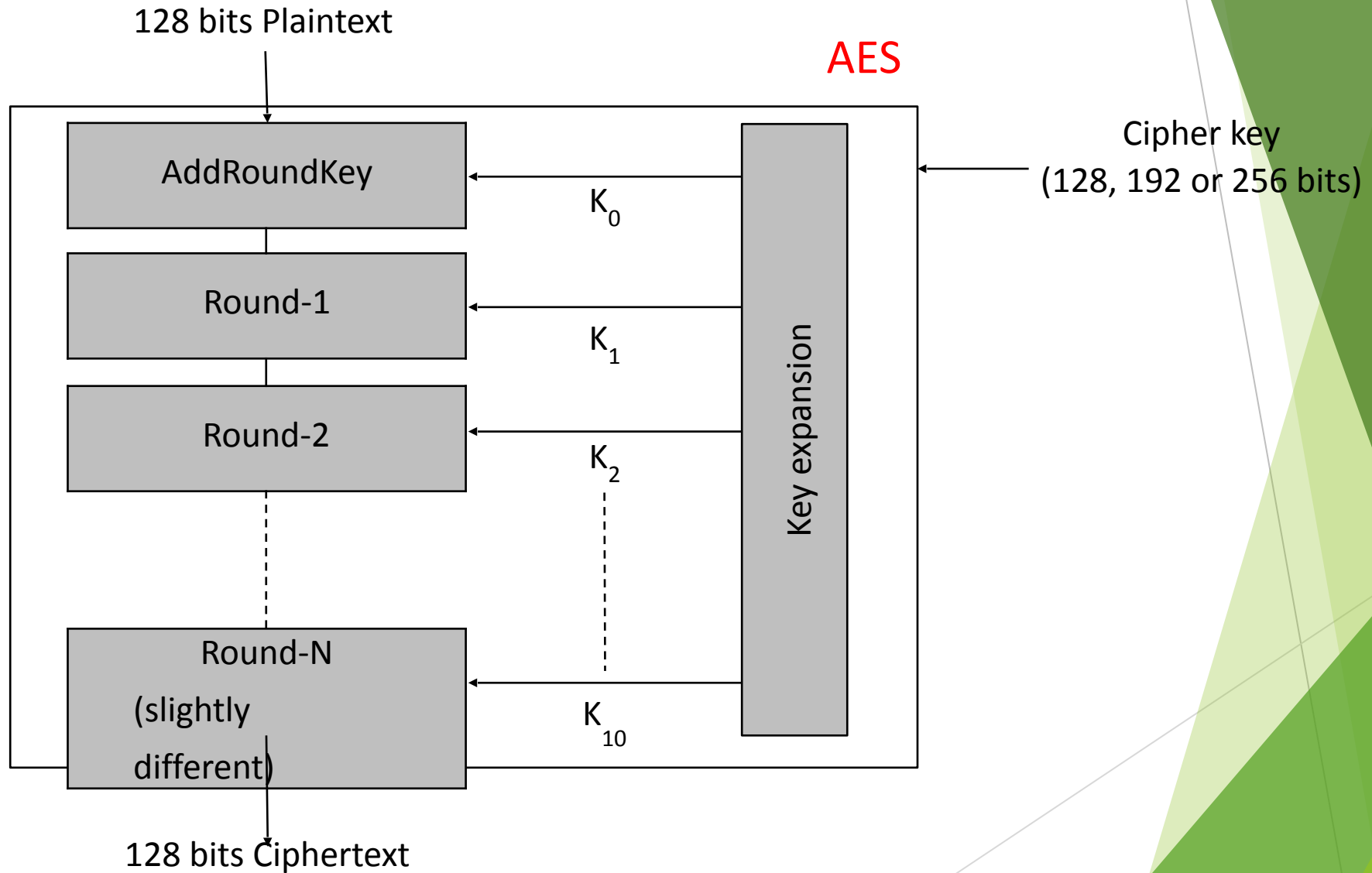
- If both the key and block size are 128 bits, there are 10 rounds
- If both the key and block size are 192 bits, there are 12 rounds
- If both the key and block size are 256 bits, there are 14 rounds

Rijndael Advantages

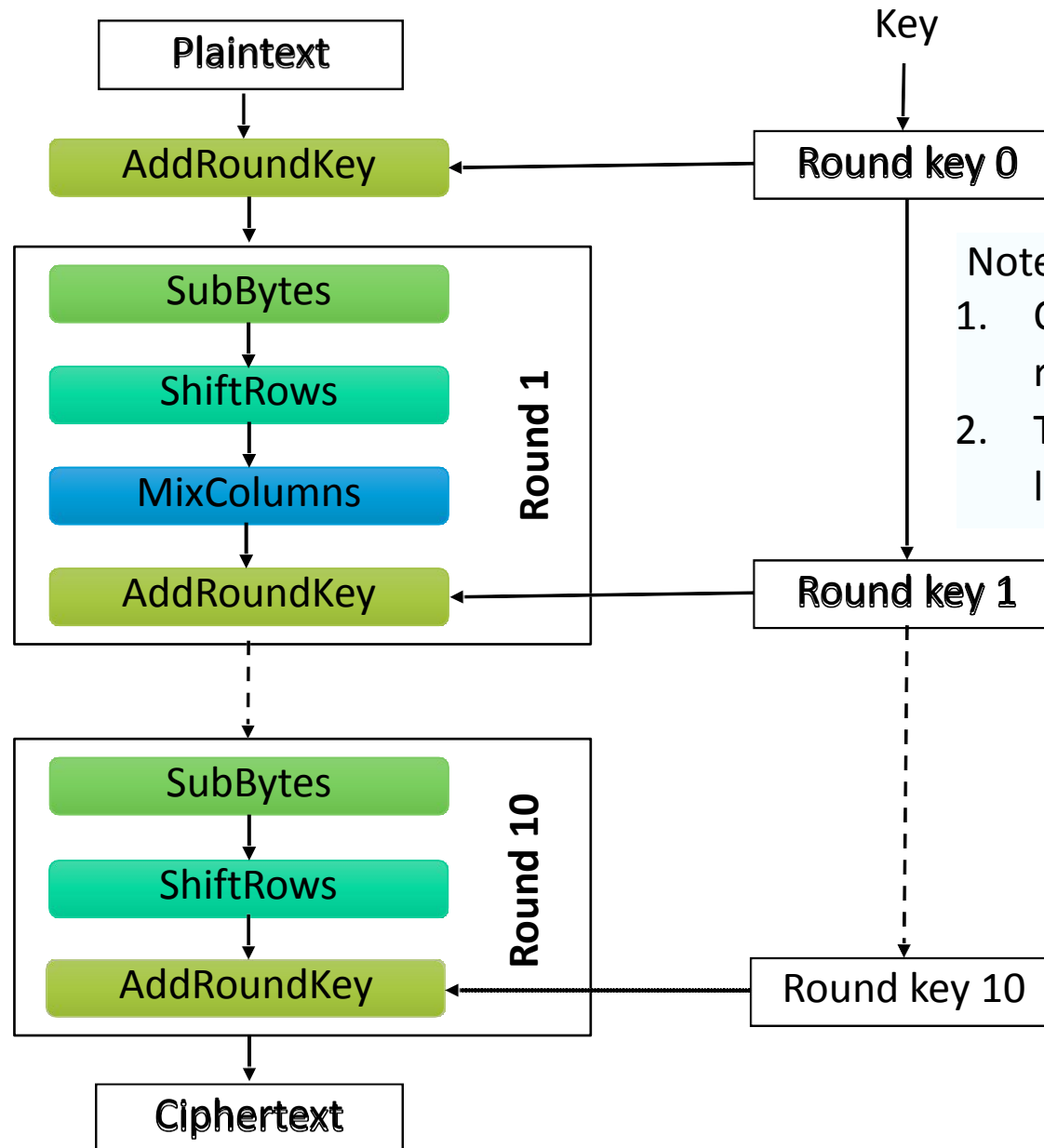
Rijndael works well when implemented in software and hardware in a wide range of products and environments. It has low memory requirements and has been constructed to easily defend against timing attacks.

Rijndael was NIST's choice to replace DES. It is now the algorithm required to protect sensitive but unclassified government information.

AES (Advanced Encryption Standard)

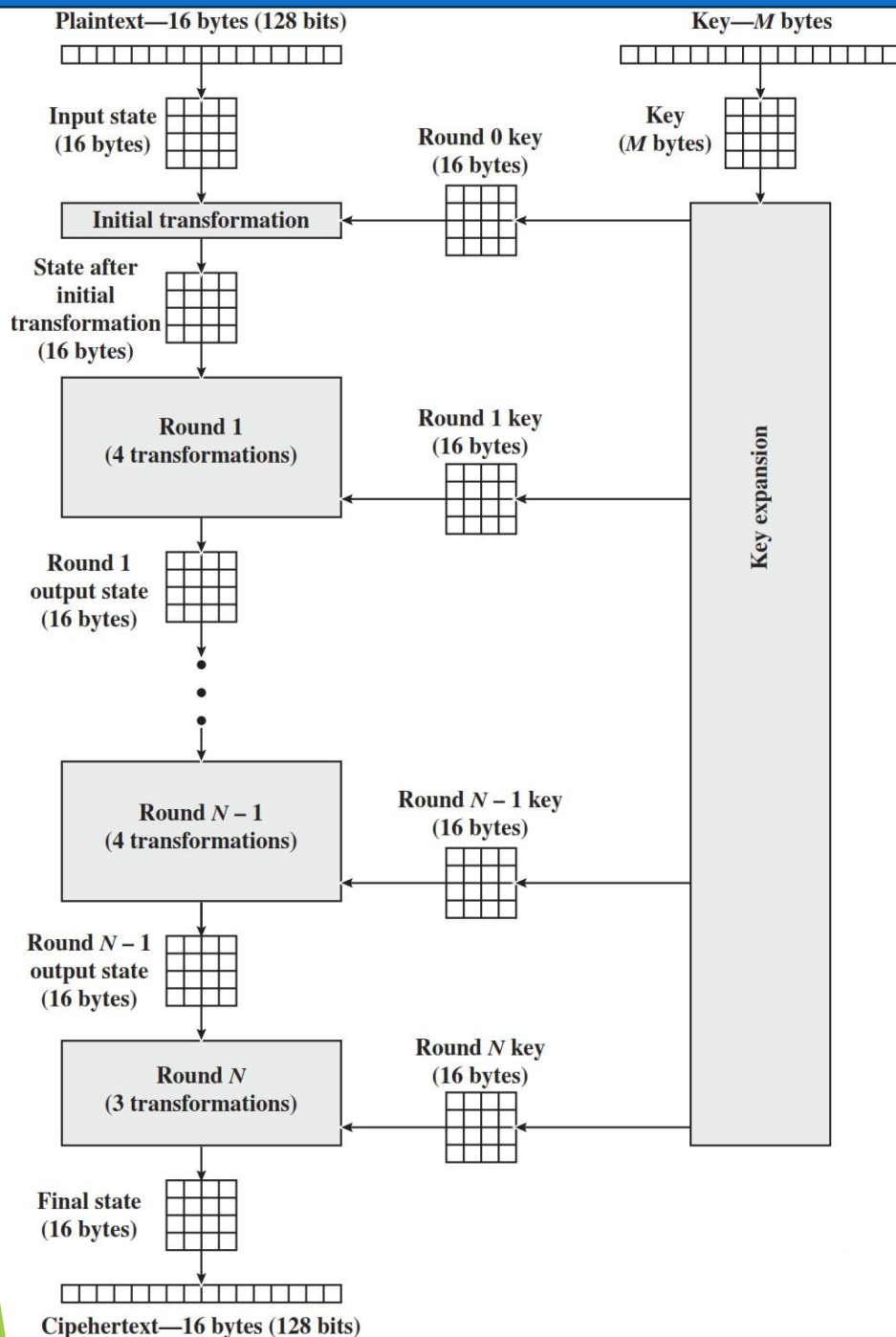


AES (Advanced Encryption Standard)



Notes:

1. One AddRoundKey is applied before the First round.
2. The third transformation is missing in the last round



AES Structure

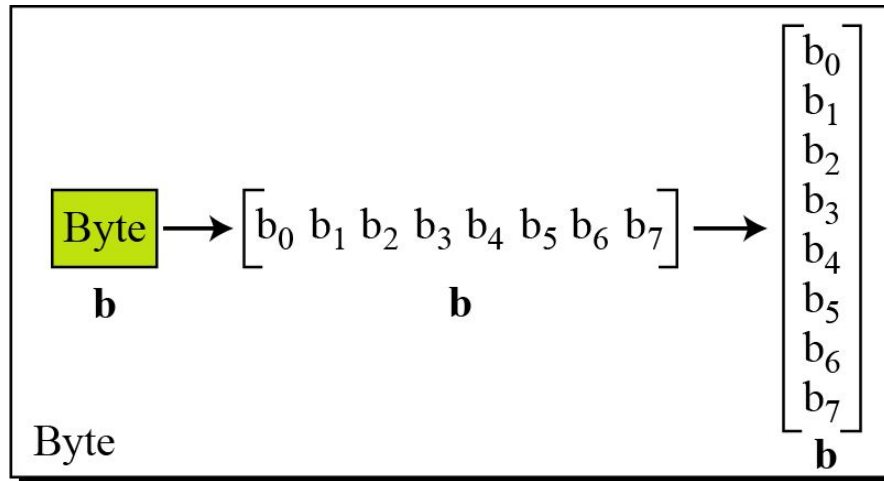
Initialization

1. Expand 16-byte key to get the actual **key block** to be used.
2. Initialize 16-byte plaintext block called as **state**.
3. XOR the **state** with the **key block**.

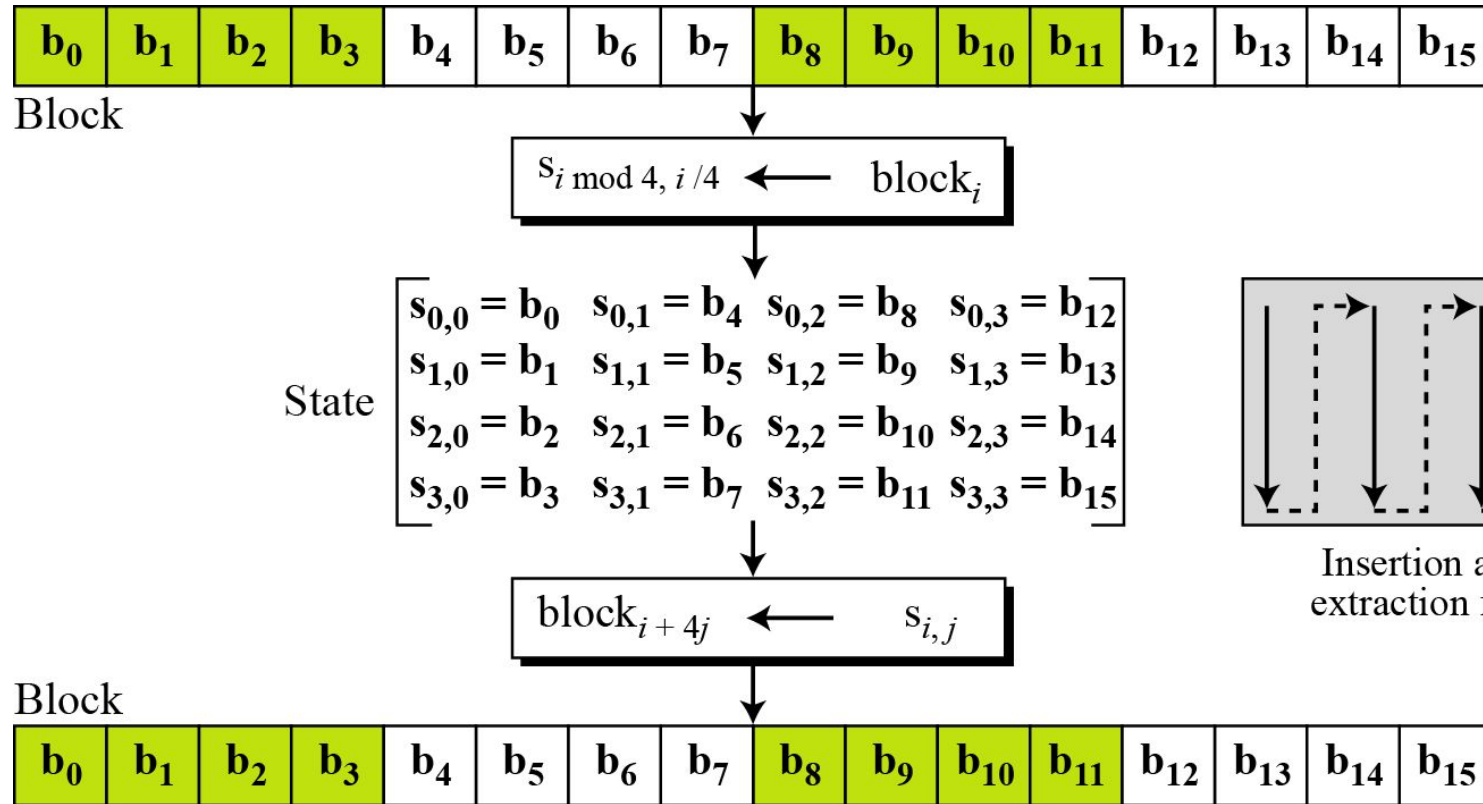
For each round

1. Apply S-box
2. Rotate rows of state
3. Mix columns
4. Add Round key: XOR the state with key block.

Data Units in AES



Block to State & State to Block



Plain Text to State

Text	A	E	S	U	S	E	S	A	M	A	T	R	I	X	Z	Z
Hexadecimal	00	04	12	14	12	04	12	00	0C	00	13	11	08	23	19	19

$$\begin{bmatrix} 00 & 12 & 0C & 08 \\ 04 & 04 & 00 & 23 \\ 12 & 12 & 13 & 19 \\ 14 & 00 & 11 & 19 \end{bmatrix} \text{ State}$$

AES Structure

- The first N-1 rounds consist of four distinct transformation functions.

SubBytes

- The 16 input bytes are substituted using an **S-box**

ShiftRows

- Each of the four rows of the matrix is shifted to the left

MixColumns

- Each column of four bytes is now transformed using a special mathematical function.

AddRoundKey
y

- The 16 bytes of the matrix are now considered as 128 bits and are XORed to the 128 bits of the round key.

AES structure

State:

32	88	31	e0
43	5a	31	37
f6	30	98	07
a8	8d	a2	34

Cipher key:

2b	28	ab	09
7e	ae	f7	cf
15	d2	15	4f
16	a6	88	3c

Initial

transformation(AddRoundKey)

AddRoundKey:

input state

Cipher key

32	88	31	e0
43	5a	31	37
f6	30	98	07
a8	8d	a2	34

$\oplus$

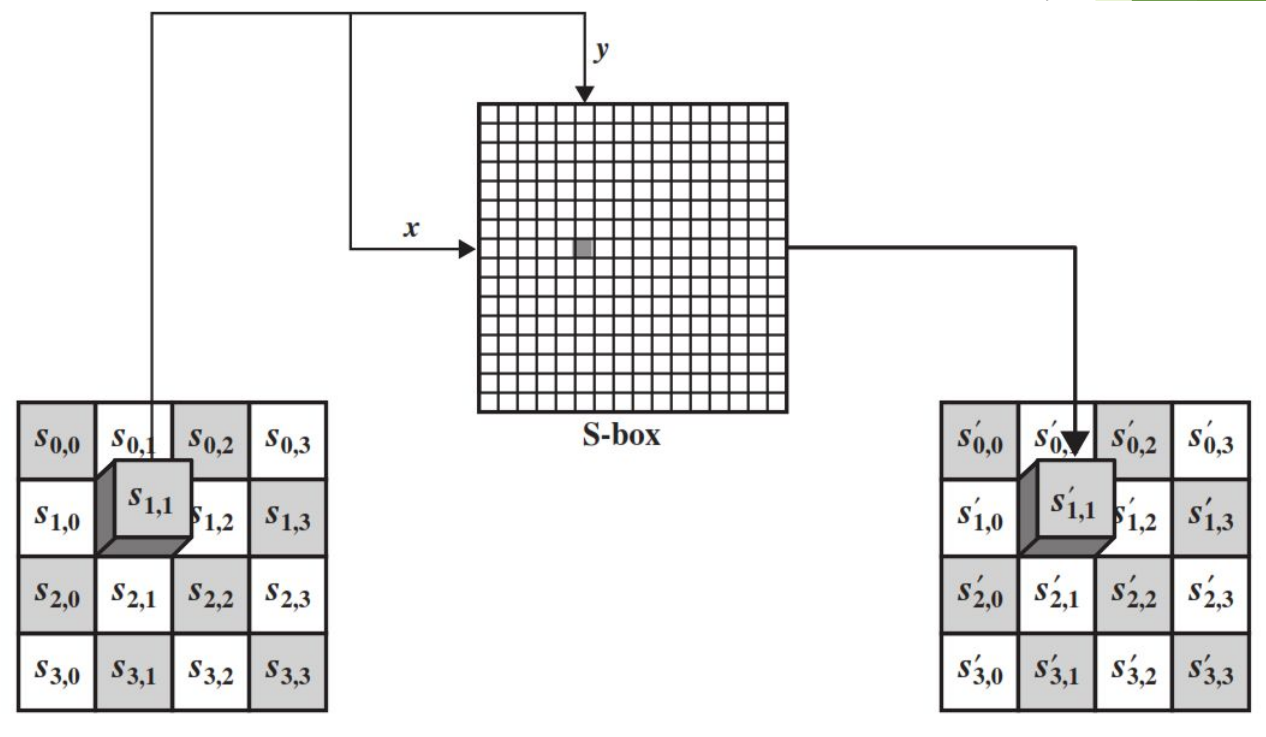
2b	28	ab	09
7e	ae	f7	cf
15	d2	15	4f
16	a6	88	3c

=

19	a0	9a	e9
3d	f4	c6	f8
e3	e2	8d	48
be	2b	2a	08

SubByte Transformation

- The forward substitute byte transformation, called **SubBytes**, is a simple table lookup



	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	63	7C	77	7B	F2	6B	6F	C5	30	01	67	2B	FE	D7	AB	76
1	CA	82	C9	7D	FA	59	47	F0	AD	D4	A2	AF	9C	A4	72	C0
2	B7	FD	93	26	36	3F	F7	CC	34	A5	E5	F1	71	D8	31	15
3	04	C7	23	C3	18	96	05	9A	07	12	80	E2	EB	27	B2	75
4	09	83	2C	1A	1B	6E	5A	A0	52	3B	D6	B3	29	E3	2F	84
5	53	D1	00	ED	20	FC	B1	5B	6A	CB	BE	39	4A	4C	58	CF
6	D0	EF	AA	FB	43	4D	33	85	45	F9	02	7F	50	3C	9F	A8
7	51	A3	40	8F	92	9D	38	F5	BC	B6	DA	21	10	FF	F3	D2
8	CD	0C	13	EC	5F	97	44	17	C4	A7	7E	3D	64	5D	19	73
9	60	81	4F	DC	22	2A	90	88	46	EE	B8	14	DE	5E	0B	DB
A	E0	32	3A	0A	49	06	24	5C	C2	D3	AC	62	91	95	E4	79
B	E7	C8	37	6D	8D	D5	4E	A9	6C	56	F4	EA	65	7A	AE	08
C	BA	78	25	2E	1C	A6	B4	C6	E8	DD	74	1F	4B	BD	8B	8A
D	70	3E	B5	66	48	03	F6	0E	61	35	57	B9	86	C1	1D	9E
E	E1	F8	98	11	69	D9	8E	94	9B	1E	87	E9	CE	55	28	DF
F	8C	A1	89	0D	BF	E6	42	68	41	99	2D	0F	B0	54	BB	16

Input: 19

Output: D4

Row
Column

SubByte output

Input for SubByte

19	a0	9a	e9
3d	f4	c6	f8
e3	e2	8d	48
be	2b	2a	08

Output of SubByte

d4	e0	b8	le
27	bf	b4	41
11	98	5d	52
ae	f1	e5	30

ShiftRows

- The first row of State is not altered.
- For the second row, a 1-byte circular left shift is performed.
- For the third row, a 2-byte circular left shift is performed.
- For the fourth row, a 3-byte circular left shift is performed.

d4	e0	b8	le	← No rotation
27	bf	b4	41	← Rotate 1 byte
11	98	5d	52	← Rotate 2 bytes
ae	f1	e5	30	← Rotate 3 bytes

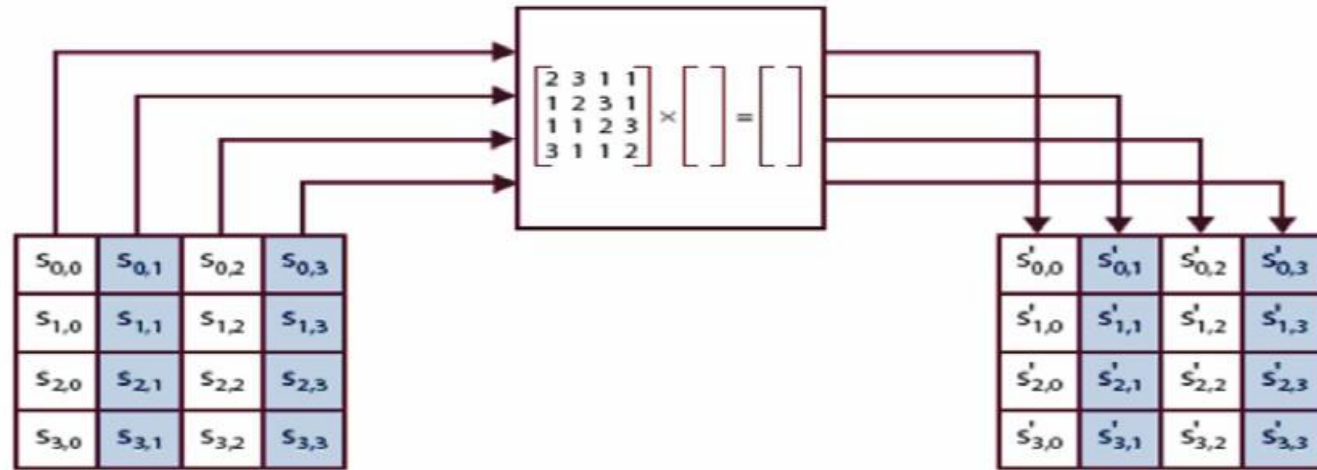
Input for ShiftRows

d4	e0	b8	le
bf	b4	41	27
5d	52	11	98
30	ae	f1	e5

Output of ShiftRows

MixColumns

- Each byte of a column is mapped into a new value that is a function of all four bytes in that column.
- Constant matrices used by MixColumns.



$$\begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix} \begin{bmatrix} s_{0,0} & s_{0,1} & s_{0,2} & s_{0,3} \\ s_{1,0} & s_{1,1} & s_{1,2} & s_{1,3} \\ s_{2,0} & s_{2,1} & s_{2,2} & s_{2,3} \\ s_{3,0} & s_{3,1} & s_{3,2} & s_{3,3} \end{bmatrix} = \begin{bmatrix} s'_{0,0} & s'_{0,1} & s'_{0,2} & s'_{0,3} \\ s'_{1,0} & s'_{1,1} & s'_{1,2} & s'_{1,3} \\ s'_{2,0} & s'_{2,1} & s'_{2,2} & s'_{2,3} \\ s'_{3,0} & s'_{3,1} & s'_{3,2} & s'_{3,3} \end{bmatrix}$$

MixColumns

d4	e0	b8	le
bf	b4	41	27
5d	52	11	98
30	ae	f1	e5

 •
$$\begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix} =$$

04	e0	48	28
66	cb	f8	06
81	19	d3	26
e5	9a	7a	4c

$$\begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix} \bullet$$

d4
bf
5d
30

$$=$$

04
66
81
e5

AddRoundKey

- In the forward add round key transformation, the 128 bits of State are bitwise XORed with the 128 bits of the round key.

04	e0	48	28
66	cb	f8	06
81	19	d3	26
e5	9a	7a	4c

 $\oplus$

a0	88	23	2a
fa	54	a3	6c
fe	2c	39	76
17	b1	39	05

 $=$

A4	68	6b	02
9c	9f	5b	6a
7f	35	Ea	50
F2	2b	43	49

04
66
81
e5

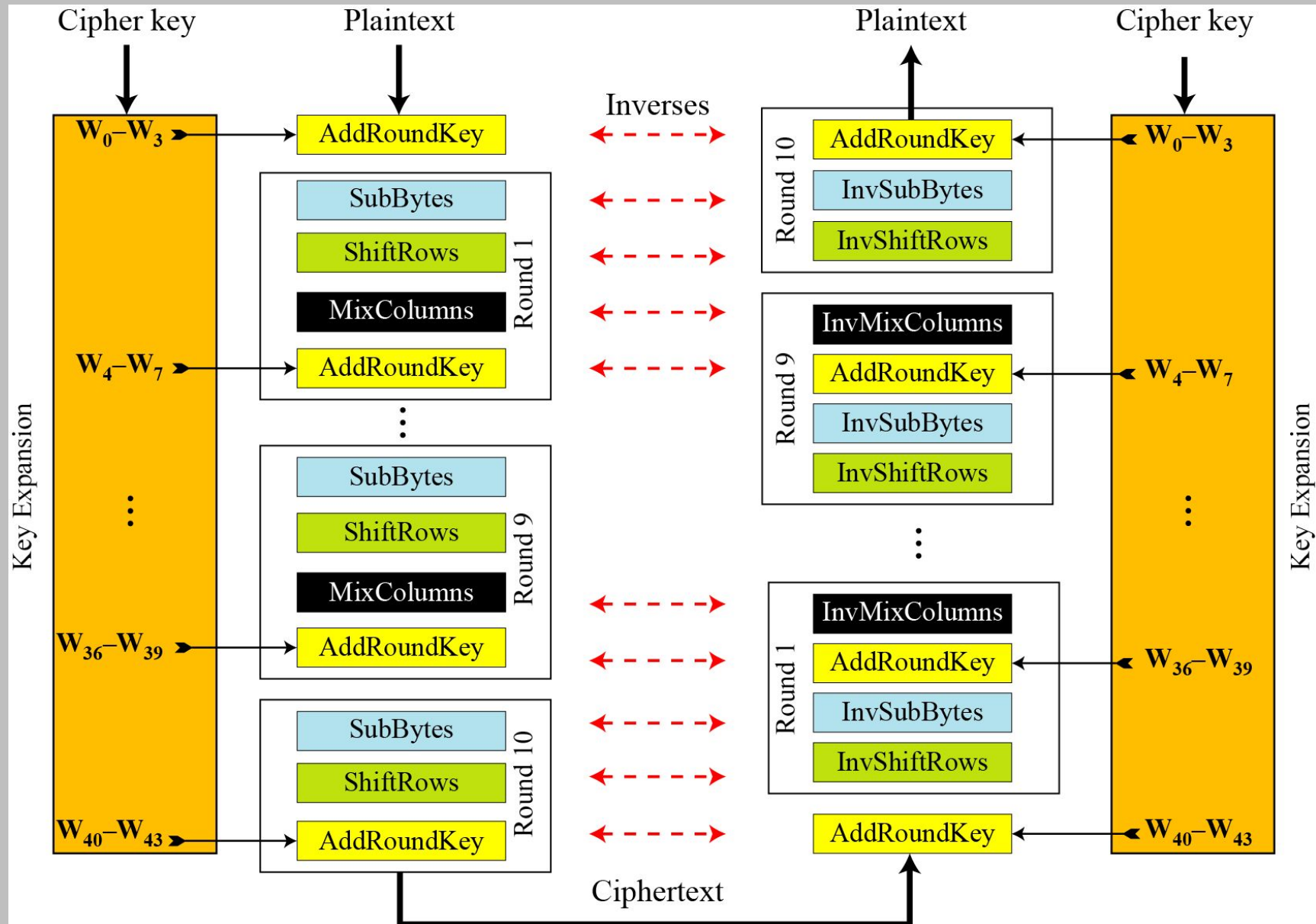
 $\oplus$

a0
fa
fe
17

 $=$

A4
9c
7f
F2

AES Overall Structure

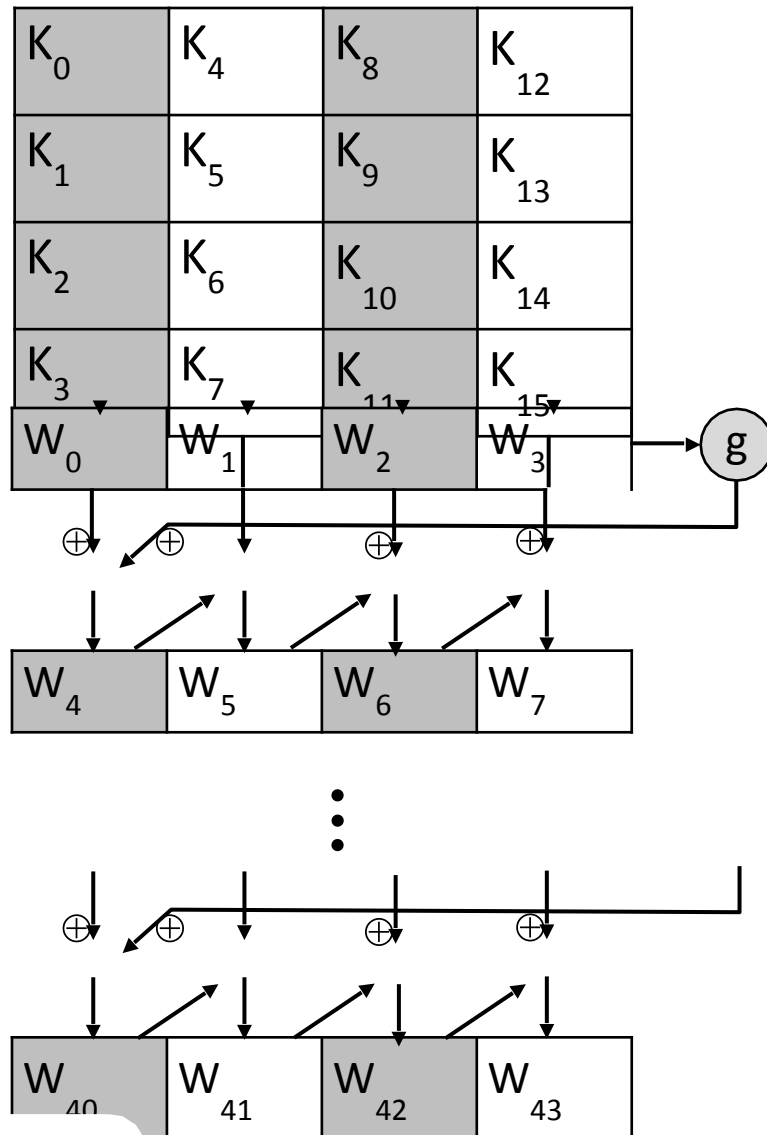


AES key expansion

Words for
each round

Round	Words			
Pre-round	W_0	W_1	W_2	W_3
Round 1	W_4	W_5	W_6	W_7
Round 2	W_8	W_9	W_{10}	W_{11}
...	...			
Round N	W_{40}	W_{41}	W_{42}	W_{43}

AES Key Expansion

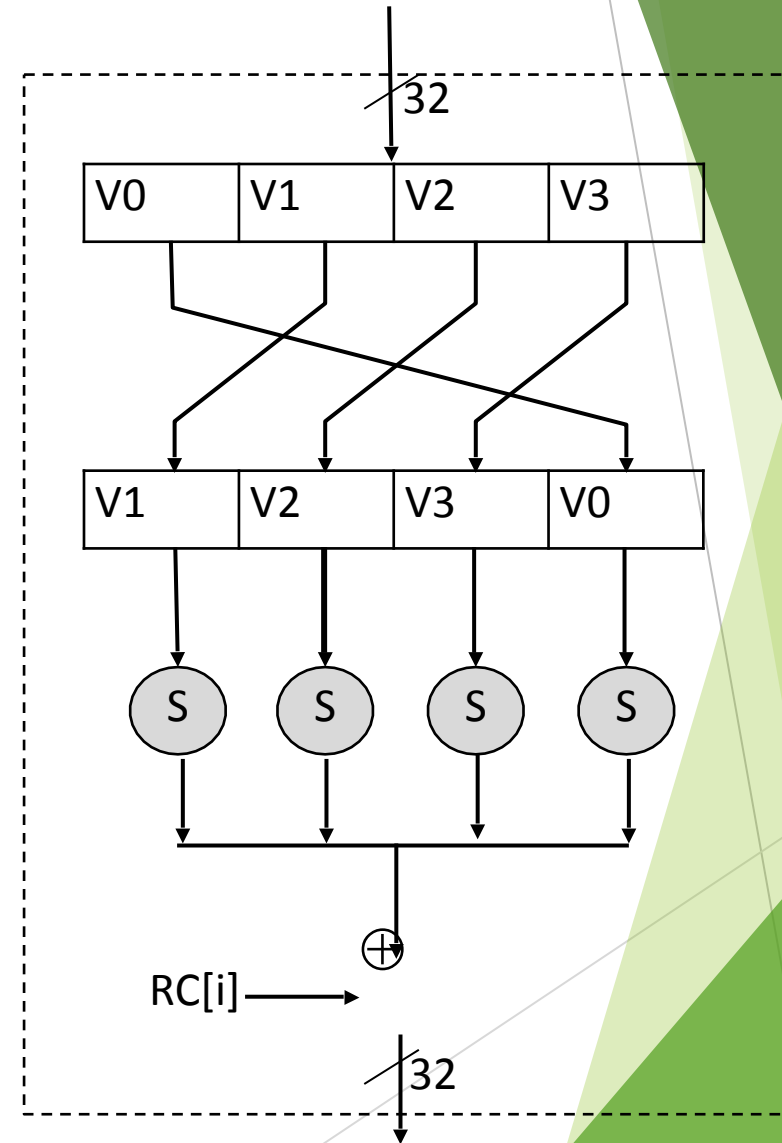


- The AES key expansion algorithm takes as input a four-word (16-byte) key and produces a linear array of **44 words (176 bytes)**.
- Each added word $w[i]$ depends on the immediately preceding word, $w[i - 1]$.
- In three out of four cases, a simple XOR is used.

g Function of Key Expansion

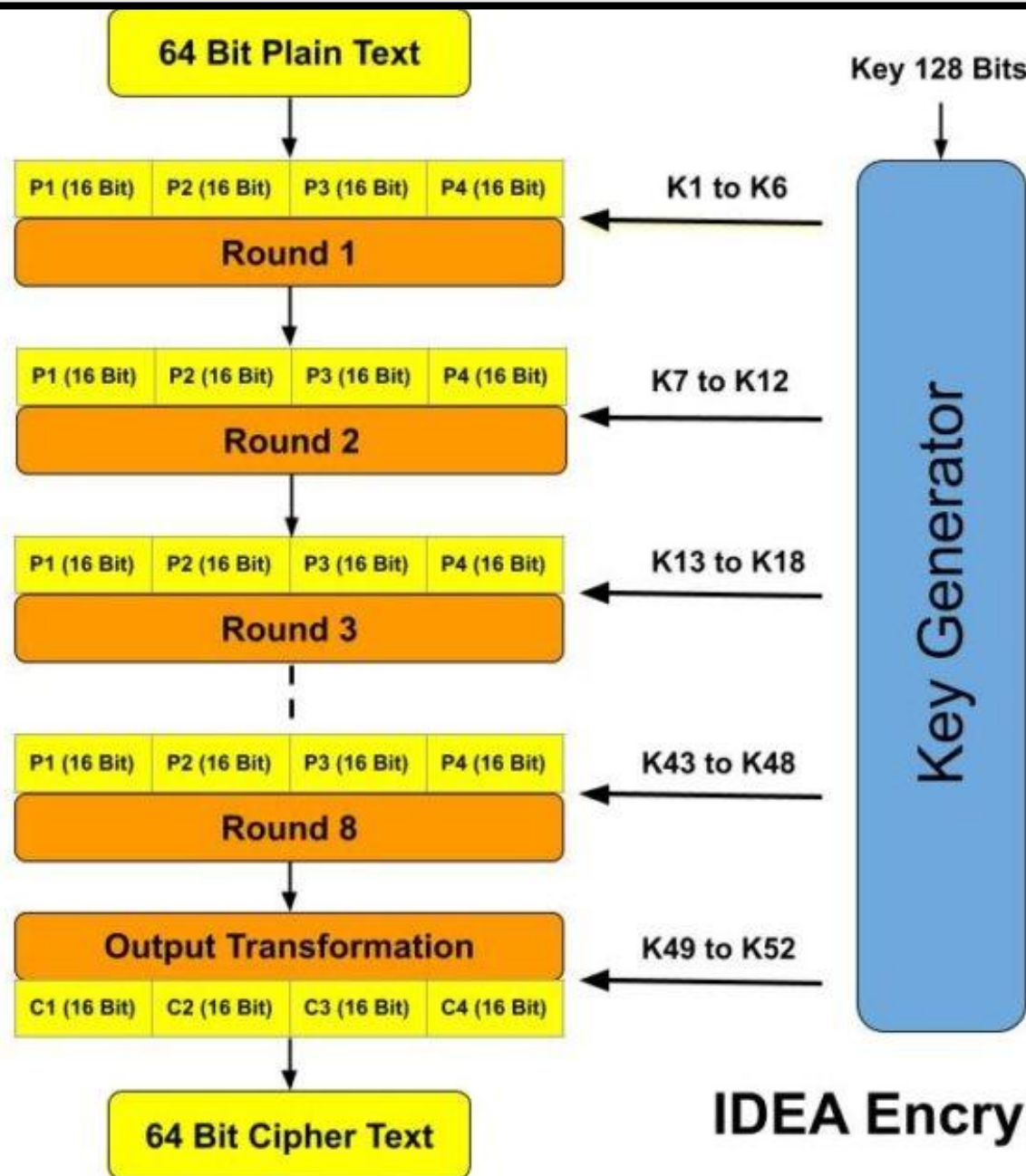
Rcon Table

1	2	3	4	5	6	7	8	9	10
01	02	04	08	10	20	40	80	1b	36
00	00	00	00	00	00	00	00	00	00
00	00	00	00	00	00	00	00	00	00
00	00	00	00	00	00	00	00	00	00



International Data Encryption Algorithm (IDEA)

- Encryption algorithms such as IDEA (**International Data Encryption Algorithm**) provide a secure method of protecting data. IDEA was initially developed as a proprietary cipher by Swiss cryptographers Xuejia Lai and James L. Massey in 1991.
- It was then standardized by the **International Organization for Standardization (ISO) in 1992**.
- It uses a **128-bit key and a series of complex mathematical operations** to scramble plaintext into ciphertext.
- IDEA encryption uses a Feistel network, which divides the plaintext into two halves and then applies a series of rounds to each half, using a combination of XOR, modulo, and multiplication operations.
- *A block cipher and operates on **64-bit blocks of data**.* The 64-bit data block is divided into 16 smaller blocks, and each **has eight rounds of mathematical functions performed on it**.
- The key is **128 bits long**, and IDEA is faster than DES when implemented in software.
- The IDEA algorithm offers different modes similar to the modes described in the DES section, but it is considered to be harder to break than DES because it has a longer key size
- IDEA is used in the **PGP** and other encryption software implementations
- It was thought to replace DES, but it is patented, meaning that licensing fees would have to be paid to use it



- 64-bit input plain text block-divided into 4 part (p1 to p4).
- There are 8 such rounds.
- Key is made up of 128 bits.
- In each round, 6 sub-keys will be produced.
- Each one of the sub-keys includes 16 bits.
- All these sub-keys will be put on the 4 input blocks p1 to p4.
- The last actions include Output Transformation, which usually benefits simply 4 sub-Keys.
- The last result is 4 blocks of ciphertext C1 to C4 (each of 16 bits).
- They are mixed to create the last 64-bit ciphertext block.

IDEA Encryption

IDEA

• How Does IDEA Encryption Work?

- IDEA encryption works by taking 64-bit blocks of data and dividing them into two halves, each containing 32 bits.
- These two halves are then passed through a series of rounds, where they undergo a combination of XOR, modulo, and multiplication operations.
- This process creates a unique key for each block of data.
- The key used in IDEA encryption is a 128-bit key that is created by the user.
- The key is then divided into eight 16-bit subkeys. These subkeys are then used in the encryption process to produce the ciphertext.
- Each round of the IDEA encryption process involves the use of six of these subkeys.

• Strengths of IDEA Encryption:

- IDEA encryption offers a number of strengths, which include:
 - Strong Security: IDEA encryption uses a 128-bit key, making it extremely difficult to crack. It is also resistant to differential and linear attacks.
 - Fast Encryption: IDEA encryption is a fast encryption algorithm, making it ideal for use in real-time applications.

Weaknesses of IDEA Encryption:

IDEA encryption also has a few weaknesses, which include:

- Vulnerable to Brute-Force Attacks: Although IDEA encryption is considered secure, it is still vulnerable to brute-force attacks.
- Limited Key Size: IDEA encryption uses a 128-bit key, which is smaller than some other encryption algorithms.

Applications of IDEA Encryption:

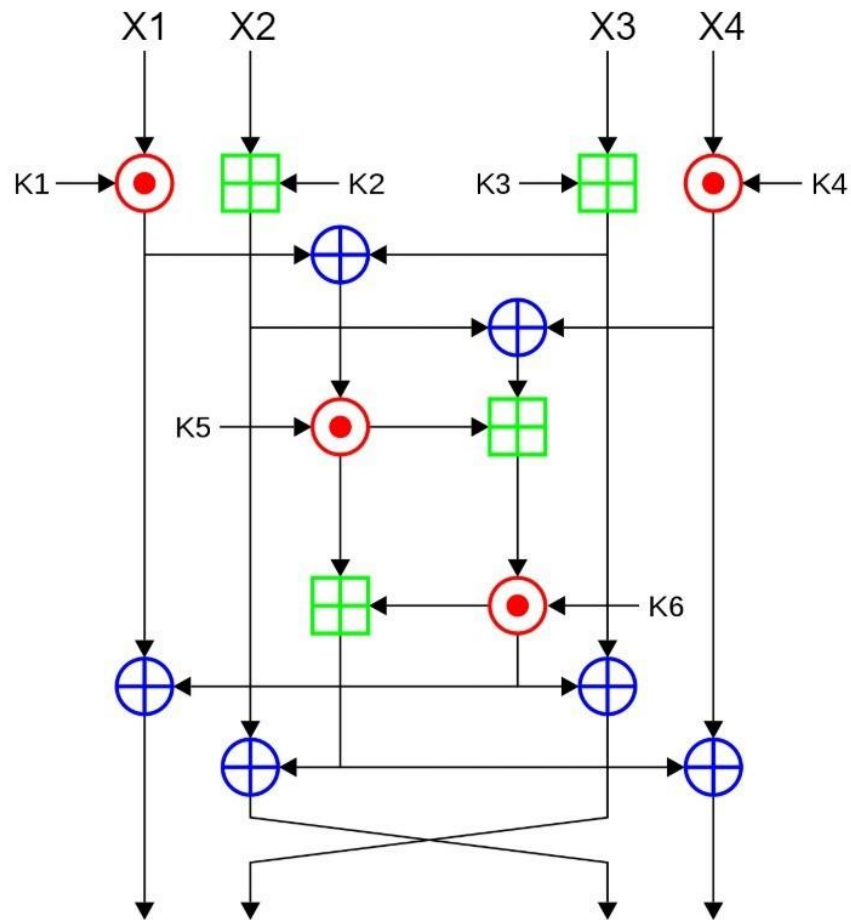
IDEA encryption has a wide range of applications, including:

- Secure Communication: IDEA encryption can be used to secure communication between two parties, such as in email or instant messaging applications.
- Data Storage: IDEA encryption can be used to encrypt sensitive data stored on a hard drive or other storage devices.
- Digital Signatures: IDEA encryption can be used to create digital signatures, which can be used to verify the authenticity of digital documents.

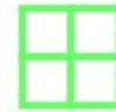
Summary:

- IDEA encryption is a widely used encryption algorithm that offers strong security and fast encryption.
- It is a symmetric-key block cipher that operates on 64-bit blocks of data and uses a 128-bit key.
- IDEA encryption is widely used in secure communication, data

International Data Encryption Algorithm(IDEA)



Where,



= Modular Addition



= Modular Multiplication



= Bitwise XOR

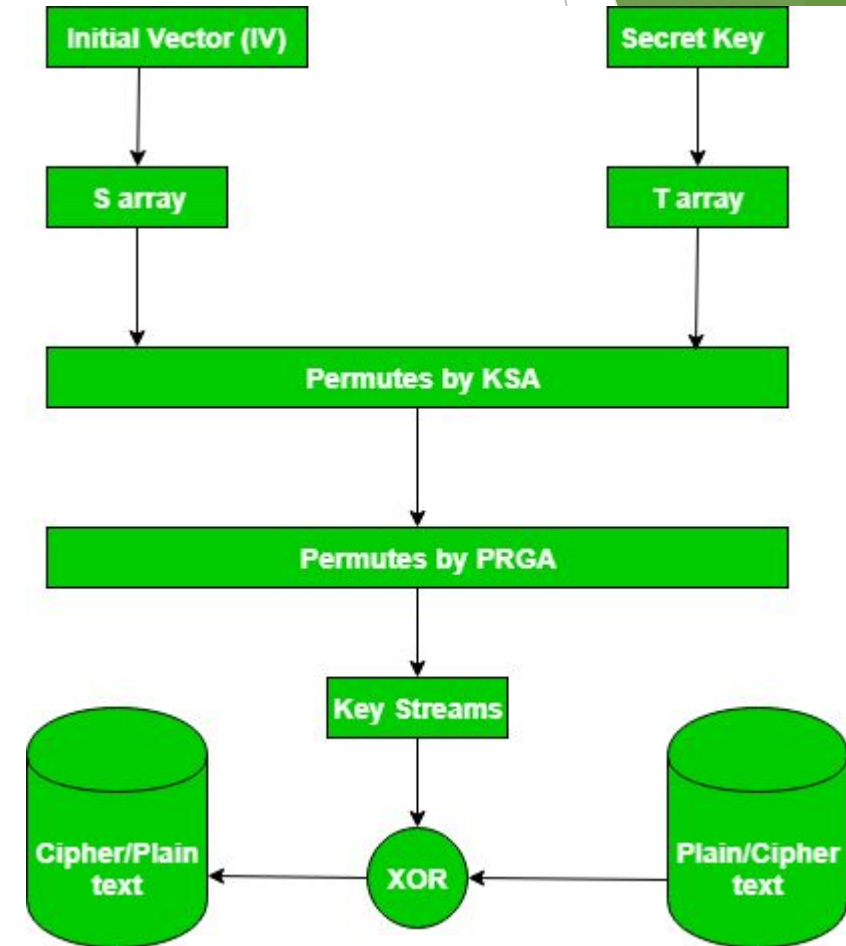
RC4

RC4 is one of the most commonly implemented stream ciphers. It has a variable key size, is used in the SSL protocol, and was (improperly) implemented in the 802.11 WEP protocol standard.

RC4 was developed in 1987 by Ron Rivest and was considered a trade secret of RSA Data Security, Inc. until someone posted the source code on a mailing list.

Since the source code was released nefariously, the stolen algorithm is sometimes implemented and referred to as ArcFour or ARC4 because the title RC4 is trademarked.

The algorithm is very simple, fast, and efficient, which is why it became so popular.



RC4

```
for i = 0 to 255 do
  S[i] = i;
  T[i] = K[i mod keylen];

/* Initial Permutation of S */
j = 0;
for i = 0 to 255 do
  j = (j + S[i] + T[i]) mod 256;
  Swap (S[i], S[j]);

/* Stream Generation */
i, j = 0;
while (true)
  i = (i + 1) mod 256;
  j = (j + S[i]) mod 256;
  Swap (S[i], S[j]);
  t = (S[i] + S[j]) mod 256;
  k = S[t];
```

RC5

RC5 is a block cipher that has a variety of parameters it can use for block size, key size, and the number of rounds used. It was created by Ron Rivest and analyzed by RSA Data Security, Inc.

The block sizes used in this algorithm are 32, 64, or 128 bits, and the key size goes up to 2,048 bits. The number of rounds used for encryption and decryption is also variable. The number of rounds can go up to 255.

RC5 is a block cipher and addresses two word blocks at a time.

Depending on input plain text block size, number of rounds and key size, various instances of RC5 can be defined and each instance is denoted as RC5-w/r/b where w=word size in bits, r=number of rounds and b=key size in bytes. Allowed values are:

Parameter	Possible Value
block/word size (bits)	16, 32, 64
Number of Rounds	0 – 255
Key Size (bytes)	0 – 255

RC5

How Does RC5 Encryption Work?

1. **Key Setup:** In this step, the key is processed to generate the key schedule. The key schedule is a set of subkeys that are used in the encryption and decryption process.
2. **Block Encryption:** The plaintext is divided into blocks, and each block is encrypted using the key schedule.
3. **Round Function:** The encryption process is performed in rounds, and each round consists of a series of operations, including bit rotations, XOR operations, and modular additions.
4. **Finalization:** After all the rounds have been completed, the final cipher text is generated.

Implementing RC5 Encryption: RC5 encryption can be implemented in software or hardware. The implementation process involves the following steps:

1. **Key Generation:** The key is generated using a random number generator or a key derivation function.
2. **Key Setup:** The key is processed to generate the key schedule.
3. **Encryption:** The plaintext is divided into blocks, and each block is encrypted using the key schedule.
4. **Decryption:** The cipher text is decrypted using the same key schedule that was used for encryption.
5. **Error Checking:** Error checking is performed to ensure that the decrypted plaintext is the same as the original plaintext.

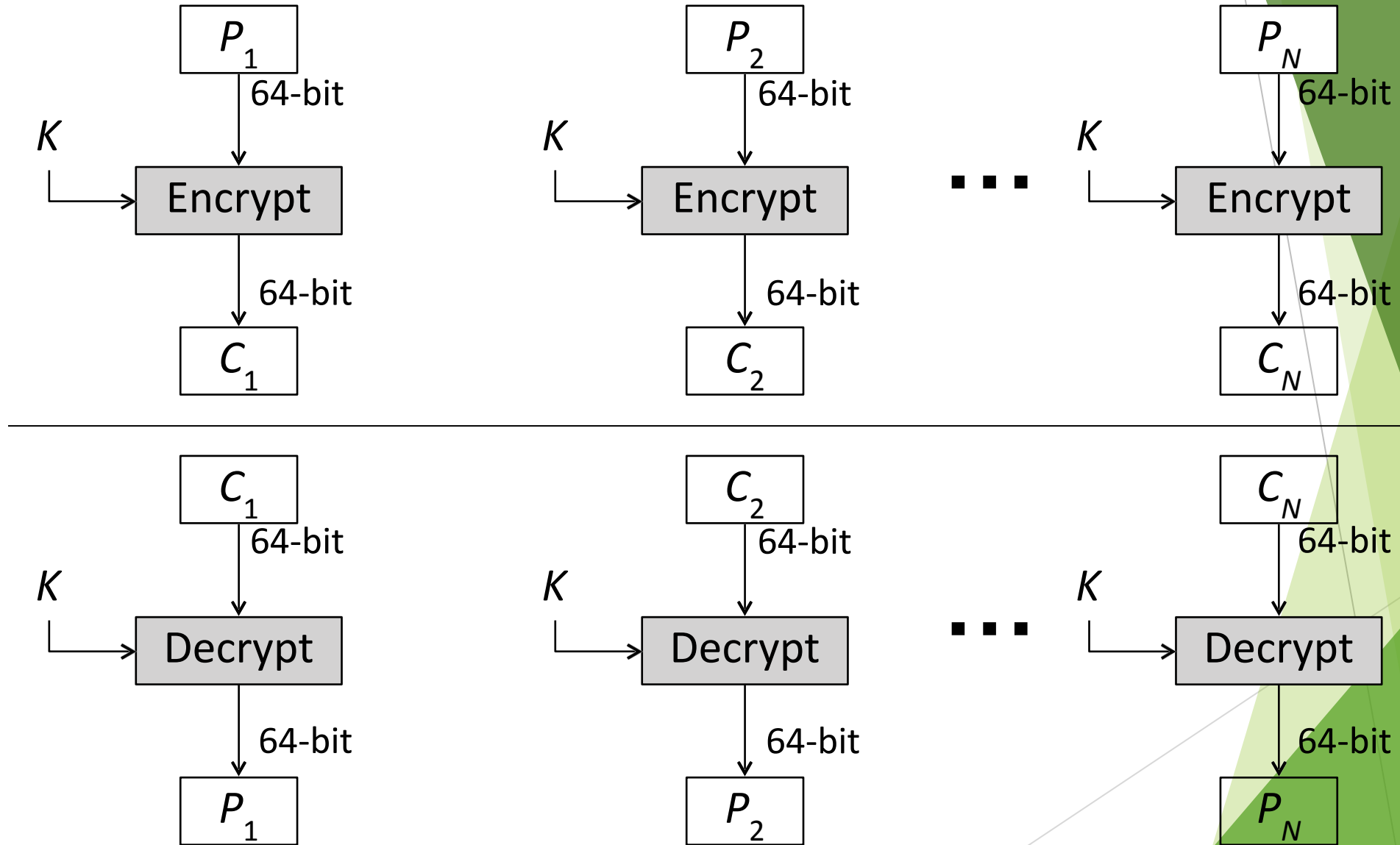
Block Cipher Modes of Operations

- To apply a block cipher in a **variety of applications**, five "modes of operation" have been defined.
 1. Electronic Code Book (ECB)
 2. Cipher Block Chaining (CBC)
 3. Cipher Feedback (CFB)
 4. Output Feedback (OFB)
 5. Counter (CTR)
- The **five modes** are intended to cover a wide variety of applications of encryption for which a block cipher could be used.
- These modes are intended for use with any **symmetric block cipher**, including triple DES and AES.

Block Cipher Modes of Operations

- **Block cipher:** operates on fixed length b -bit input to produce b -bit ciphertext.
- What about encrypting plaintext longer than b bits?
- Break plaintext into b -bit blocks (padding if necessary) and apply cipher on each block.

1. ECB Encryption & Decryption



Electronic Code Book (ECB)

- In ECB Mode Plaintext handled **one block at a time** and each block of plaintext is encrypted using the same key.
- The term **codebook** is used because, for a **given key**, there is a **unique ciphertext** for every b-bit block of plaintext.

$$C_j = E(K, P_j) \quad j = 1, \dots, N$$

$$P_j = D(K, C_j) \quad j = 1, \dots, N$$

• ECB Advantages:

- No block synchronization between sender and receiver is required.
 - OK if some blocks are lost in transit.
- Bit errors caused by noisy channels only affect the corresponding block but not succeeding blocks.
- Block cipher operating can be parallelized.

• ECB Disadvantages:

- Identical plaintexts result in identical ciphertexts.
- An attacker recognizes if the same message has been sent twice simply by looking at the ciphertext.
- Plaintext blocks are encrypted independently of previous blocks.
 - An attacker may reorder ciphertext blocks which results in valid plaintext.

Substitution Attack on

ECB • Consider an **electronic bank transfer**

1	2	3	4	5
Sending Bank A	Sending Account #	Receiving Bank B	Receiving Account #	Amount \$

- The attacker sends \$1.00 transfers from his account at bank A to his account at bank B repeatedly.
- He can check for ciphertext blocks that repeat, and he stores blocks 1,3 and 4 of these transfers.
- He now simply replaces block 4 of other transfers with the block 4 that he stored before.
- All transfers from some account of bank A to some account of bank B are redirected to go into the attacker's B account.

Strength: it's simple.

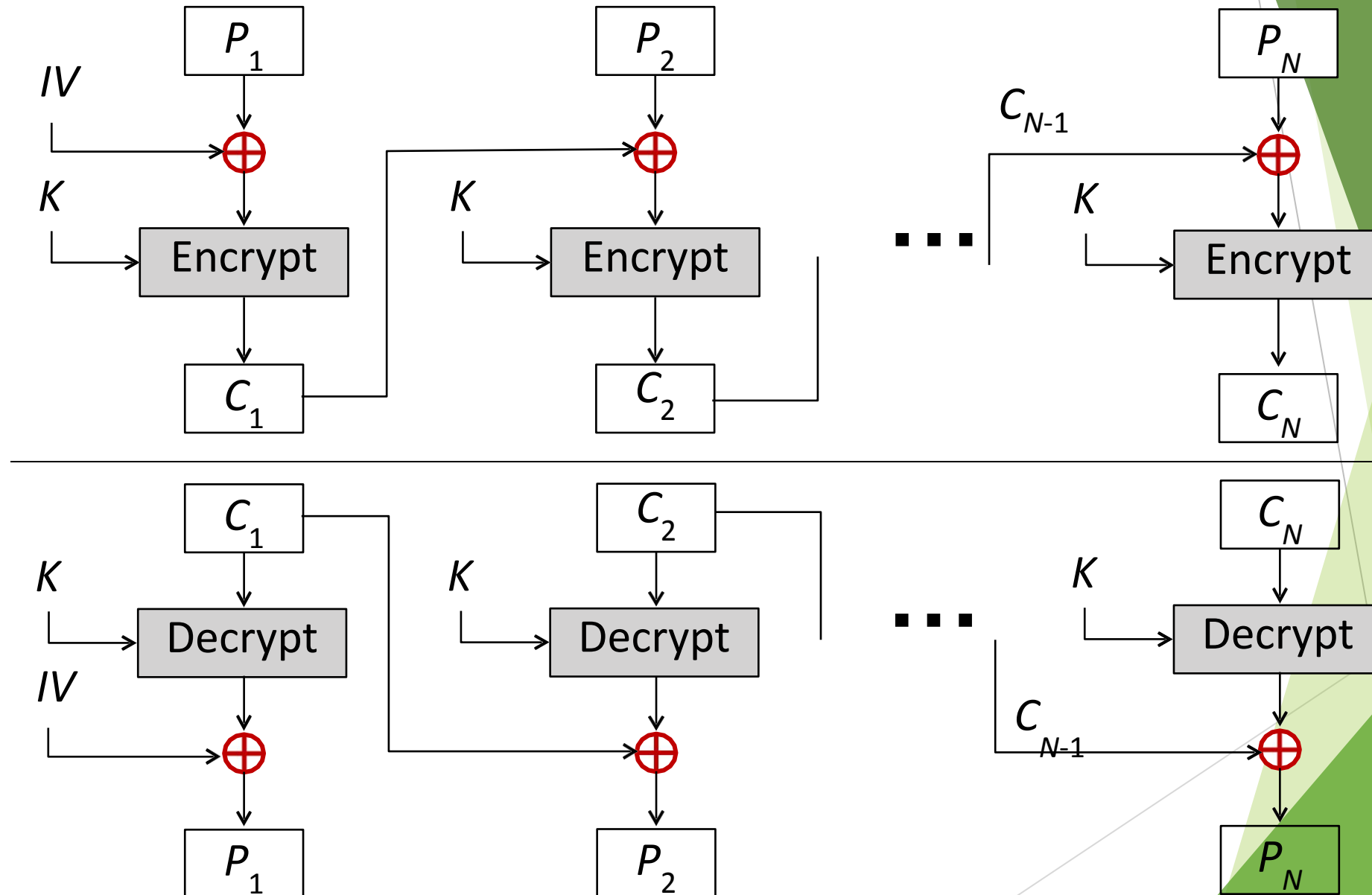
Weakness:

Problem: with long message, repetition in plaintext may cause repetition in ciphertext.

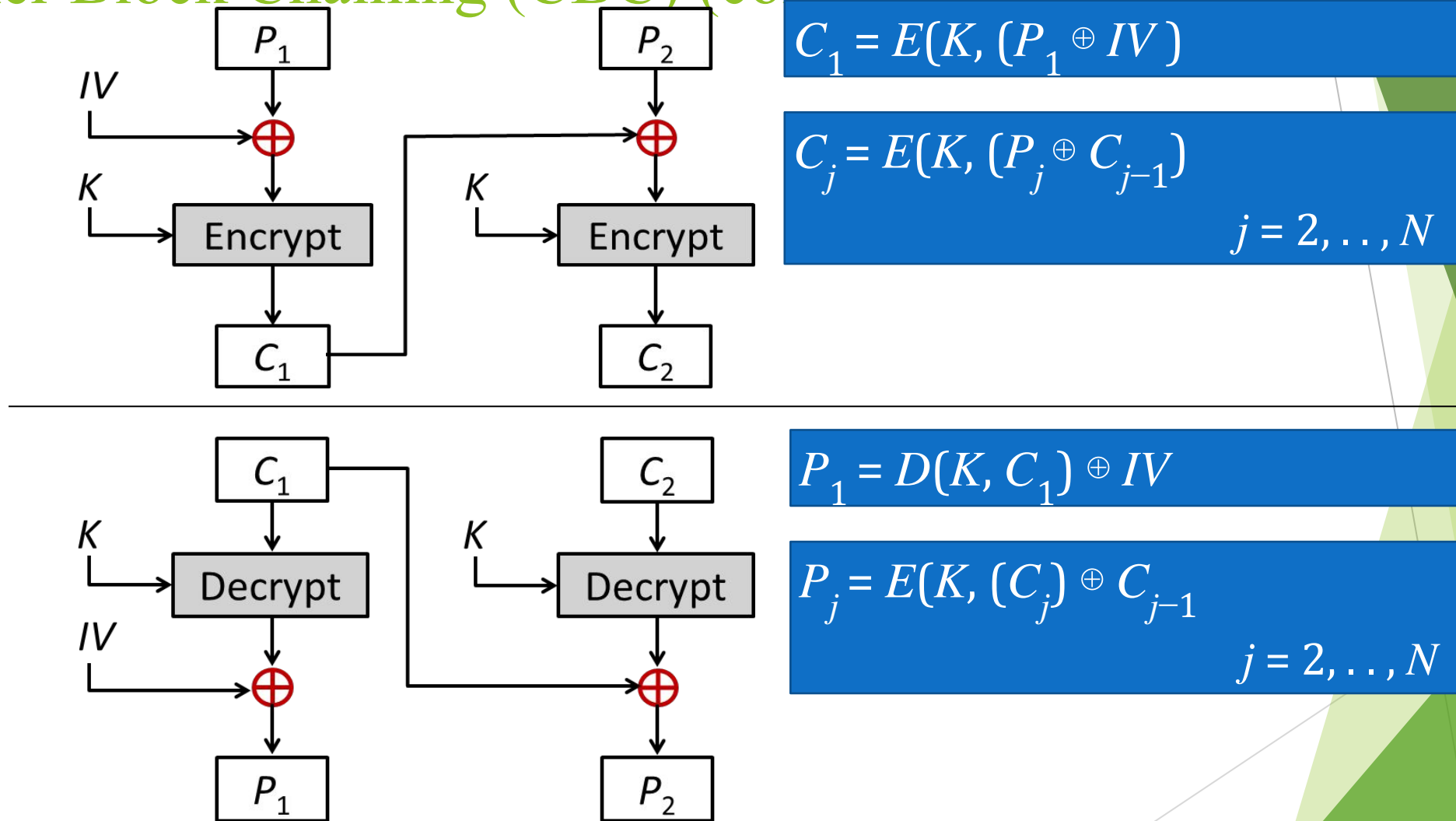
Typical application:

Secure transmission of short pieces of information (e.g. a temporary encryption key).

2. CBC - Encryption & Decryption



Cipher Block Chaining (CBC) (cont...)



Cipher Block Chaining (CBC) (cont...)

- CBC is a technique in which the same plaintext block, if repeated, produces different ciphertext blocks.
- In this scheme, the input to the encryption algorithm is the XOR of the current plaintext block and the preceding ciphertext block and the same key is used for each block.
- To produce the first block of ciphertext, an initialization vector (IV) is XORed with the first block of plaintext.
- Initialisation Vector (IV) must be known by sender/receiver, but it should be kept secret from attacker.
- On decryption, the IV is XORed with the output of the decryption algorithm to recover the first block of plaintext.

Cipher Block Chaining (CBC)

Substitution Attack on CBC

- Consider the last example (electronic bank transfer).
- If the **IV is properly chosen** for every wire transfer, the **attack will not work** at all.
- If the **IV is kept the same** for several transfers, the **attacker** would **recognize the transfers** from his account at bank A to bank B.

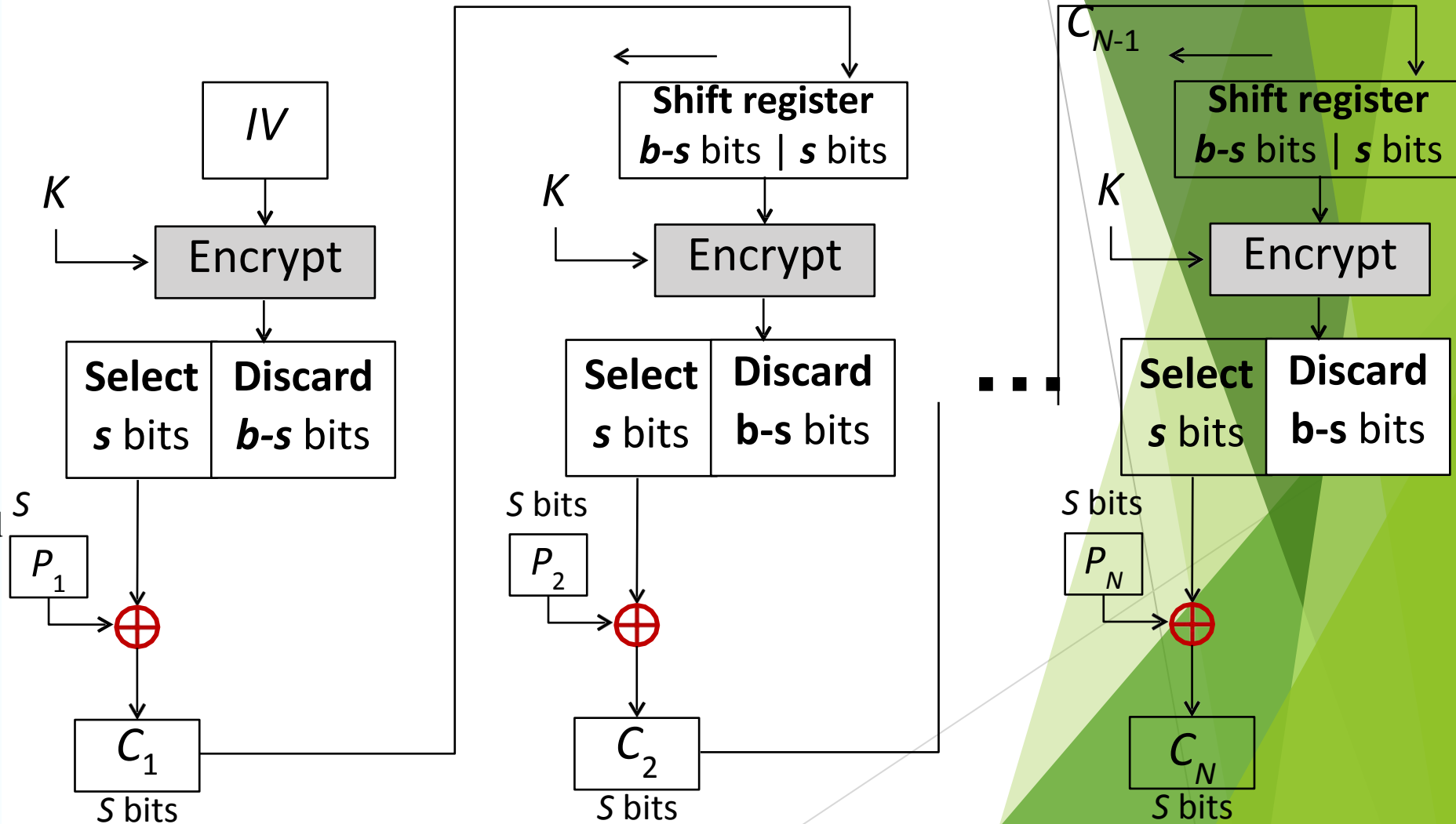
Strength: because of the chaining mechanism of CBC, it is an appropriate mode for encrypting messages of length greater than b bits.

Typical application:

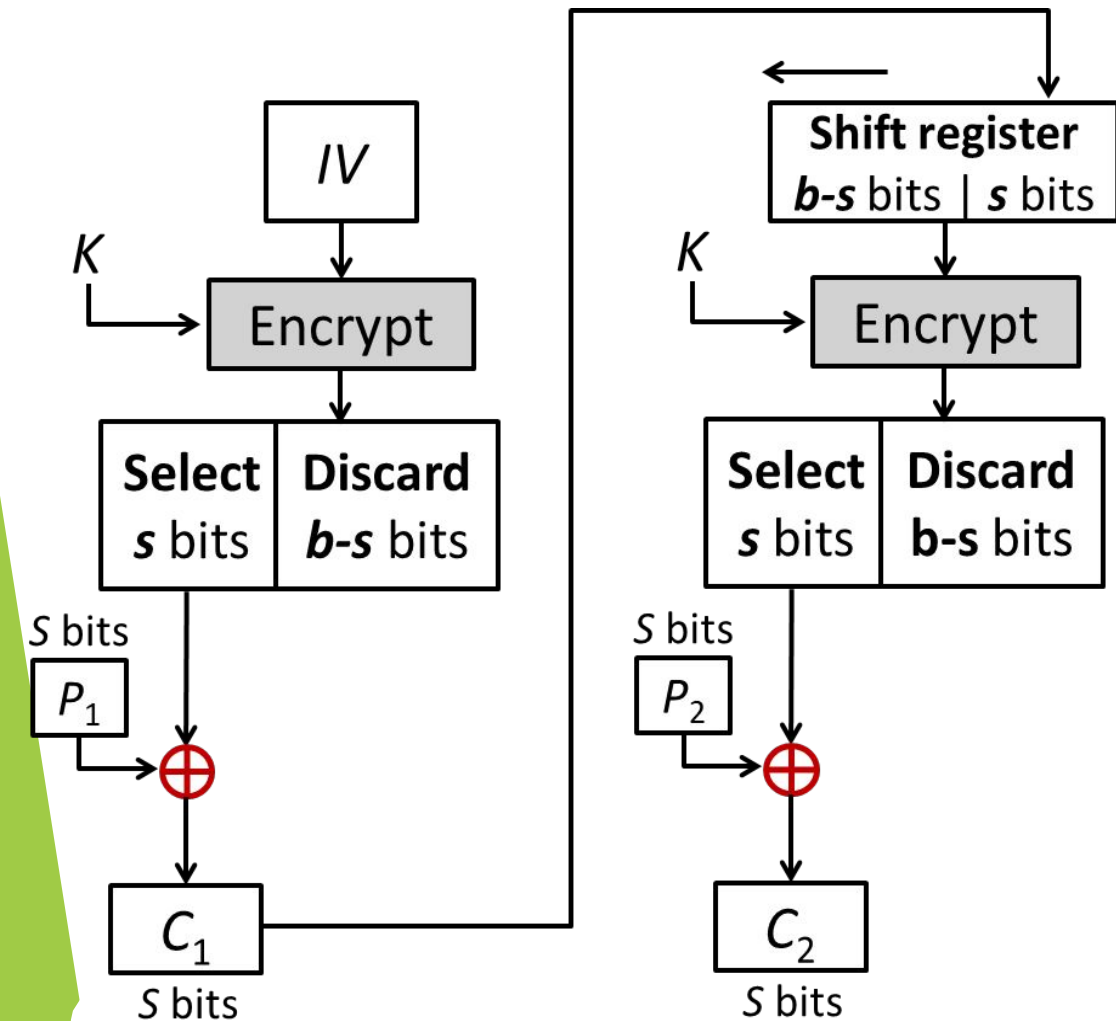
General-purpose block oriented transmission
Authentication

Cipher Feedback Mode CFB Encryption

For AES, DES, or any block cipher, encryption is performed on a block of b bits. In DES, $b = 64$ and in AES, $b = 128$. However, it is possible to **convert a block cipher into a stream cipher**, using cipher feedback (CFB) mode, output feedback (OFB) mode, and counter (CTR) mode. A stream cipher eliminates the need to pad a message to be an integral number of blocks.



CFB Encryption (cont...)



$$I_1 = IV$$

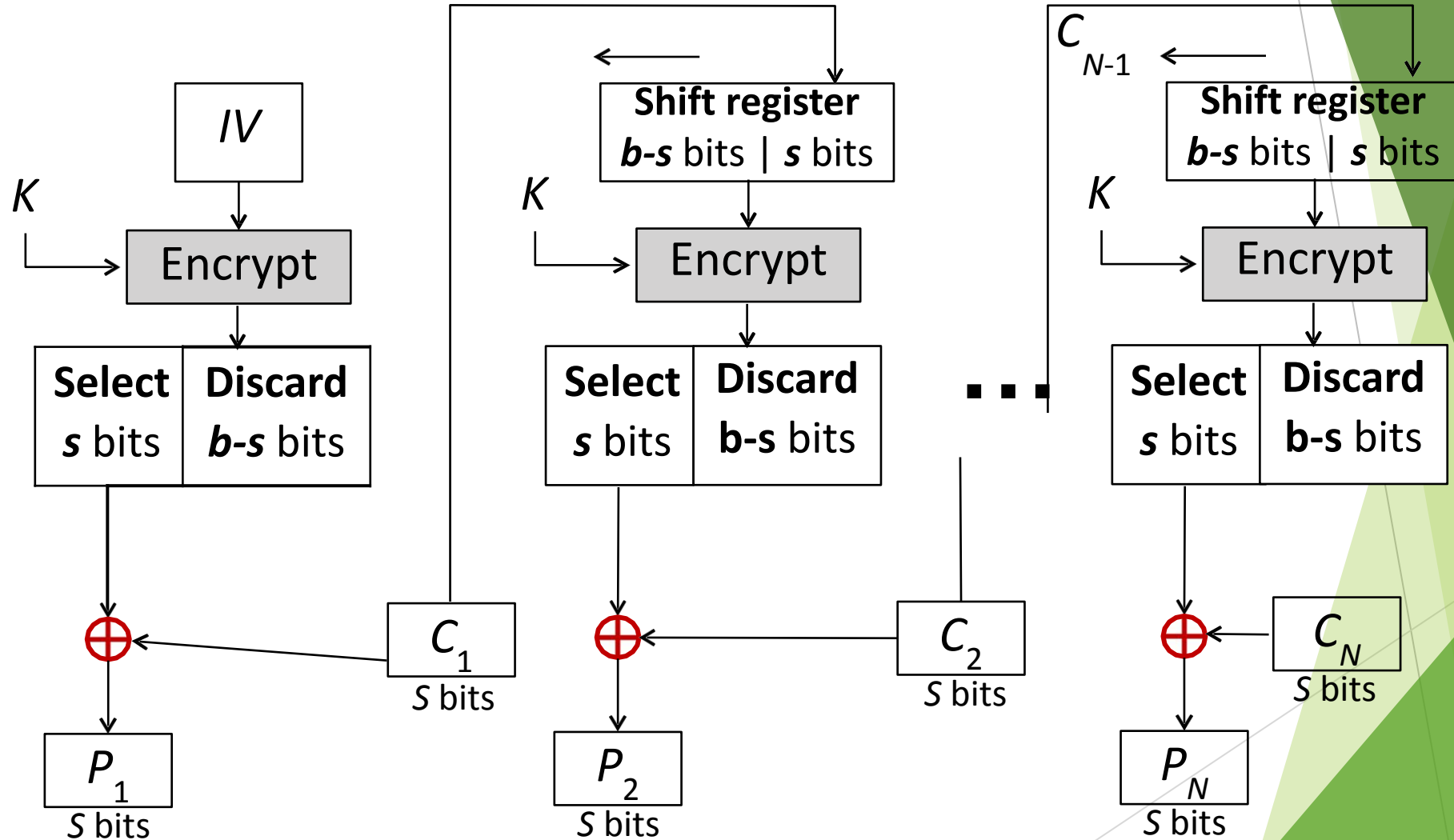
$$I_j = LSB_{b-s}(I_{j-1}) || C_{j-1}$$

$$j = 2, \dots, N$$

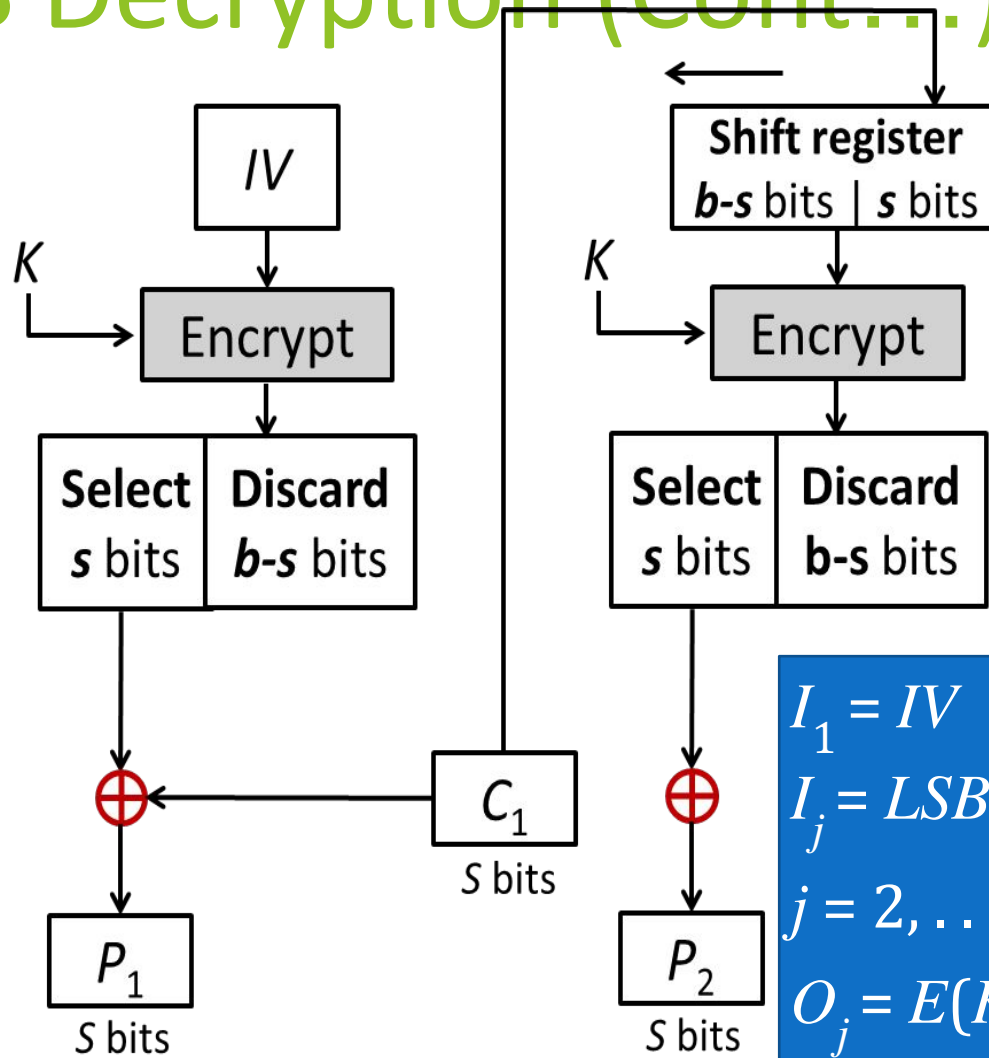
$$O_j = E(K, I_j) \quad j = 1, \dots, N$$

$$C_j = P_j \oplus MSB_s(O_j)$$

CFB Decryption



CFB Decryption (Cont...)



$$I_1 = IV$$

$$I_j = LSB_{b-s}(I_{j-1}) || C_{j-1}$$

$$j = 2, \dots, N$$

$$O_j = E(K, I_j)$$

$$P = C \oplus MSB(O)$$

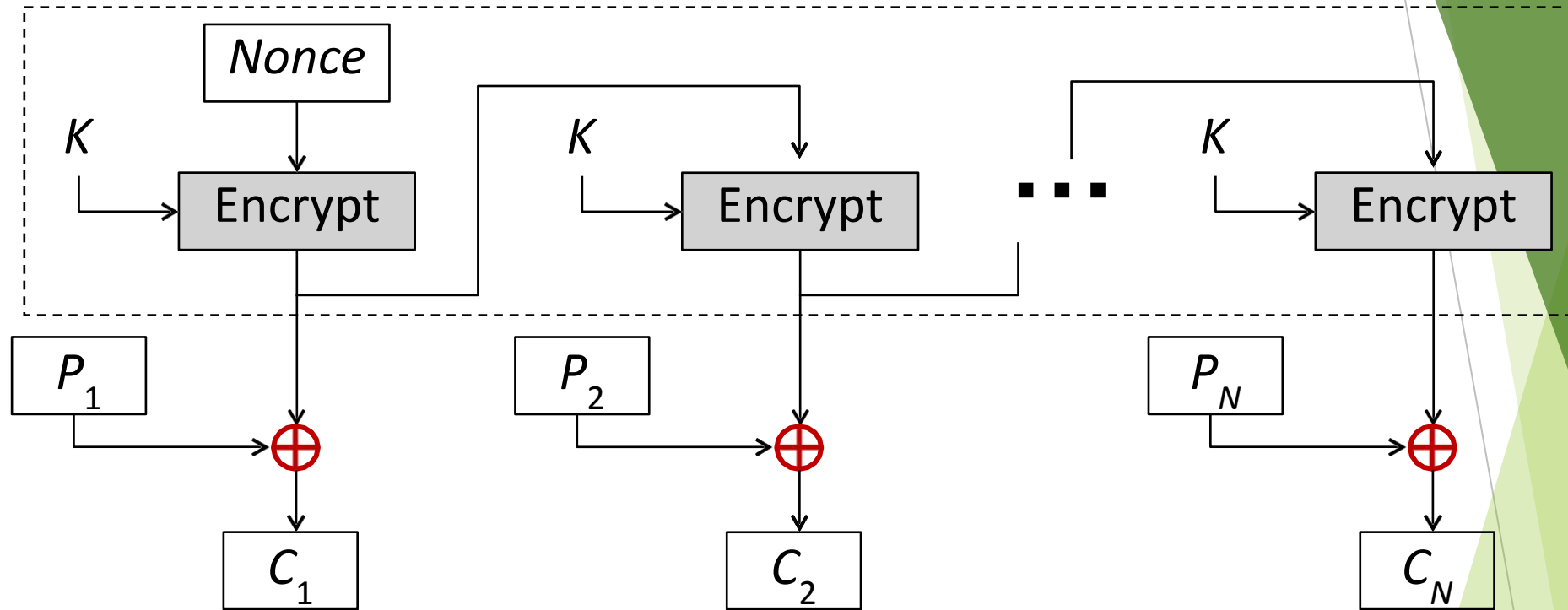
$$j = 1, \dots, N$$

$$j = 1, \dots, N$$

Cipher Feedback Mode (CFB)

- The input to the encryption function is a **b-bit shift register** that is initially set to some initialization vector (IV).
- The **leftmost (most significant) s bits** of the output of the encryption function are **XORed** with the first segment of plaintext **P1** to produce the first unit of ciphertext **C1**, which is then transmitted.
- In addition, the contents of the **shift register are shifted left by s bits**, and **C1 is placed in the rightmost** (least significant) s bits of the shift register.
- For decryption, the same scheme is used, except that the received ciphertext unit is XORed with the output of the encryption function to produce the plaintext unit.

4. OFB Encryption



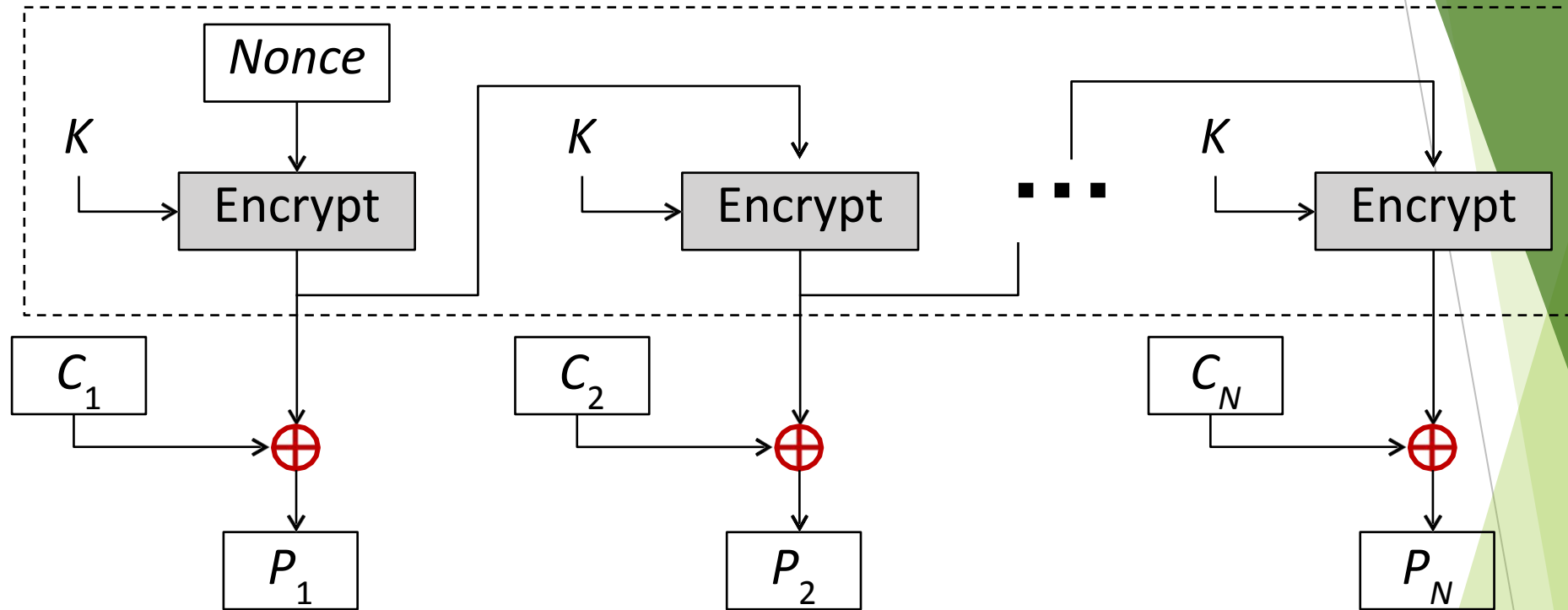
$$I_1 = \text{Nonce}$$

$$I_j = O_{j-1} \quad j = 2, \dots, N$$

$$O_j = E(K, I_j) \quad j = 1, \dots, N$$

$$C_j = P_j \oplus O_j \quad j = 1, \dots, N-1$$

OFB Decryption



$$I_1 = \text{Nonce}$$

$$I_j = O_{j-1} \quad j = 2, \dots, N$$

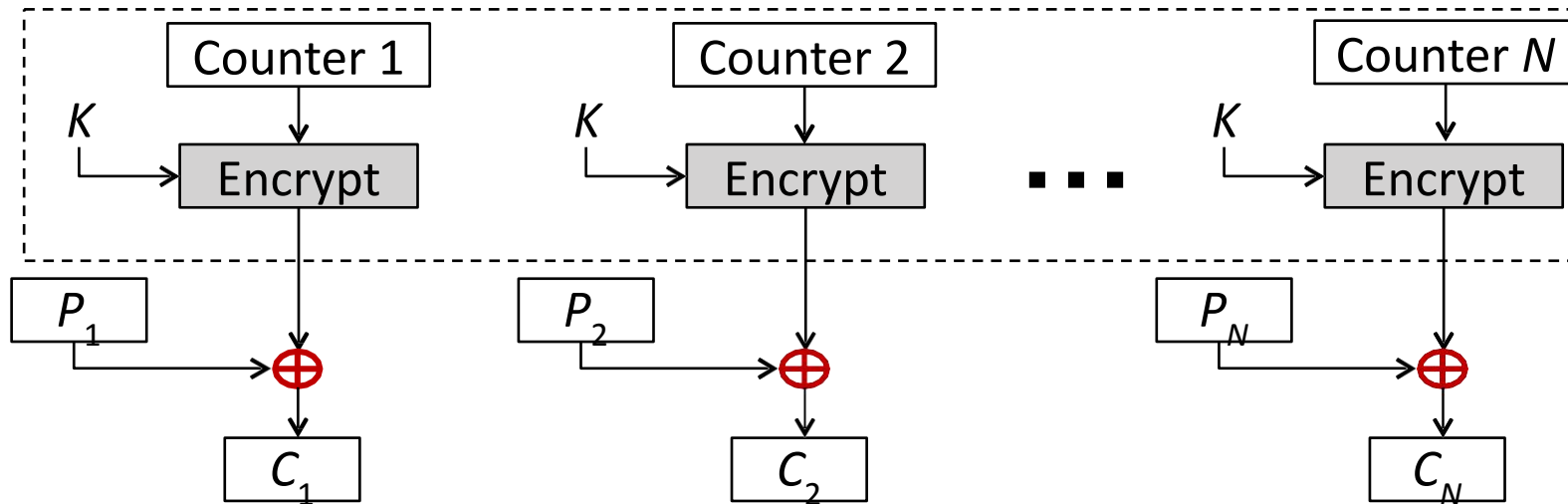
$$O_j = E(K, I_j) \quad j = 1, \dots, N$$

$$P_i = C_i \oplus O_i \quad i = 1, \dots, N-1$$

Output Feedback Mode(OFB) (cont..)

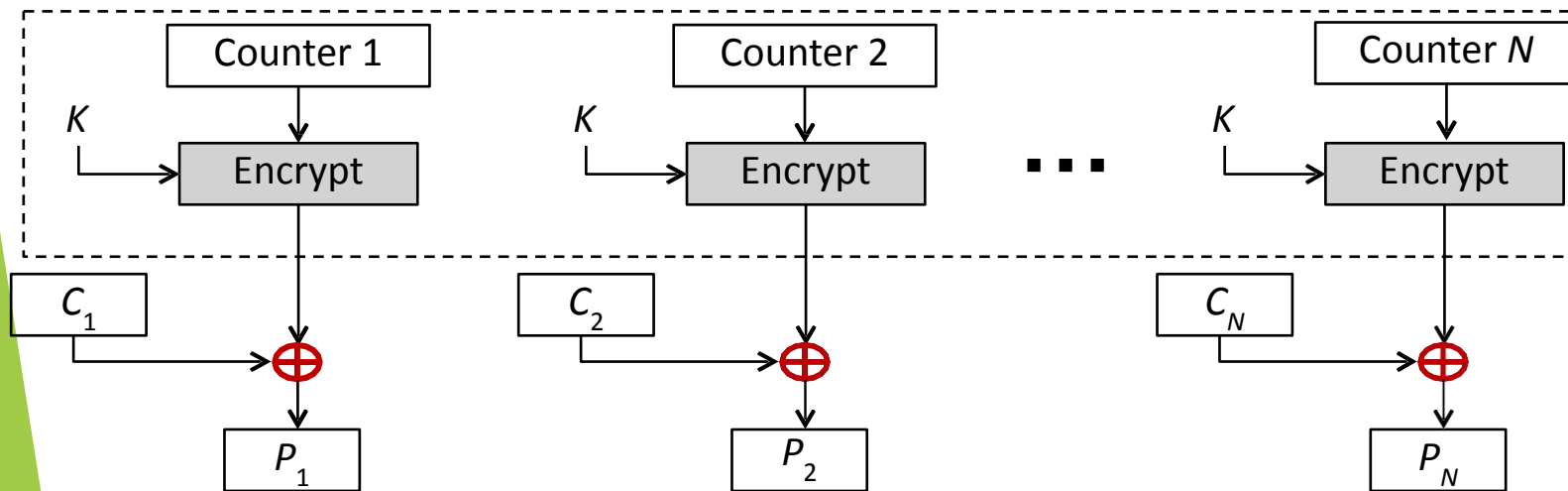
- The output feedback (OFB) mode is similar in structure to that of CFB.
- For OFB, the output of the encryption function is fed back to become the input for encrypting the next block of plaintext.
- In CFB, the output of the XOR unit is fed back to become input for encrypting the next block.
- The other difference is that the **OFB mode operates on full blocks** of plaintext and ciphertext, whereas **CFB operates on an s-bit** subset.
- **Nonce:** A time-varying value that has at most a negligible chance of repeating, for example, a **random value** that is generated anew for each use, a timestamp, a sequence number, or some combination of these.
- Each bit in the ciphertext is independent of the previous bit or bits.
- This avoids error propagation.
- Pre-compute of forward cipher is possible.

5. CTR Encryption(CTR)



$$C_j = P_j \oplus E(K, T_j) \quad j = 1, \dots, N$$

CTR Decryption



$$P_j = C_j \oplus E(K, T_j) \quad j = 1, \dots, N$$

- Counter (CTR) mode has increased recently with applications to ATM (asynchronous transfer mode) network security and IP sec (IP security).

- A counter equal to the plaintext block size is used.
- The counter value must be different for each plaintext block that is encrypted.
- Typically, the counter is initialized to some value and then incremented by 1 for each subsequent block.

Advantages of the CTR Mode

Strengths:

- Needs only the encryption algorithm.
- Random access to encrypted data blocks.
- blocks can be processed (encrypted or decrypted) in parallel.

Counter must be

- Simple and fast encryption/decryption.
- Must be unknown and unpredictable.
- pseudo-randomness in the key stream is a goal.

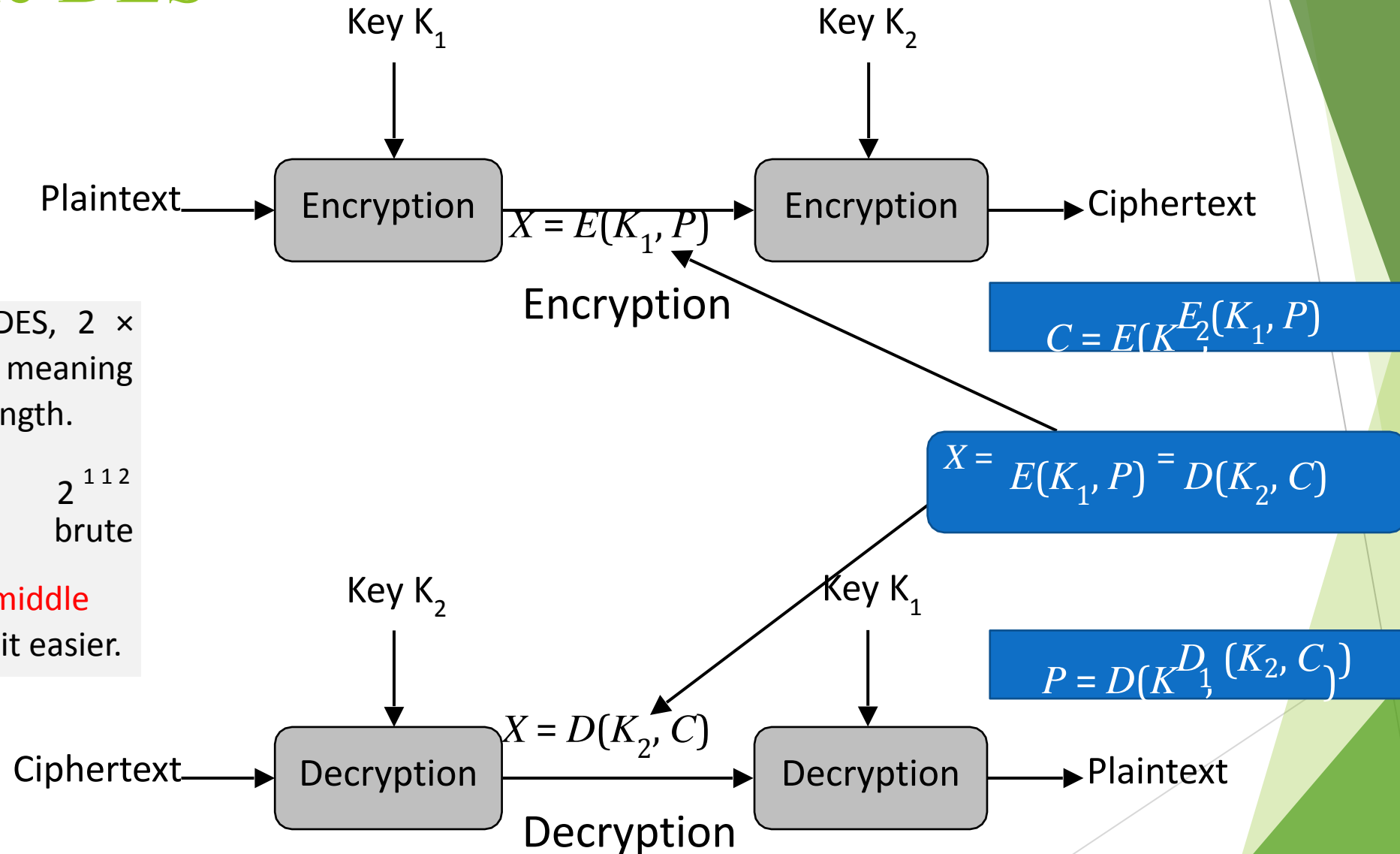
Summary of all modes

Operation Mode	Description	Type of Result
ECB	Each n-bit block is encrypted independently with same key.	Block Cipher
CBC	Same as ECB, but each block is XORed with previous cipher text.	Block Cipher
CFB	Each s-bit block is XORed with s-bit key which is part of previous cipher text.	Stream Cipher
OFB	Same as CFB, but input to the encryption is preceding encryption output.	Stream Cipher
CTR	Same as OFB, but a counter is used instead of nonce.	Stream Cipher

Multiple Encryption

- Given the potential vulnerability of DES to a brute-force attack, there has been considerable interest in finding an alternative.
- For DES requires 2^{56} operations for brute force attack.
- One approach is to design a completely new algorithm, of which AES is a prime example.
- Another alternative, which would preserve the existing investment in software and equipment, is to use multiple encryption with DES and multiple keys.

Double DES

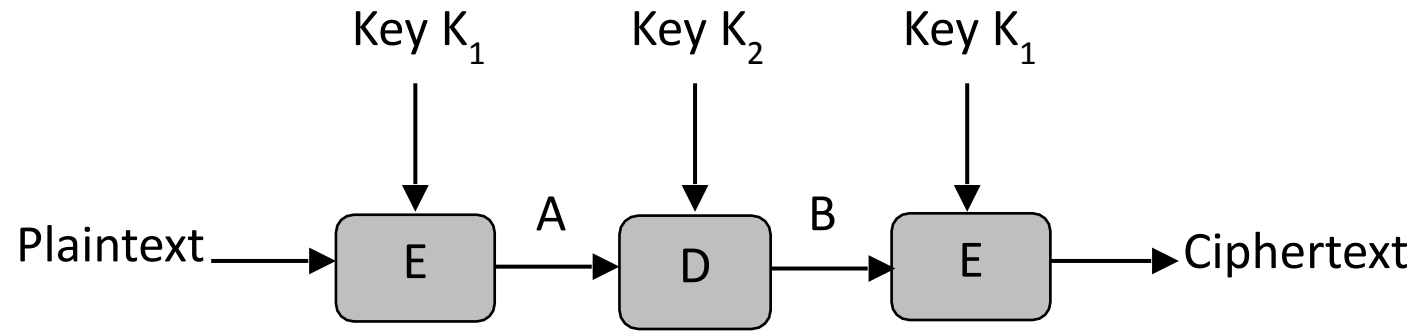


For double DES, $2 \times$ 56-bit keys, meaning 112-bit key length.

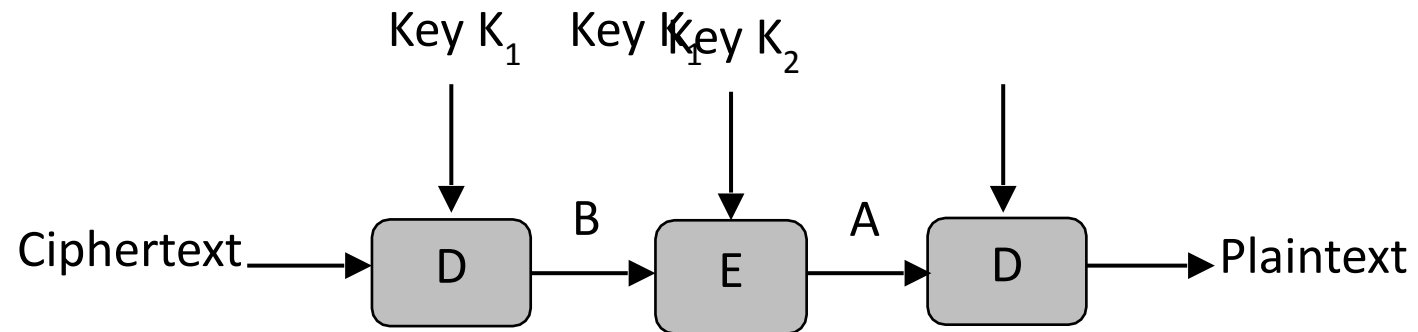
Requires 2^{112} operations for brute force

Meet-in-the-middle attack makes it easier.

Triple DES



$$C = E(K_1, D(K_2, E(K_1, P)))$$

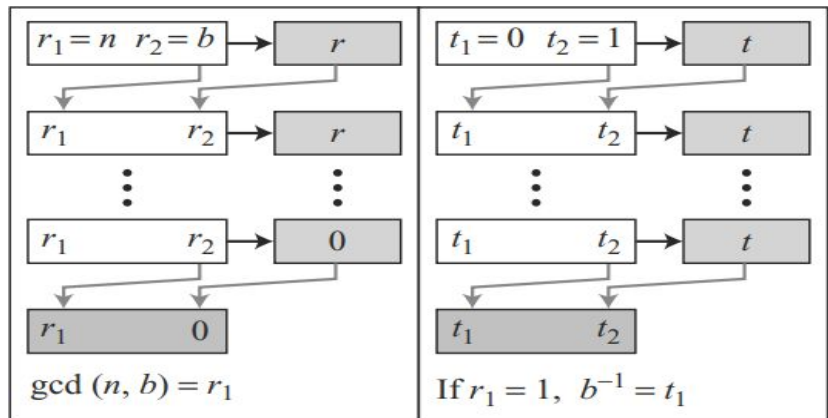


$$P = D(K_1, E(K_2, D(K_1, C)))$$

Asymmetric Key Cryptography



- **Public Key Cryptosystems with Applications**
- **Requirements and Cryptanalysis**
- **RSA algorithm**
- **RSA computational aspects and security**
- **Diffie-Hillman Key Exchange algorithm**
- **Man-in-Middle attack**



a. Process

Extended Euclidean algorithm to find the multiplicative inverse

Find the multiplicative inverse of 11 in $\mathbb{Z}_{26}$

Solution

We use a table similar to the one we used before with $r_1 = 26$ and $r_2 = 11$. We are interested only in the value of t .

q	r_1	r_2	r	t_1	t_2	t
2	26	11	4	0	1	-2
2	11	4	3	1	-2	5
1	4	3	1	-2	5	-7
3	3	1	0	5	-7	26
	1	0		-7	26	

The $\gcd(26, 11)$ is 1, which means that the multiplicative inverse of 11 exists. The extended Euclidean algorithm gives $t_1 = -7$. The multiplicative inverse is $(-7) \bmod 26 = 19$. In other words, 11 and 19 are multiplicative inverse in $\mathbb{Z}_{26}$. We can see that $(11 \times 19) \bmod 26 = 209 \bmod 26 = 1$.

```

 $r_1 \leftarrow n; r_2 \leftarrow b;$ 
 $t_1 \leftarrow 0; t_2 \leftarrow 1;$ 

```

```

while ( $r_2 > 0$ )
{
   $q \leftarrow r_1 / r_2;$ 

```

```

   $r \leftarrow r_1 - q \times r_2;$ 
   $r_1 \leftarrow r_2; r_2 \leftarrow r;$ 

```

```

   $t \leftarrow t_1 - q \times t_2;$ 
   $t_1 \leftarrow t_2; t_2 \leftarrow t;$ 

```

```

}
if ( $r_1 = 1$ ) then  $b^{-1} \leftarrow t_1$ 

```

b. Algorithm

Example 2.26

Find the multiplicative inverse of 23 in $\mathbf{Z}_{100}$.

Solution

We use a table similar to the one we used before with $r_1 = 100$ and $r_2 = 23$. We are interested only in the value of t .

q	r_1	r_2	r	t_1	t_2	t
4	100	23	8	0	1	-4
2	23	8	7	1	-4	19
1	8	7	1	-4	9	-13
7	7	1	0	9	-13	100
	1	0		-13	100	

The gcd (100, 23) is 1, which means the inverse of 23 exists. The extended Euclidean algorithm gives $t_1 = -13$. The inverse is $(-13) \bmod 100 = 87$. In other words, 13 and 87 are multiplicative inverses in $\mathbf{Z}_{100}$. We can see that $(23 \times 87) \bmod 100 = 2001 \bmod 100 = 1$.

Euler's Phi-Function

- Euler's Phi-Function Euler's phi-function, $\phi(n)$, which is sometimes called the **Euler's totient function** plays a very important role in cryptography

Example 9.7

What is the value of $\phi(13)$?

Solution

Because 13 is a prime, $\phi(13) = (13 - 1) = 12$.

Example 9.8

What is the value of $\phi(10)$?

Solution

We can use the third rule: $\phi(10) = \phi(2) \times \phi(5) = 1 \times 4 = 4$, because 2 and 5 are primes.

Example 9.9

What is the value of $\phi(240)$?

Solution

We can write $240 = 2^4 \times 3^1 \times 5^1$. Then

$$\phi(240) = (2^4 - 2^3) \times (3^1 - 3^0) \times (5^1 - 5^0) = 64$$

Example 9.10

Can we say that $\phi(49) = \phi(7) \times \phi(7) = 6 \times 6 = 36$?

Solution

No. The third rule applies when m and n are relatively prime. Here $49 = 7^2$. We need to use the fourth rule: $\phi(49) = 7^2 - 7^1 = 42$.

Example 9.11

What is the number of elements in $\mathbf{Z}_{14}^*$?

Solution

The answer is $\phi(14) = \phi(7) \times \phi(2) = 6 \times 1 = 6$. The members are 1, 3, 5, 9, 11, and 13.

Multiplicative Inverses Euler's theorem can be used to find multiplicative inverses modulo a prime; Euler's theorem can be used to find multiplicative inverses modulo a composite. If n and a are coprime, then $a^{-1} \bmod n = a^{\phi(n)-1} \bmod n$.

This can be easily proved if we multiply both sides of the equality by a :

$$a \times a^{-1} \bmod n = a \times a^{\phi(n)-1} \bmod n = a^{\phi(n)} \bmod n = 1 \bmod n$$

Example 9.17

The answers to multiplicative inverses modulo a composite can be found without using the extended Euclidean algorithm if we know the factorization of the composite:

- a. $8^{-1} \bmod 77 = 8^{\phi(77)-1} \bmod 77 = 8^{59} \bmod 77 = 29 \bmod 77$
- b. $7^{-1} \bmod 15 = 7^{\phi(15)-1} \bmod 15 = 7^7 \bmod 15 = 13 \bmod 15$
- c. $60^{-1} \bmod 187 = 60^{\phi(187)-1} \bmod 187 = 60^{159} \bmod 187 = 53 \bmod 187$
- d. $71^{-1} \bmod 100 = 71^{\phi(100)-1} \bmod 100 = 71^{39} \bmod 100 = 31 \bmod 100$

Let us find the lowest primitive root of 761:

- $s = \phi(761) = 760 = 2^3 \times 5 \times 19$
- the powers to test are: $760/2 = 380$, $760/5 = 152$ and $760/19 = 40$ (just 3 instead of testing all of them)
- test 2:
 - $2^{380} \equiv 1 \pmod{761}$ oops
- test 3:
 - $3^{380} \equiv -1 \pmod{761}$ OK
 - $3^{152} \equiv 1 \pmod{761}$ oops
- test 5 (skip 4 because it is 2^2):
 - $5^{380} \equiv 1 \pmod{761}$ oops
- test 6:
 - $6^{380} \equiv -1 \pmod{761}$ OK
 - $6^{152} \equiv 67 \pmod{761}$ OK
 - $6^{40} \equiv -263 \pmod{761}$ hooray!

So, the least primitive root of 761 is 6.

Q.: Find the Primitive Root of 7

① $N=7$

② $\phi(7) = \phi(7^1)$
 $= 7^1 - 7^0 = 6$

③ $\phi(N) = 6 = 3 \times 2$
 $f_1 = 3$ $f_2 = 2$

④ Test values:
 $T_1 = \frac{\phi(N)}{f_1}$ $T_2 = \frac{\phi(N)}{f_2}$
 $T_1 = \frac{6}{3} = 2$ $T_2 = \frac{6}{2} = 3$

$T_1 = 2$ $T_2 = 3$

1

Possible P.R of 7 $1 < PR < N$

2	3	4	5	6	
--------------	---	--------------	---	--------------	--

$a =$ But GCD with 7

check if $a^{T_1} \bmod N \neq 1$ $\rightarrow a$ is Primitive Root.
 $a^{T_2} \bmod N \neq 1$

check 2:
 $2^2 \bmod 7 \rightarrow 4$
 $2^3 \bmod 7 \rightarrow 1$

2 is not a P.R.

check 4:
 $4^2 \bmod 7 \rightarrow 2$
 $4^3 \bmod 7 \rightarrow 1$

2

Asymmetric Key Encryption example

Public key cryptography has become an essential means of providing confidentiality, especially through its need of key distribution, where users seeking private connection exchange encryption keys.

Public key Cryptosystem – Asymmetric algorithms depends on one key for encryption and a distinct but related key for decryption. These algorithms have the following characteristics which are as follows –

- It is computationally infeasible to decide the decryption key given only information of the cryptographic algorithm and the encryption key.
- There are two related keys such as one can be used for encryption, with the other used for decryption.

New Idea:

Use the good old mailbox principle:

Everyone can drop a letter

But Only the owner has the correct key to open the box

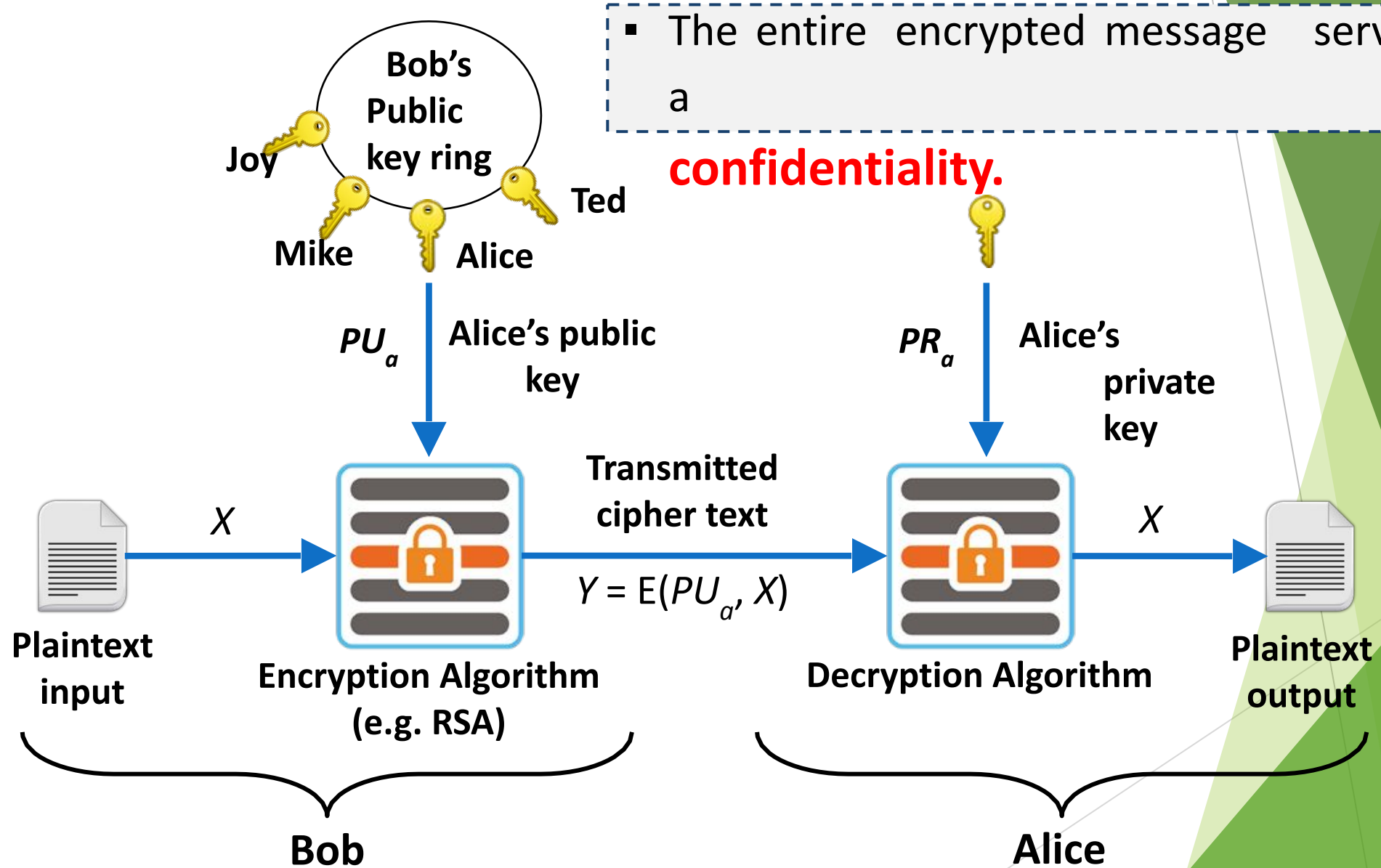


Public Key



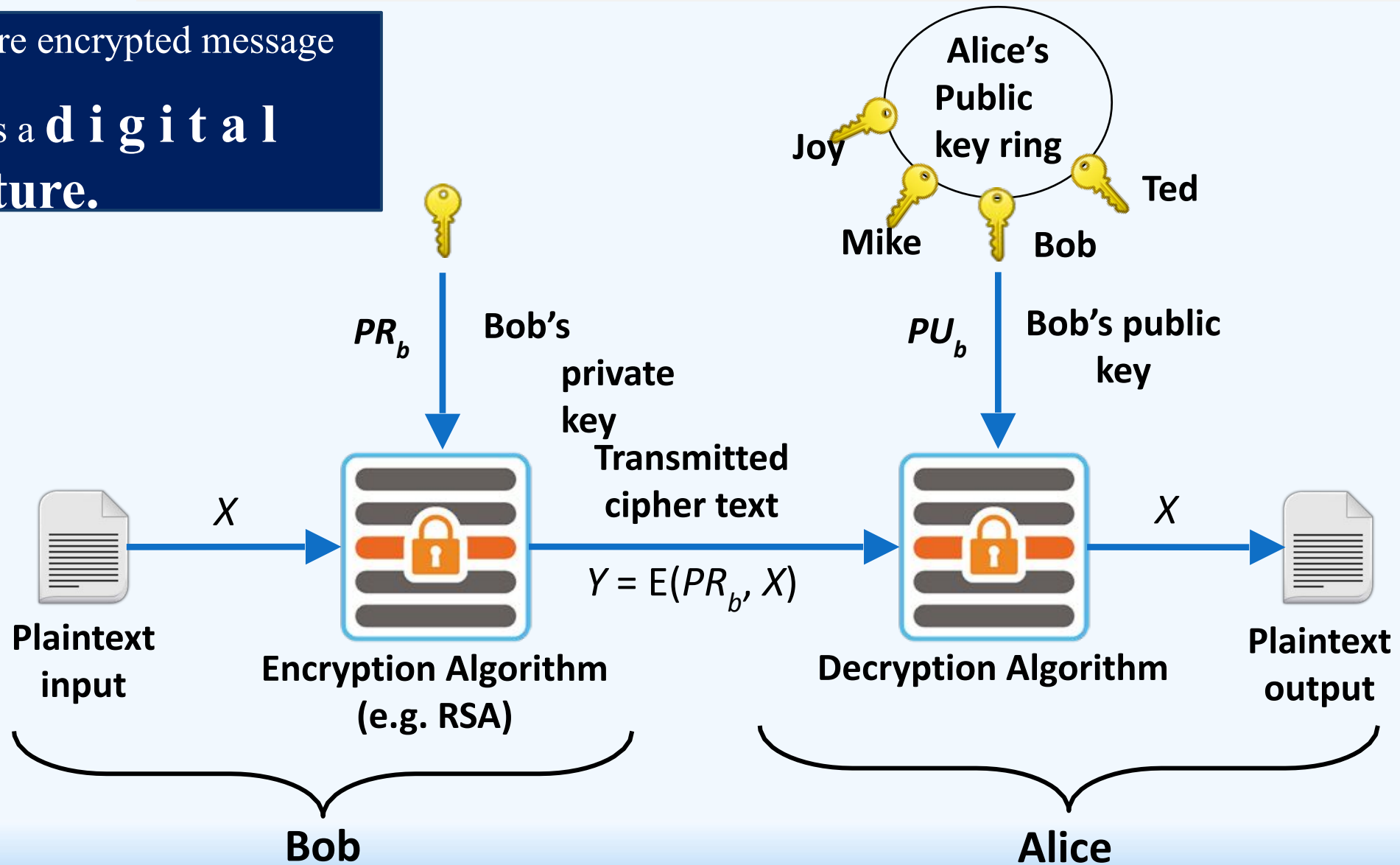
Private Key

Asymmetric Key Encryption with Public Key

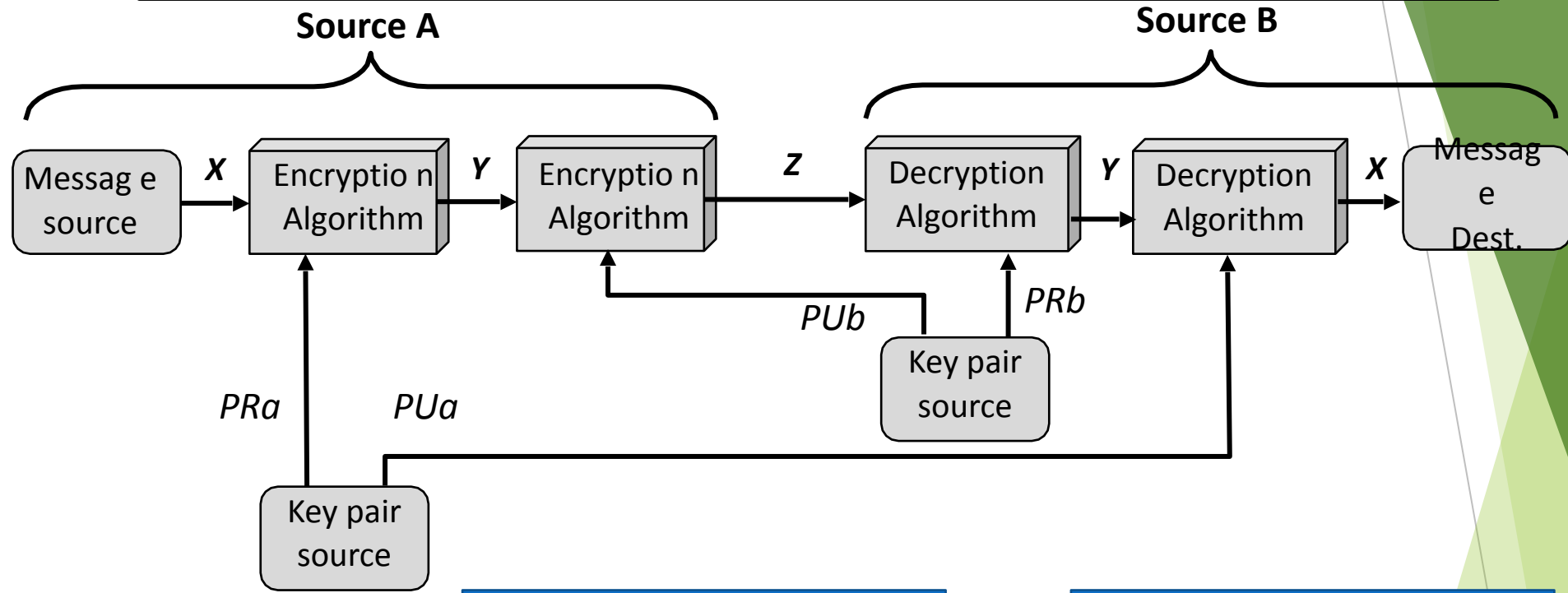


Asymmetric Key Encryption with Private Key

- The entire encrypted message serves as a **digital signature**.



Authentication and Confidentiality



$$Z = E(PU_b, E(PR_a, X))$$

$$X = D(PU_a, D(PR_b, Z))$$

Authentication and Confidentiality using Public key cryptography.

It also features digital signatures which enable users to sign keys to check their identities.

Applications for Public-Key Cryptosystems

- **Encryption/decryption:** The sender encrypts a message with the recipient's public key.
- **Digital signature:** The sender “signs” a message with its private key. Signing is achieved by a cryptographic algorithm applied to the message or to a small block of data that is a function of the message.
- **Key exchange:** Two sides cooperate to exchange a session key. Several different approaches are possible, involving the private key(s) of one or both parties.

Examples of Asymmetric Key Algorithms

RSA (Rivest-Shamir-Adleman)

Elliptic curve cryptosystem (ECC)

Diffie-Hellman

El Gamal

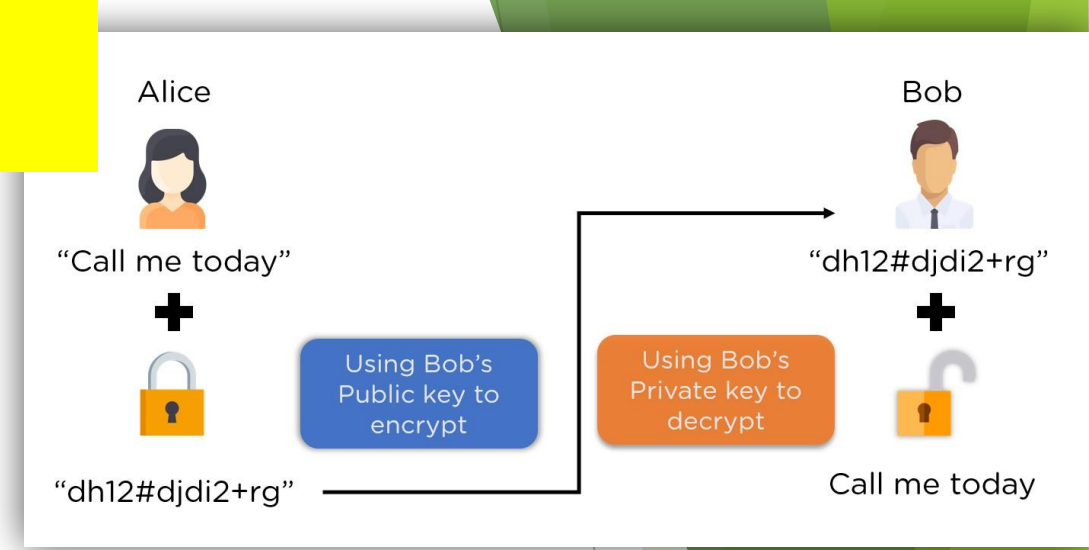
Digital Signature Algorithm (DSA)

Merkle-Hellman Knapsack

RSA Algorithm

- RSA is a block cipher in which the **Plaintext and Ciphertext** are represented as **integers between 0 and $n-1$** for some value of **n** .
- Large messages can be broken up into a number of blocks.
- Each block would then be represented by an integer.
Step-1: Generate Public key and Private key.
Step-2: Encrypt message using Public key.
Step-3: Decrypt message using Private key.

RSA (Rivest–Shamir–Adleman) is a public-key cryptosystem, one of the oldest widely used for secure data transmission. The initialism "RSA" comes from the surnames of Ron Rivest, Adi Shamir and Leonard Adleman, who publicly described the algorithm in 1977.



The idea! The idea of RSA is based on the fact that it is difficult to factorize a large integer. The public key consists of two numbers where one number is a multiplication of two large prime numbers. And private key is also derived from the same two prime numbers. So if somebody can factorize the large number, the private key is compromised. Therefore encryption strength totally lies on the key size and if we double or triple the key size, the strength of encryption increases exponentially. RSA keys can be typically 1024 or 2048 bits long, but experts believe that 1024-bit keys could be broken in the near future. But till now it seems to be an infeasible task.

Step-1: Generate Public key and Private key

- Select two large prime numbers: p and q
- Calculate modulus : $n = p * q$
- Calculate Euler's totient function : $\varphi(n) = (p-1) * (q-1)$
- Select e such that e is relatively prime to $\varphi(n)$ and $1 < e < \varphi(n)$

Two numbers are relatively prime if they have no common factors other than 1 or $(\gcd(\varphi(n), e)=1)$.

- Determine d such that $d * e \equiv 1 \pmod{\varphi(n)}$
- Publickey : $PU = \{ e, n \}$
- Privatekey : $PR = \{ d, n \}$

Step-2 and 3: Encryption and Decryption

Step-2: Encryption using Public Key:

Plaintext: M

Ciphertext: $C = M^e \bmod n$

Step-3: Decryption using Private Key:

Ciphertext: C

Plaintext: $M = C^d \bmod n$

Step-1: Generate Public key and Private key

- Select two large prime numbers: $p = 3$ and $q = 11$
- Calculate modulus : $n = p * q = 3 * 11 = 33$
- Calculate Euler's totient function : $\phi(n) = (p-1) * (q-1)$
$$\phi(n) = (3 - 1) * (11 - 1) = 20$$
- Select e such that e is relatively prime to $\phi(n)$ and $1 < e < \phi(n)$
- We have several choices for e : 3, 7, 11, 13, 17, 19 Let's take $e = 7$
- Determine d such that $d * e \equiv 1 \pmod{\phi(n)}$
- $? * 7 \equiv 1 \pmod{20}, 3 * 7 \equiv 1 \pmod{20}$
- $? * 7 \pmod{20} \equiv 1, 3 * 7 \pmod{20} \equiv 1$
- Public key : $PU = \{ e, n \}, PU = \{ 7, 33 \}$
- Private key : $PR = \{ d, n \}, PR = \{ 3, 33 \}$

- This is equivalent to finding d which satisfies $de = 1 + j.\phi(n)$ where j is any integer.
- We can rewrite this as $d = (1 + j.\phi(n)) / e$

Step-2 : Encrypt Message

- Encryption using Public Key:

Ciphertext

Input
Message

Public key

$$C = M^e \bmod n$$

$$PU = \{ e, n \}, PU = \{ 7, 33 \}$$

For message $M = 14$

$$C = 14^7 \bmod 33$$

$$C = [(14^1 \bmod 33) \times (14^2 \bmod 33) \times (14^4 \bmod 33)] \bmod 33$$

$$C = (14 \times 31 \times 4) \bmod 33 = 1736 \bmod 33$$

$$C = 20$$

Step-3 : Decrypt Message

- Decryption using Private Key:

Plaintext
Message

Cipher
Message

Private key

$$M = C^d \bmod n$$

$$PR = \{ d, n \}, PR = \{ 3, 33 \}$$

For Ciphertext $C = 20$

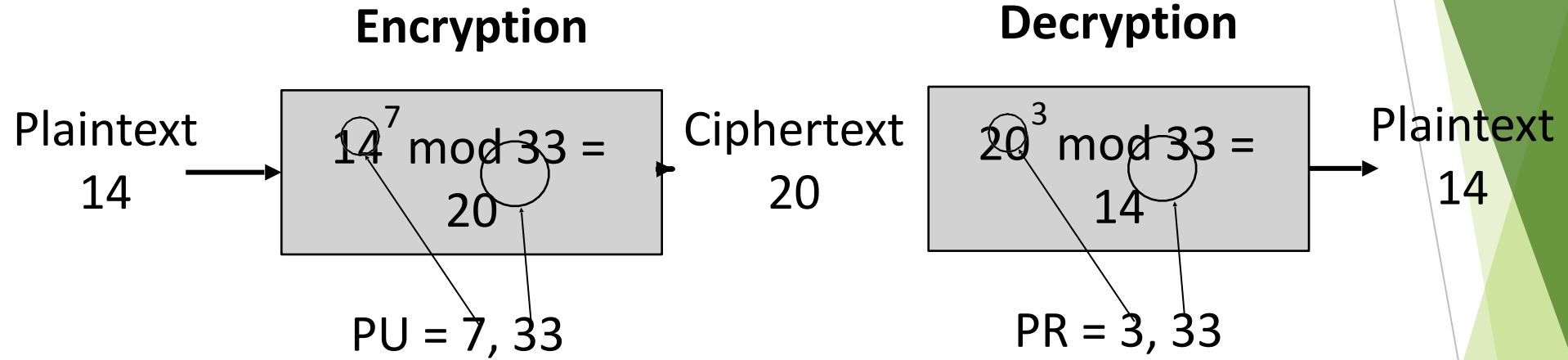
$$M = 20^3 \bmod 33$$

$$M = [(20^1 \bmod 33) \times (20^2 \bmod 33)] \bmod 33$$

$$M = (20 \times 4) \bmod 33 = 80 \bmod 33$$

$$M = 14$$

Example RSA Algorithm



RSA Example - 1

- Find n , $\phi(n)$, e , d for $p=7$ and $q=19$ then demonstrate encryption and decryption for $M = 6$

$$n = p * q = 7 * 19 = 133$$

$$\phi(n) = (p - 1) * (q - 1) = 108$$

Finding e relatively prime to 108

$$e = 2 \Rightarrow \text{GCD}(2, 108) = 2 \text{ (no)}$$

$$e = 3 \Rightarrow \text{GCD}(3, 108) = 3 \text{ (no)}$$

$$e = 5 \Rightarrow \text{GCD}(5, 108) = 1 \text{ (Yes)}$$

- Finding d such that $(d * e) \bmod \phi(n) = 1$
- Or we need to find multiplicative inverse of 5 under mod of 108

Public key :

$$\text{PU} = \{ e, n \} = \{ 5, 133 \}$$

Private key :

$$\text{PR} = \{ d, n \} = \{ 65, 133 \}$$

RSA Example – cont...

- Encryption:

$$C = M^e \bmod n$$

$$PU = \{ e, n \}, PU = \{ 5, 133 \}$$

For message $M = 6$

$$C = 6^5 \bmod 133$$

$$C = 7776 \bmod 133$$

$$C = 62$$

- Decryption:

$$M = C^d \bmod n$$

$$PR = \{ d, n \}, PU = \{ 65, 133 \}$$

For $C = 62$

$$M = 62^{65} \bmod 133$$

$$M = 2666 \bmod 133$$

$$M = 6$$

RSA Example - 2

- P and Q are two prime numbers. $P=7$, and $Q=17$. Take public key $E=5$. If plain text value is 10, then what will be cipher text value according to RSA algorithm?
- $n = 119$
- $\phi(n) = 96$
- $e = 5$
- $d = 77$
- $PU = \{ 5, 119 \}$
- $PR = \{77, 119\}$
- $C = 10^5 \bmod 119 \Rightarrow C = 40$

RSA Example - 3

- P and Q are two prime numbers. $P=17$, and $Q=11$. If plain text value is 88, then what will be cipher text value according to RSA algorithm?
- $n = 187$
- $\phi(n) = 160$
- $e = 7$
- $d = 23$
- $PU = \{ 7, 187 \}$
- $PR = \{23, 187\}$
- $C = 88^7 \bmod 187 \Rightarrow C = 11$

Diffie-Hellman key Exchange

- The purpose of the **Diffie-Hellman** algorithm is to enable two users to securely exchange a key that can be used for subsequent encryption of message.
- The effectiveness of an algorithm is depends on the difficulty of computing **discrete logarithms**.
- User A and User B agree on two large prime numbers q and α . User A and User B can use insecure channel to agree on them.
- User A selects a random integer $X_A < q$ and calculates Y_A .
- User B selects a random integer $X_B < q$ and calculates Y_B .

Primitive root

- Let p be a prime number.
- Then a is a primitive root for p , if the powers of a modulo p generates all integers from 1 to $p - 1$ in some permutation.

$$a \bmod p, a^2 \bmod p, \dots, a^{p-1} \bmod p$$

- Example: $p = 7$ then primitive root is 3 because powers of 3 mod 7 generates all the integers from 1 to 6.

$$3^1 = 3 \equiv 3 \pmod{7}$$

$$3^2 = 9 \equiv 2 \pmod{7}$$

$$3^3 = 27 \equiv 6 \pmod{7}$$

$$3^4 = 81 \equiv 4 \pmod{7}$$

$$3^5 = 243 \equiv 5 \pmod{7}$$

$$3^6 = 729 \equiv 1 \pmod{7}$$

Diffie-Hellman Key Exchange Illustration

Alice **Bob**

Alice and Bob share a prime number q and an integer a , such that $a < q$ and a is a primitive root of q

Alice generates a private key X_A such that $X_A < q$

Alice calculates a public key

$$Y_A = a^{X_A} \bmod q$$

Alice receives Bob's public key Y_B in plaintext

Alice calculates shared secret key

$$K = (Y_B)^{X_A} \bmod q$$

Alice and Bob share a prime number q and an integer a , such that $a < q$ and a is a primitive root of q

Bob generates a private key X_B such that $X_B < q$

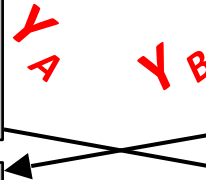
Bob calculates a public key

$$Y_B = a^{X_B} \bmod q$$

Bob receives Alice's public key Y_A in plaintext

Bob calculates shared secret key

$$K = (Y_A)^{X_B} \bmod q$$



Diffie-Hellman Key Exchange – Cont...

Global Public Elements

prime number

$\alpha < q$ and α is primitive root of q

User A Key Generation

Select private X_A $X_A < q$

Calculate public Y_A $Y_A = \alpha^{X_A} \bmod q$

User B Key Generation

Select private X_B $X_B < q$

Calculate public Y_B $Y_B = \alpha^{X_B} \bmod q$

Diffie-Hellman Key Exchange – Cont...

User A Key Generation

Select private X_A $X_A < q$

Calculate public Y_A $Y_A = \alpha^{X_A} \bmod q$

User B Key Generation

Select private X_B $X_B < q$

Calculate public Y_B $Y_B = \alpha^{X_B} \bmod q$

Calculation of Secret Key by User A

$K =$
 $(Y_B)^{X_A}$

Calculation of Secret Key by User b

$K =$
 $(Y_A)^{X_B}$

Diffie-Hellman Key Exchange – Cont...

User A Key Generation

Private X_A $X_A < q$, User

Public Y_A

$$Y_A = \alpha^{X_A} \bmod q$$

User B Key Generation

Private X_B $X_B < q$, Public Y_B

$$Y_B = \alpha^{X_B} \bmod q$$

Secret Key by User A : $K = (Y_B)^{X_A} \bmod q$

Secret Key by User B :

$$K_A = (Y_B)^{X_A} \bmod q$$

$$K = (\alpha^{X_B} \bmod q)^{X_A} \bmod q$$

$$K = (\alpha^{X_B})^{X_A} \bmod q$$

$$K = \alpha^{X_B X_A} \bmod q$$

$$K = (\alpha^{X_A})^{X_B} \bmod q$$

$$K = (\alpha^{X_A} \bmod q)^{X_B} \bmod q$$

$$K_B = (Y_A)^{X_B} \bmod q$$

Diffie-Hellman Key Exchange Example

- Alice and Bob agree on a prime number $q = 23$
- $\alpha = 5$ as primitive root of q .
- Alice selects a private integer $X_A = 6$
- Alice computes $Y_A = \alpha^{X_A} \bmod q \Rightarrow Y_A = 5^6 \bmod 23 = 8$
- Bob selects a private integer $X_B = 15$
- Bob computes $Y_B = \alpha^{X_B} \bmod q \Rightarrow Y_B = 5^{15} \bmod 23 = 19$
- Alice sends Y_A to Bob and Bob sends Y_B to Alice.
- Both compute the shared key $K = Y_B^{X_A} \bmod q = 19^6 \bmod 23 = 2$
- Bob computes key $K = (Y_A)^{X_B} \bmod q \Rightarrow K = (8)^{15} \bmod 23 = 2$

Diffie-Hellman Key Exchange Example

- Alice and Bob agree on a prime number $q = 19$
- $\alpha = 10$ as primitive root of q .
- Alice selects a private integer $X_A = 7$
- Alice computes $Y_A = \alpha^{X_A} \bmod q \Rightarrow Y_A = 10^7 \bmod 19 = 15$
- Bob selects a private integer $X_B = 8$
- Bob computes $Y_B = \alpha^{X_B} \bmod q \Rightarrow Y_B = 10^8 \bmod 19 = 17$
- Alice sends Y_A to Bob and Bob sends Y_B to Alice.
• Alice computes key $K = (Y_B)^{X_A} \bmod q \Rightarrow K = (17)^7 \bmod 19$
• $K = 5$
- Bob computes key $K = (Y_A)^{X_B} \bmod q \Rightarrow K = (15)^8 \bmod 19$
• $K = 5$

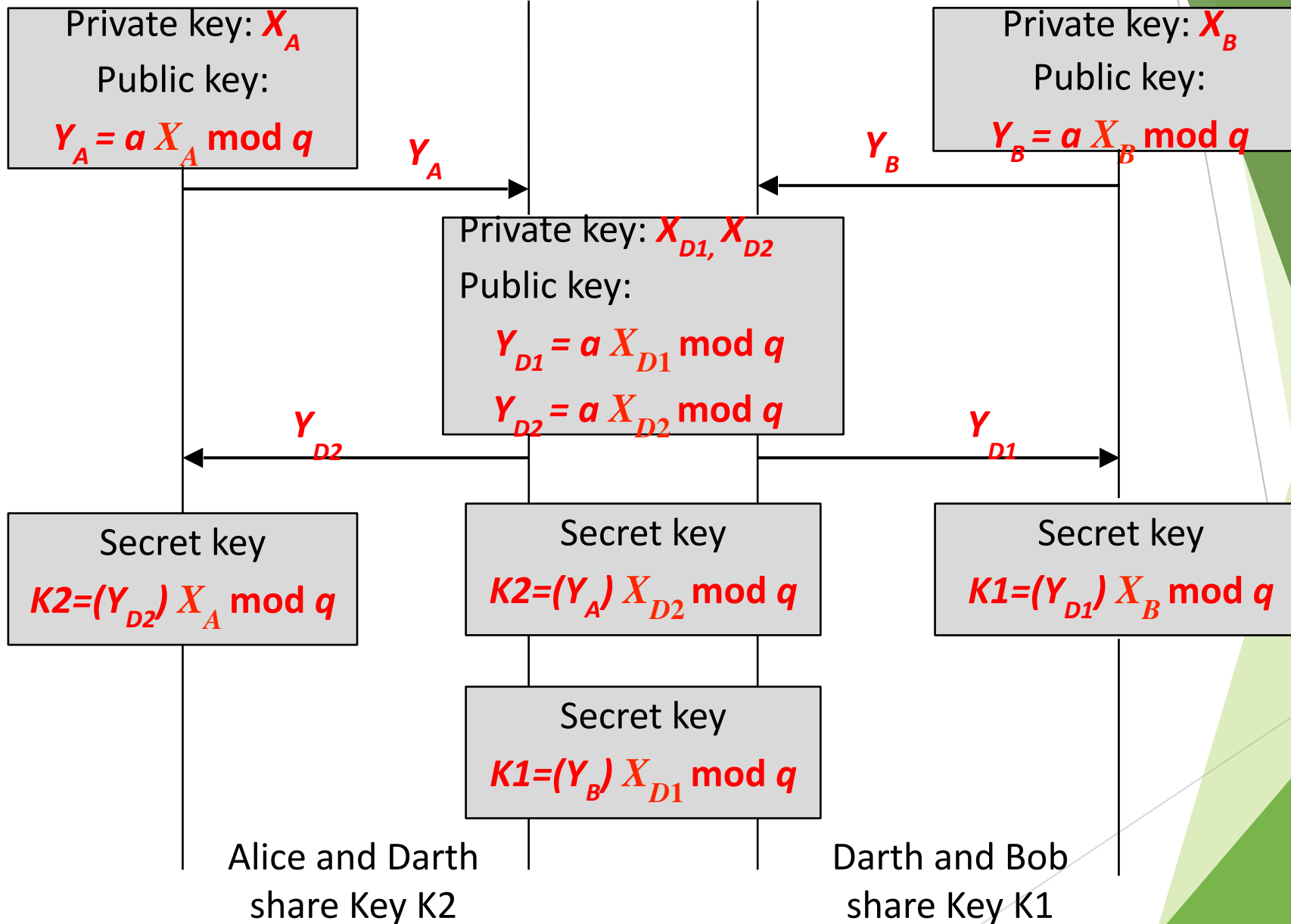
Man in the middle attack

- Suppose Alice and Bob wish to exchange keys, and Darth is the adversary.
1. Darth prepares for the attack by generating two random private keys X_{D1} and X_{D2} and then computes corresponding public keys Y_{D1} and Y_{D2} .
 2. Alice transmits Y_A to Bob.
 3. Darth intercepts Y_A and transmits Y_{D1} to Bob. Darth also calculates $K_2 = (Y_A) X_{D2} \bmod q$.
 4. Bob receives Y_{D1} and calculates $K_1 = (Y_{D1}) X_B \bmod q$.
 5. Bob transmits Y_B to Alice.
 6. Darth intercepts Y_B and transmits Y_{D2} to Alice. Darth calculates $K_1 = (Y_B) X_{D1} \bmod q$.
 7. Alice receives Y_{D2} and calculates $K_2 = (Y_{D2}) X_A \bmod q$.

Alice

Darth

Bob



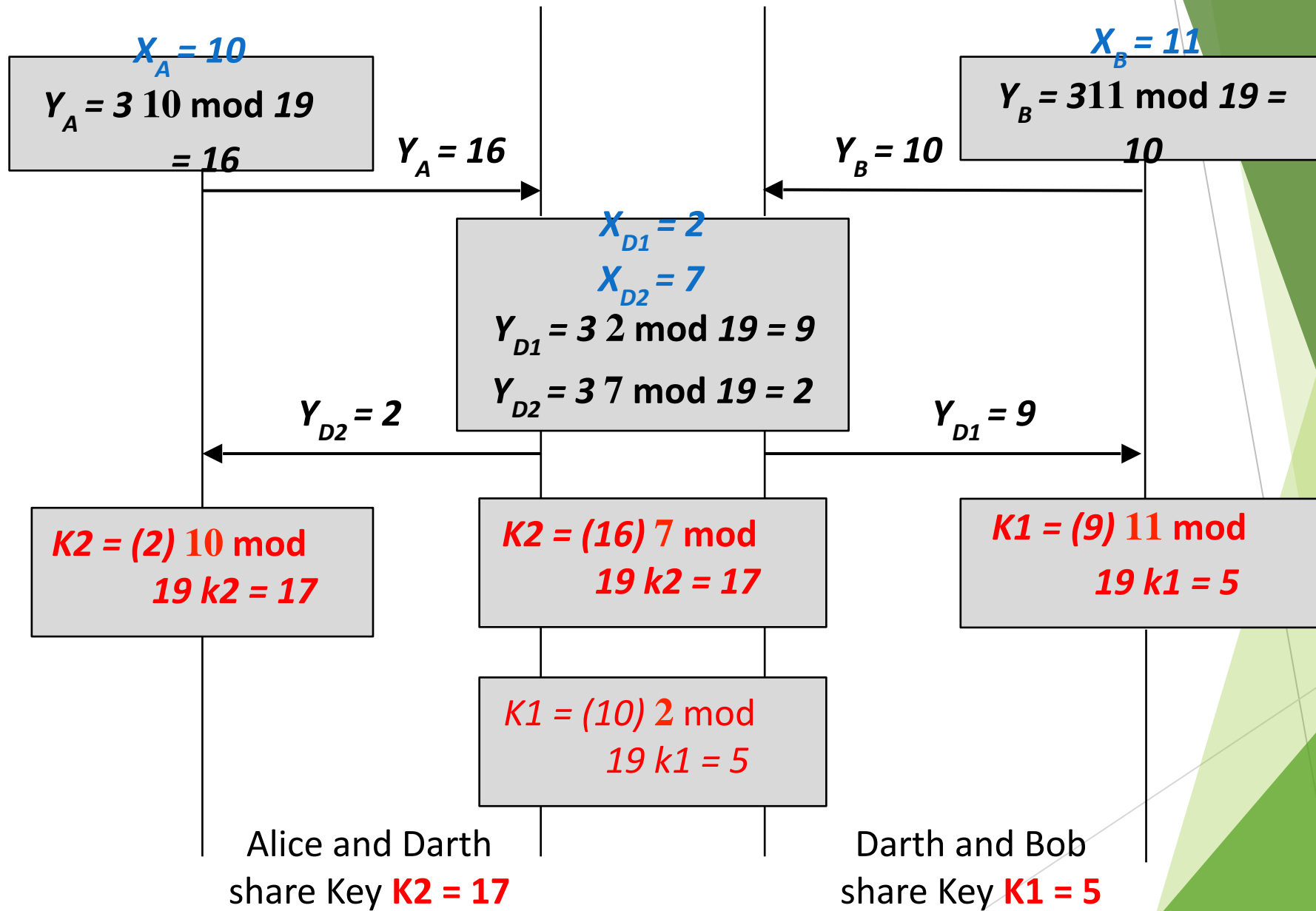
Alice

$q = 19$ and $a = 3$

Darth

$q = 19$ and $a = 3$

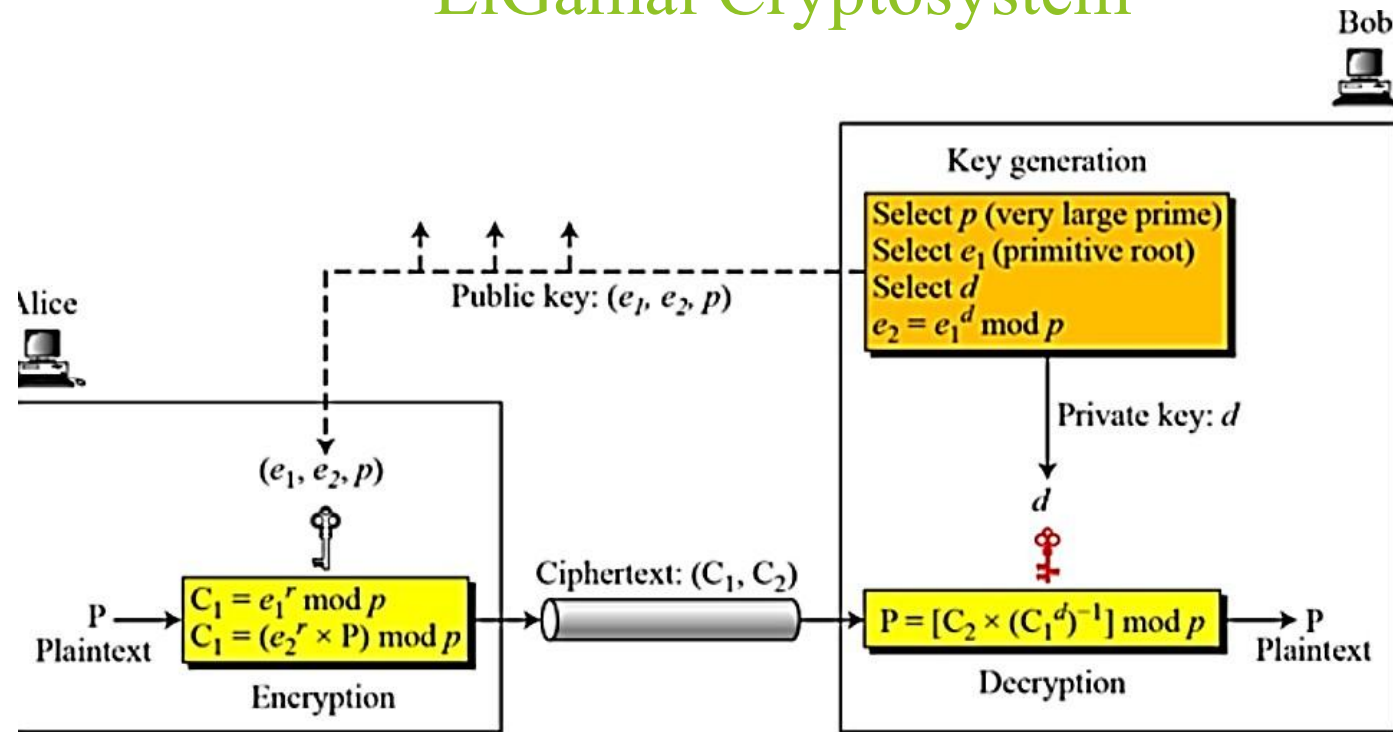
Bob



ElGamal Cryptosystem

- ❖ In 1976, Whiteld Diffie and Martin Hellman proposed a paper where they proposed the thought of public key cryptography in which two diverse yet scientifically related keys are used-a public key and a private key.
- ❖ A public key frame-work is constructed to the point that configuring of one key (the 'private key') is computationally infeasible from the other (the 'public key'), despite the fact that they are essentially related. Rather, both keys are produced secretly, as an interrelated pair.
- ❖ The ElGamal cryptosystem includes three major processes: **the key generation, the encryption, and the decryption.**
- ❖ In this whole process, Eve(third person) could not be able to know the secret key because its security is based on solving the Discrete Logarithm Algorithm. So, ElGamal PKC is one of many algorithms that utilizes randomization in the encryption process.
- ❖ The ElGamal Algorithm provides an alternative to the RSA for public key encryption.
- ❖ Security of the ElGamal algorithm depends on the (presumed) difficulty of computing discrete logs in a large prime modulus.
- ❖ The encryption process requires two modular exponentiations. ElGamal has the disadvantage that the cipher text is twice as long as the plaintext. It has the advantage the same plaintext gives a different cipher text (with near certainty) each time it is encrypted.

ElGamal Cryptosystem



- Let us consider a chance to think about that as a sender called Alice needs to send a private message to the recipient Bob, and a third individual called Eve tries to know this message. The ElGamal PKC procedure works as follows:
 - In the first step, Bob (Receiver) has to compute a public key and send it to the Alice (Sender).
 - After receiving the Bob's public key she does the Encryption, which comprises of computing the cipher text from the plaintext.
 - She sends this ciphertext to Bob.
 - Then Bob decrypts the cipher text to compute a plaintext using his private key.

ElGamal Cryptosystem

Algorithm 10.9 ElGamal key generation

ElGamal_Key_Generation

```
{
  Select a large prime  $p$ 
  Select  $d$  to be a member of the group  $\mathbf{G} = \langle \mathbf{Z}_p^*, \times \rangle$  such that  $1 \leq d \leq p - 2$ 
  Select  $e_1$  to be a primitive root in the group  $\mathbf{G} = \langle \mathbf{Z}_p^*, \times \rangle$ 
   $e_2 \leftarrow e_1^d \bmod p$ 
  Public_key  $\leftarrow (e_1, e_2, p)$            // To be announced publicly
  Private_key  $\leftarrow d$                    // To be kept secret
  return Public_key and Private_key
}
```

```
ElGamal_Encryption( $e_1, e_2, p, P$ )           //  $P$  is the plaintext
{
  Select a random integer  $r$  in the group  $\mathbf{G} = \langle \mathbf{Z}_p^*, \times \rangle$ 
   $C_1 \leftarrow e_1^r \bmod p$ 
   $C_2 \leftarrow (P \times e_2^r) \bmod p$            //  $C_1$  and  $C_2$  are the ciphertexts
  return  $C_1$  and  $C_2$ 
}
```

Decryption

Bob can use Algorithm 10.11 to decrypt the ciphertext message received.

Algorithm 10.11 ElGamal decryption

```
ElGamal_Decryption( $d, p, C_1, C_2$ )           //  $C_1$  and  $C_2$  are the ciphertexts
{
   $P \leftarrow [C_2 (C_1^d)^{-1}] \bmod p$        //  $P$  is the plaintext
  return  $P$ 
}
```

The bit-operation complexity of encryption or decryption in ElGamal cryptosystem is polynomial.

Example 10.10

Here is a trivial example. Bob chooses 11 as p . He then chooses $e_1 = 2$. Note that 2 is a primitive root in $\mathbf{Z}_{11}^*$ (see Appendix J). Bob then chooses $d = 3$ and calculates $e_2 = e_1^d = 8$. So the public keys are (2, 8, 11) and the private key is 3. Alice chooses $r = 4$ and calculates C_1 and C_2 for the plaintext 7.

Plaintext: 7

$$C_1 = e_1^r \bmod 11 = 16 \bmod 11 = 5 \bmod 11$$

$$C_2 = (P \times e_2^r) \bmod 11 = (7 \times 4096) \bmod 11 = 6 \bmod 11$$

Ciphertext: (5, 6)

Bob receives the ciphertexts (5 and 6) and calculates the plaintext.

$$[C_2 \times (C_1^d)^{-1}] \bmod 11 = 6 \times (5^3)^{-1} \bmod 11 = 6 \times 3 \bmod 11 = 7 \bmod 11$$

Plaintext: 7

Example 10.11

Instead of using $P = [C_2 \times (C_1^d)^{-1}] \bmod p$ for decryption, we can avoid the calculation of multiplicative inverse and use $P = [C_2 \times C_1^{p-1-d}] \bmod p$ (see Fermat's little theorem in Chapter 9). In Example 10.10, we can calculate $P = [6 \times 5^{11-1-3}] \bmod 11 = 7 \bmod 11$.